

EC327: Introduction to Software Engineering

C++ for People Who Already Code

Jeffrey Carruthers

June 2026

Table of contents

Preface	1
Who this book is for	1
How the book is organized	1
Conventions used in this book	2
On using AI while reading this book	2
Acknowledgments	2
I Week 1 — From Python to C++	3
1 Course intro, first C++ file, Linux basics	7
1.1 What this course is	7
1.2 Lab 0: the ungraded standup	7
1.3 Anatomy of a C++ file	8
1.4 The compile / link / run cycle	9
1.5 A first program you can extend	10
1.6 Linux basics, in 90 seconds	10
1.7 What to do before the next session	11
1.8 What you should understand by the end of this chapter	11
2 Types, operators, selection, and the compile cycle	13
2.1 The plan: quadratic0.cpp through quadratic3.cpp	13
2.2 Stage 0: variables, input, and output	14
2.3 Stage 1: arithmetic and <code><cmath></code>	15
2.3.1 What goes wrong	17
2.4 Stage 2: selection — the <code>if</code> statement	17
2.5 Stage 3: three-way selection and the <code>and</code> keyword	18
2.6 A second worked example: a grade calculator	20
2.6.1 Stage 1: a sequence of independent <code>ifs</code>	21
2.6.2 Stage 2: a single <code>if</code> / <code>else if</code> / <code>else chain</code>	21
2.6.3 Where the progression goes next	22
2.7 The compile/link cycle, revisited	23
2.8 Interpreted vs compiled	23
2.9 Whitespace is not significant	24
2.10 Practice: ways to break each stage	25
2.11 What you should understand by the end of this chapter	25
3 <code>std::vector</code>, range-for, and your first struct	27
3.1 Stage 3: parallel arrays and the <code>while</code> loop	27
3.2 Why we do not write C-style arrays in 2026	29

3.3	Stage 4: <code>std::vector</code> replaces the C-style array	29
3.4	Iterating a vector — three idioms you should know	30
3.4.1	Idiom 1: range-based <code>for</code> (<i>the default you reach for first</i>)	31
3.4.2	Idiom 2: indexed <code>for</code> (<i>when the position matters</i>)	31
3.4.3	Idiom 3: iterator-based (<i>seen everywhere, written rarely</i>)	31
3.5	Your first <code>struct</code> : putting fields together	32
3.6	Stage 5: <code>vector<Grade></code> + range- <code>for</code> + early <code>break</code>	33
3.7	Structured bindings: peeking inside a row	34
3.8	Two-dimensional structures	35
3.8.1	Option 1: <code>vector<vector<T>></code> (<i>the flexible default</i>)	35
3.8.2	Option 2: <code>array<array<T, N>, M></code> (<i>when the size is fixed and small</i>)	35
3.8.3	Option 3: flat row-major (<i>when performance is the brief</i>)	35
3.9	What changes in stages 6 through 9 (a preview)	36
3.10	Practice: try to break each stage	37
3.11	What you should understand by the end of this chapter	37
II	Week 2 — Functions, recursion, classes	39
4	Functions, headers, and recursion	43
4.1	From Python <code>def</code> to a typed C++ function	43
4.2	The signature as a contract	44
4.3	Passing arguments: value vs <code>const&</code> vs reference	45
4.3.1	Pass-by-value — for small things	45
4.3.2	Pass-by- <code>const&</code> — for big things you only read	45
4.3.3	Pass-by-non- <code>const</code> reference — almost never	45
4.3.4	The decision, as a tree	46
4.4	Returning values	46
4.4.1	Return by value is the default	46
4.4.2	Two return values? A <code>struct</code> or a <code>std::tuple</code>	47
4.4.3	<code>void</code> returns exist for a side effect	47
4.4.4	“I might not have an answer” — <code>std::optional</code>	48
4.5	Pure vs effectful functions	48
4.6	Globals: don’t (with one exception)	48
4.7	A gallery of good and bad functions	50
4.7.1	Good	50
4.7.2	Bad	50
4.7.3	The loudest smell: “and” in the name	50
4.8	The library-check reflex	51
4.9	Recursion	52
4.9.1	Fibonacci and the cost of recursion	53
4.9.2	Recursion and iteration are duals	54
4.10	Splitting across files: headers and translation units	55
4.10.1	<code>Point.h</code> — the declaration (the <i>what</i>)	55
4.10.2	<code>Point.cpp</code> — the definition (the <i>how</i>)	55
4.10.3	<code>main.cpp</code> — the caller	56
4.10.4	Building it: three steps, by hand	56
4.11	Overloading and default arguments, briefly	57

4.12	Signaling errors: three tools	58
4.13	The shared-parameter smell — a bridge to classes	58
4.14	What you should understand by the end of this chapter	60
5	Classes: encapsulation, RAII, and earning a class’s existence	61
5.1	The smell that motivates a class	61
5.2	The one-sentence difference between <code>struct</code> and <code>class</code>	62
5.2.1	How this maps to Python — and where it stops mapping	64
5.3	Constructors: the only way in	64
5.3.1	Establishing an invariant in the constructor	65
5.4	Member functions and the implicit <code>this</code>	66
5.5	<code>const</code> member functions and const-correctness	67
5.5.1	Why const-correctness is not optional	67
5.6	RAII: acquire in the constructor, release in the destructor	68
5.6.1	The C answer, and why it is a trap	69
5.6.2	The destructor	69
5.6.3	RAII, the central idea	69
5.6.4	The destructor cascade on scope exit	71
5.6.5	You have been using RAII since week 1	72
5.7	The Rule of Zero	72
5.8	Invariants: the rule a class keeps	73
5.9	What makes a class earn its existence	74
5.10	AI touchpoint — “does this class earn its existence?”	75
5.11	What you should understand by the end of this chapter	76
6	Loop hygiene, “name your loop”, and STL algorithms	77
6.1	Part 1: loop hygiene — the spine	77
6.1.1	Invariants: the sentence that stays true	77
6.1.2	What goes inside the loop, and what goes outside	78
6.1.3	Init and exit conditions: the two ends of the loop	80
6.1.4	DRY the loop body; extract a helper when it grows	80
6.2	Part 2: name your loop	81
6.3	Part 3: lambdas — the customization point	83
6.4	Part 4: the algorithms, by shape	84
6.4.1	<code>std::accumulate</code> — fold a range into one value (<code><numeric></code>)	84
6.4.2	<code>std::max_element</code> — find the largest (<code><algorithm></code>)	85
6.4.3	<code>std::count_if</code> — count what matches (<code><algorithm></code>)	86
6.4.4	<code>std::find</code> and <code>std::find_if</code> — the first match (<code><algorithm></code>)	86
6.4.5	<code>std::sort</code> — put them in order (<code><algorithm></code>)	86
6.5	Part 5: the answer to Part 1’s questions	87
6.6	Build to learn, use to ship	88
6.7	AI touchpoint — “what’s the STL function that does this?”	89
6.8	What you should understand by the end of this chapter	89

III Week 3 — What’s underneath: the layered machine and memory model	91
7 The layered machine I: levels, ISA, and assembly	95
7.1 Six levels, top to bottom	95
7.2 The toy machine: a CPU you can see all of	96
7.2.1 Program one: <code>sum</code>	97
7.2.2 Program two: <code>max</code> — flags and a branch	97
7.2.3 Program three: <code>factorial</code> , two ways	98
7.3 The same program, on a real machine	99
7.3.1 What the optimizer does	101
7.4 Two machines, one model: x86-64 vs ARM64	102
7.4.1 The same <code>add</code> , two dialects	102
7.5 AI touchpoint — have an LLM explain a chunk of <code>-O2</code> assembly	103
7.6 What this sets up	104
7.7 What you should understand by the end of this chapter	104
8 The layered machine II: stack, heap, and the lifetime question	105
8.1 Python hid this from you, and that was the point	105
8.2 One memory, two ends	106
8.3 The stack: automatic, LIFO, bounded	106
8.4 Stack overflow, on purpose	107
8.5 What a <code>std::vector</code> actually allocates	108
8.6 Seeing both regions in GDB	110
8.6.1 The stack growing: <code>bt full</code> on recursion	110
8.6.2 Raw bytes: <code>x/8xg</code>	111
8.6.3 The vector picture, made real	111
8.7 Addresses are values	112
8.8 The lifetime question	113
8.9 AI touchpoint — narrate a debugger session	114
8.10 What you should understand by the end of this chapter	114
9 Raw pointers, references, and the lifetime problem	117
9.1 Python has no pointers — but it has the harder half already	117
9.2 References: a second name, bound once	118
9.3 Pointers: <code>&</code> and <code>*</code> are inverses	119
9.4 Pointer vs. reference: nullable, re-seatable, arithmetic	120
9.5 <code>nullptr</code> and the billion-dollar mistake	121
9.6 Raw arrays and decay: the <code>sizeof</code> lie	122
9.6.1 Why <code>std::vector</code> exists, in one sentence	123
9.7 The lifetime problem	124
9.7.1 <code>new</code> and <code>delete</code> , shown exactly once	124
9.7.2 Three failure modes raw pointers cannot prevent	125
9.7.3 The question raw pointers cannot answer	126
9.8 Watching undefined behavior fire: <code>AddressSanitizer</code>	126
9.9 What you should understand by the end of this chapter	127

10 Modern pointers: <code>unique_ptr</code>, <code>shared_ptr</code>, and <code>move</code>	129
10.1 <code>std::unique_ptr</code> — the default owning pointer	130
10.1.1 Move-only: there is exactly one owner	131
10.1.2 <code>unique_ptr</code> kills all three lifetime bugs at once	132
10.2 <code>std::shared_ptr</code> — when ownership is genuinely shared	133
10.3 <code>std::weak_ptr</code> — observe without owning	134
10.3.1 Which owning pointer? — the decision	135
10.4 Move semantics, conceptually	136
10.4.1 Copy vs. move — the <code>std::vector</code> cost model	136
10.4.2 <code>std::move</code> is a cast — and a promise	136
10.5 RAII, generalized	138
10.6 Looking ahead: this is the backbone of the next two chapters	138
10.7 What you should understand by the end of this chapter	139
 IV Week 4 — Linked structures, modular design, BST climax	 141
11 Linked lists, maps, and table-driven design	145
11.1 A self-referential <code>struct</code>	145
11.2 Growing the list: <code>push_front</code> moves ownership	147
11.3 The chain frees itself	147
11.4 Recursion mirrors the structure	148
11.4.1 The iterative twin	150
11.5 Associative containers: lookup by key, not position	151
11.6 The frequency-count idiom	152
11.7 Four ways to access — and they differ on a <i>missing</i> key	153
11.8 Table-driven design: when an <code>if/else</code> chain wants to be a <code>map</code>	154
11.9 The container decision tree	155
11.10 AI touchpoint — review the table-driven refactor	156
11.11 What you should understand by the end of this chapter	157
 12 File I/O and the three I/O dialects	 159
12.1 A stream is a typed conduit	159
12.2 The same line, three ways	160
12.2.1 Dialect 1: <code>printf</code> — the C heritage	160
12.2.2 Dialect 2: <code>std::cout <<</code> — idiomatic, type-safe, verbose	161
12.2.3 Dialect 3: <code>std::format</code> — the C++20 default	161
12.3 What each dialect costs you	162
12.4 The footgun, named: <code>printf</code> cannot check you	163
12.5 The format mini-language	164
12.6 Files are just streams pointed at disk	165
12.6.1 Two things that bite Python programmers	166
12.7 Reading: <code>getline</code> vs. <code>>></code> , and the loop condition	166
12.8 Make your own class streamable: overload <code>operator<<</code>	168
12.9 Why streams earn their keep: user-extensibility	169
12.10 Putting it together: read → compute → write a report	170
12.11 What you should understand by the end of this chapter	172

13 Problem decomposition, namespaces, and modular design	173
13.1 Outline	173
14 A tiny BST: data structures as a design exercise	175
14.1 Outline	175
 V Week 5 — Generics, complexity, tooling deep-dive	 177
15 Templates and concepts	181
15.1 Outline	181
16 Algorithmic complexity and when to optimize	183
16.1 Outline	183
17 Skill block: docs, linters, formatters, sanitizers, unit testing	185
17.1 Outline	185
18 Skill block: debugger, profiler, optimization-as-practice	187
18.1 Outline	187
 VI Week 6 — AI tools, project ramp, capstone	 189
19 Buffer day; function pointers, std::function, callbacks	193
19.1 Outline	193
20 AI tools — retrospective and discipline	195
20.1 Outline	195
21 Code review as a practice	197
21.1 Outline	197
22 Project demos, wrap-up, what to learn next	199
22.1 Outline	199
 Appendices	 201
A Build systems: g++, make, and cmake	201
A.1 Outline	201
B Sanitizers and clang-tidy workflow	203
B.1 Outline	203
C GDB reference	205
C.1 Outline	205
D Git and the GitHub team workflow	207
D.1 Outline	207

E	Unit testing in C++ with Catch2 / doctest	209
E.1	Outline	209
F	Common errors	211
F.1	Outline	211
G	C++ language summary	213
G.1	Outline	213

Preface

This book is the companion text for **EC327, Introduction to Software Engineering**, taught in the first six-week summer session at Boston University by Jeffrey Carruthers. It is built directly from the lectures, labs, and in-class demos of the course, and it tracks the course’s day-by-day arc one chapter at a time.

Who this book is for

You have written code before — most likely in **Python**, possibly in MATLAB or JavaScript, occasionally in something compiled. You know how to write a function, a conditional, and a loop, and you have probably encountered objects. You have not necessarily written C++, you have not necessarily compiled anything from a shell, and you may not yet have a clear mental model of what happens between “I saved a file” and “the program ran.”

This book takes that as its starting line. It is not a *first* book about programming. It is, instead, a book about **what changes** when you move from a managed, interpreted, garbage-collected language into a compiled language that exposes the machine — and about the engineering discipline that makes such code livable.

How the book is organized

The chapters are grouped into six parts, one per week of the course. Each chapter corresponds, for the most part, to one lecture session, and the chapters run in the order the course delivers them. A few lectures merge or split in practice — classes, for instance, are introduced and completed in a single combined session here — so the chapter count and the lecture count do not line up one-for-one. The opening chapters take you from a Python mindset into a working C++ inner loop; the middle chapters introduce objects, the layered machine, pointers, and modern memory management; the later chapters reach a deliberate climax with linked structures and a small binary search tree implemented from first principles, then pivot to generics, complexity, tooling, code review, and the final capstone project.

A separate **Lab Tooling** appendix collects the build-system, sanitizer, debugger, version-control, and unit-testing material that comes up across the labs but does not belong inside the conceptual flow.

Conventions used in this book

- **Code listings** are formatted to the course's `.clang-format` style. Where the course's style differs from common practice, the choice is named.
- **Callouts** flag specific kinds of advice:
 - *Note* — clarifying detail
 - *Tip* — a practical move that experienced C++ programmers reach for
 - *Warning* — a pitfall worth remembering
 - *AI touchpoint* — a moment where the course explicitly asks you to bring an AI assistant into the loop
- **and / or / not** are this book's primary spellings for the logical operators, with `&&` / `||` / `!` named on first use and used freely in any imported third-party code. You must be able to read both.

On using AI while reading this book

This course does not treat AI assistants as a contamination risk to be policed. It treats them as a normal part of the modern programming environment, and asks you to use them deliberately. Throughout the book, **AI touchpoints** mark moments where an LLM is the right next tool — explaining a compiler error, generating a unit test, reviewing a function, translating an idiom from Python into C++. You are expected to disclose AI use on graded work; the eight named roles, the disclosure form, and the discipline around what is still cheating even with this stance are spelled out in the AI Tools chapter.

Acknowledgments

Material is reused, adapted, and modernized from the author's previous EC327 offerings between 2018 and 2025. Earlier versions of several chapters appeared as instructor notes in the Summer 2022, Summer 2023, and Summer 2024 sections.

— J. Carruthers, Boston, 2026

Part I

Week 1 — From Python to C++

The opening week takes you from a working Python developer to a working C++ developer with a compiled, running first program, a configured toolchain, and the beginnings of a development rhythm. By the end of these three sessions every reader should have Multipass (or WSL2) running, have compiled and run a program, have used `clang-format` at least once, and have typed `gdb` at least to inspect a variable.

The chapters of this part:

- Chapter 1 — course intro, anatomy of a C++ file, Linux basics, first program
- Chapter 2 — types, expressions, the compile/link cycle, the `and/or/not` keywords, `clang-format` on save
- Chapter 3 — `std::vector<T>`, range-based for, structured bindings, the first user-defined `struct`

1 Course intro, first C++ file, Linux basics

The first day of EC327 has four jobs. By the time you leave you should know **what this course is asking of you**, you should have a **working development environment** you can return to tomorrow, you should be able to **read a small C++ file aloud** and explain what each line does, and you should have a small, working mental model of what *a Linux shell* is.

This chapter walks each in turn.

1.1 What this course is

EC327 is the second programming course in the BU ECE undergraduate sequence. It is a course in **software engineering** taught through the vehicle of **modern C++** (C++20, in 2026). The choice of C++ matters: C++ is the language in which most performance-critical and most embedded code is written, and it *exposes the machine* in ways that Python deliberately hides. The course's deeper goal is for you to leave with a mental model of *what is actually happening* when a program runs, and with the engineering discipline — testing, debugging, version control, code review — that lets you ship code you trust.

You will not be required to buy a textbook. (You have one here, but it is free.) You are *recommended* to read Stroustrup's *Programming: Principles and Practice Using C++* alongside this book; it is the canonical C++-from-scratch text by the language's designer.

1.2 Lab 0: the ungraded standup

The course's first interaction with a terminal is **Lab 0**, an ungraded walkthrough we do together in class. Its goal is to get every machine in the room — Mac, Windows, Linux — to a common standing-start state:

1. **Pick a terminal.** macOS: the built-in `Terminal.app` or `iTerm2`. Windows: Windows Terminal with WSL2 underneath.
2. **Install Multipass** (Mac) or **WSL2 + Ubuntu** (Windows). Both give you a real Linux environment beneath your everyday OS. Native Linux users already have what they need.

3. **Lay out two directories:** `~/ec327/course/` for read-only material the instructors hand you, and `~/ec327/work/` for everything *you* write. You will be told once, and reminded often: **never edit inside course/**. Copy a file out, then edit the copy.
4. **Clone the course repository** into `course/`.
5. **Install the toolchain:** `g++` (or `clang++`), `clang-format`, `clang-tidy`, `gdb`, `git`.
6. **Pick an editor.** Sublime, VS Code, Neovim, and Helix all have working C++ LSP setups; per-editor instructions live in the course's *Editor Setup* handout (in the Tools & Development Environment section of the course site).
7. **Compile and run the starter program.** See Section 1.5 below.

We do all of this *together*, on a projected screen, with everyone in the room typing along. If your machine breaks during the walkthrough, that is the best possible time for it to break.

💡 Security through-line: the *blast radius* of `sudo`

Lab 0 is also the first appearance of the course's *security through-line*. Whenever you type `sudo` you are handing a piece of code permission to do anything on your machine. Pause for a second before each `sudo` and answer: *what code, exactly, am I about to run as root?* This is the same discipline that Claude Code's permission prompts try to teach — the prompt is not in your way, it is asking the question you should have asked yourself.

1.3 Anatomy of a C++ file

Here is the smallest interesting C++ program. Open it line by line.

```
// hello.cpp - your first program in EC327, Summer 2026.

#include <iostream>
#include <string>

int main() {
    std::string name = "EC327";
    std::cout << "Hello, " << name << "! This is C++.\n";
    return 0;
}
```

There are five things to notice, in order.

The comment. Lines starting with `//` are *line comments*; everything from `//` to the end of the line is for humans, ignored by the compiler. A file should always carry one identifying comment at the top. There is also a `/* ... */` block-comment form. You will use line comments 99% of the time.

The two `#include` lines. These are *preprocessor directives*. They are not C++ statements; they are textual instructions to a separate program (the *preprocessor*) that runs before the C++ compiler proper. `#include <iostream>` brings in the declarations needed for `std::cout`; `#include <string>` brings in the declarations needed for `std::string`. In Python you write `import math` and call `math.sqrt`; in C++ you `#include <cmath>` and call `std::sqrt`. The mechanics are different — Python *imports* a module object at runtime, C++ *splices in* a header file at compile time — but the mental model (“I am pulling in code someone else wrote”) is the same.

The `std::` prefix. Names in the C++ standard library live inside a *namespace* called `std`, and `::` is the operator that says “look inside that namespace.” So `std::cout` is the `cout` that lives inside `std`. This is direct analogue of Python’s `math.sqrt`. You will see code (often older code) that says `using namespace std`; near the top so that `cout` works without the prefix. **This course does not write `using namespace std`; in any code we distribute** — there are good reasons, which we will come to in the Modular Design chapter. For now: every standard library name is `std::` prefixed.

`int main()`. This is the *entry point* of the program. When you run a compiled C++ program, the operating system finds a function called `main` and calls it. The `int` says `main` returns an integer; that integer is the program’s *exit code* (0 means success, anything else means failure).

The body, in braces. The block `{ ... }` is what `main` actually does: declare a string, write a line to standard output, return 0. Statements end in semicolons. Whitespace and newlines inside a block have no meaning to the compiler — they exist for *you*.

1.4 The compile / link / run cycle

In Python you run a `.py` file directly:

```
$ python hello.py
```

In C++ the lifecycle has two distinct steps, and you need a working mental model of both.

```
$ g++ -std=c++20 -Wall -Wextra -o hello hello.cpp    # compile + link
$ ./hello                                           # run
Hello, EC327! This is C++.
```

The first command does three things at once:

1. **Preprocess** — the preprocessor handles every `#include`, splicing the header files into the source, and processes any `#define` or conditional compilation directives. The output is a single, large *translation unit*.
2. **Compile** — the compiler translates the translation unit into *machine code* for your specific CPU, packaged in an *object file* (a `.o` file you do not normally see when you use `-o`).

3. **Link** — the linker stitches the object files together with the standard library and produces an executable file named `hello`.

The executable is yours, runnable directly, no interpreter needed. This is the biggest difference between your C++ inner loop and your Python inner loop: you *compile*, you *run*, and if either step fails you fix something and try again.

i The course’s compile invocation

For this course we compile with `-std=c++20 -Wall -Wextra` as a baseline. `-std=c++20` selects the C++20 language standard; `-Wall -Wextra` asks the compiler to warn about a wide range of suspect-looking code. In Lab 2 we add `-Wpedantic` and a small set of sanitizers. The full course `Makefile` and a working `CMakeLists.txt` are in the Build Systems appendix.

1.5 A first program you can extend

In class we extend `hello.cpp` by hand, one small step at a time. The goal is that you finish the first session having *compiled at least four versions* of this same program — not because you needed four versions, but because we are deliberately training the **compile-every-few-lines, run-every-compile** inner loop that is your single most important habit this semester.

The progression looks roughly like this. Each version compiles and runs before moving to the next:

1. Print “Hello, EC327!” — the listing above.
2. Read a name from `std::cin` and greet it back. (Hello to your `<iostream>` input side: `std::cin >> name;`.)
3. Greet it three times by hand, with three `std::cout` lines. (We are *deliberately* not writing a loop yet — that comes tomorrow.)
4. Greet it once, but with a *starter-code* `Point` struct already defined for you in `point.h`, used to print the cursor position. (Your first exposure to a header you did not write; your first use of a struct.)

Each step is its own little change. Each ends with `g++` and `./hello`.

1.6 Linux basics, in 90 seconds

You do not need to be a Linux expert to take this course. You need a working mental model of three things.

The shell vs the terminal. A *terminal* is a window that types characters to a program and shows you what that program types back. A *shell* (`bash`, `zsh`, ...) is a particular program — the one that reads what you type, finds the program you named, and runs it. Today: open your terminal

and notice the *prompt* (typically ending in `$`). Type a command and press enter. That is the shell handing you a *new* prompt after the command finished.

Files and directories. The shell always has a *current directory* (`pwd` prints it). You move with `cd`, list with `ls`, look at file contents with `cat`, edit with your editor of choice. The full reference card is the course's *Terminal Cheatsheet* handout (in the Tools & Development Environment section of the course site).

Processes and `argv`. When you ran `./hello`, the shell *forked* a new process, *exec'd* your program into it, waited for it to finish, and printed the next prompt. Your program runs as its own *process* with its own memory. If you pass arguments — `./hello Alice` — the shell hands them to your program through `argv`, which we will use in earnest in the File I/O chapter.

1.7 What to do before the next session

1. Finish Lab 0 if anything did not work in class. (Office hours are open.)
2. Re-type `hello.cpp` from scratch, without copy/paste. Compile and run it.
3. Modify it three different ways and compile each time.
4. Read Chapter 2.

1.8 What you should understand by the end of this chapter

- What a `.cpp` file looks like and why each part is there.
- What `#include` does and how it compares to Python's `import`.
- What the `std::` prefix means.
- The three steps of the compile/link/run cycle.
- The shape of your development environment for the semester.
- The course's *compile every few lines, run every compile* rhythm.

If any of these feel shaky, you are not behind — you are on schedule. We will revisit each in the next two sessions.

2 Types, operators, selection, and the compile cycle

The first chapter got you a running program. This chapter introduces the small set of language pieces you need to write programs that actually *compute* something interesting: **variables and types**, **operators**, **input from the user**, and **selection** (the `if` statement and friends). The running example is a program almost every engineering student already understands as mathematics — the **quadratic formula** — built up in four stages, each adding exactly one new language feature.

This chapter also takes the second look at the **compile/link cycle** you saw at the end of Chapter 1, this time with enough vocabulary to talk about *why* it works the way it does.

2.1 The plan: quadratic0.cpp through quadratic3.cpp

The four stages are deliberately small. Each one compiles. Each one runs. Each one is broken in a way the next one fixes.

Stage	What it does	What's new
0	reads <code>a</code> , <code>b</code> , <code>c</code> and prints them back; hard-codes “roots are 3 and 5”	variables, <code>std::cin</code> , <code>std::cout</code>
1	computes the real roots using <code>std::sqrt</code> ; breaks if discriminant is negative	<code><cmath></code> , arithmetic operators
2	detects the complex case and refuses to compute it	comparison (<code><</code>), <code>if</code> -statement, exit codes
3	handles real-distinct, real-repeated, complex, and <code>a==0</code> linear cases	<code>if/else if/else</code> , <code>and</code> keyword

The full source for all four lives in `ec327/textbook/code/quadratic{0,1,2,3}.cpp`. You should plan to type each one in by hand (no copy-paste) the first time through. The point of this chapter is the *typing*, not the reading.

2.2 Stage 0: variables, input, and output

```
// quadratic0.cpp
#include <iostream>

int main() {
    double a{};
    double b{};
    double c{};

    std::cout << "Please enter the coefficients a, b, c: ";
    std::cin >> a >> b >> c;

    std::cout << "You typed " << a << " " << b << " " << c << ".\n";
    std::cout << "(The roots are 3 and 5. Probably not, but we will fix that.)\n";

    return 0;
}
```

There are five things to notice.

Declarations with a type. `double a{};` says “make a variable named `a`, its type is `double` (a floating-point number with about 15 digits of precision), and initialize it to zero.” In Python you would just write `a = 0.0` and the language would *infer* a type. In C++ the type comes *before* the name, and you have to write it. This is *static typing* — the compiler knows the type of every variable before the program runs, and uses that to catch entire classes of bugs at compile time.

Uniform initialization with `{}`. The empty braces, `double a{};`, mean “default-initialize”. For `double` that means zero. You can also write `double a{3.14};` to initialize to a particular value. The `{}` form works uniformly for every type — `int`, `double`, `std::string`, your own classes, arrays — and it is the form this book uses by default. The older forms `double a = 0.0;` and `double a(0.0);` mean the same thing here and you will see them in older code.

Uninitialized variables are a real bug class

If you write `double a;` (no `{}`) the variable has *some* value — whatever bits were already in that piece of stack memory. Reading it before assigning to it is *undefined behavior*: the program might appear to work, might crash, might give a different answer every run. The course’s `.clang-tidy` enables the `cppcoreguidelines-init-variables` check, which flags this at lint time. Default-initializing with `{}` is the modern habit that makes the warning go away and the code correct.

Reading input with `std::cin`. The `>>` operator on `std::cin` reads one value at a time, skipping whitespace. `std::cin >> a >> b >> c;` reads three values in a row. If the user types `1 -5 6` and hits enter, `a` is 1, `b` is -5, `c` is 6. If the user types 1 and hits enter, the program waits for the next number. Python’s analogue is `input()` plus a manual parse; C++’s `>>` parses while it reads.

Writing output with `std::cout`. The `<<` operator on `std::cout` writes one value at a time, in left-to-right order. You can chain as many as you like. The `"\n"` at the end of a string literal is a newline character; without it, output keeps going on the same line. (You may have seen `std::endl` in older code; we prefer `"\n"` because `std::endl` also forces a *flush* of the output buffer, which is rarely what you want.)

return 0;. The integer that `main` returns is the program’s *exit code*. By convention, 0 means “everything went fine”. Other values mean “something went wrong, and here’s a number telling you what.” We use this in stage 2.

Compile and run

```
$ g++ -std=c++20 -Wall -Wextra -o quadratic0 quadratic0.cpp
$ echo "1 -5 6" | ./quadratic0
Please enter the coefficients a, b, c: You typed 1 -5 6.
(The roots are 3 and 5. Probably not, but we will fix that.)
```

The `echo ... |` is a *pipe* — the shell hands the string `"1 -5 6"` to your program’s standard input as if you had typed it. Useful for non-interactive testing.

2.3 Stage 1: arithmetic and <cmath>

The mathematics: the roots of $ax^2 + bx + c = 0$ are

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

The C++:

```
// quadratic1.cpp -- new bits only
#include <iostream>
#include <cmath>

int main() {
    double a{}; double b{}; double c{};
    std::cout << "Please enter the coefficients a, b, c: ";
```

```

std::cin >> a >> b >> c;

double disc = b * b - 4 * a * c;

double root1 = (-b + std::sqrt(disc)) / (2 * a);
double root2 = (-b - std::sqrt(disc)) / (2 * a);

std::cout << "The roots are " << root1 << " and " << root2 << ".\n";
return 0;
}

```

Three things changed.

#include <cmath> and std::sqrt. The C++ standard library spreads its mathematics functions across a small set of headers. `<cmath>` gives you `std::sqrt`, `std::pow`, `std::sin`, `std::log`, `std::abs`, and friends. Notice that `std::sqrt` carries the `std::` prefix for the same reason `std::cout` did in Chapter 1: it lives in the `std` namespace.

Arithmetic operators. The five standard ones, with their meaning:

Operator	Meaning	Note
+	addition	
-	subtraction (binary) and negation (unary)	-b is unary - applied to b
*	multiplication	
/	division	integer if both operands are integer; floating if either is floating
%	remainder (modulo)	integer operands only

A subtlety worth remembering: in C++, `13 / 5` is 2, not 2.6. Both operands are integers, so integer division is used. To get 2.6 you write `13.0 / 5` or `13 / 5.0`. (Python 3 made the opposite choice — `13 / 5` is always 2.6, and integer division gets the special `//` operator.)

Precedence and parentheses. The expression `b * b - 4 * a * c` evaluates in the order you would expect from mathematics: the multiplications happen first, *then* the subtraction. C++ has a *precedence table* that pins down the order for every operator combination. You do not need to memorize it. You *do* need to use parentheses freely whenever the order matters and is not obvious

— every senior C++ programmer reads $(a + b) * c$ faster than $a + b * c$, even though only the first one *requires* parentheses.

i The full operator-precedence table

You can find the complete operator-precedence table at https://en.cppreference.com/w/cpp/language/operator_precedence. It is worth glancing at once and then bookmarking. We will add operators to it throughout the book.

The intermediate variable for `disc`. Notice that $b * b - 4 * a * c$ appears in both root expressions. We compute it once, store it in `disc`, and use `disc` in both places. This is the same DRY principle (“*Don’t Repeat Yourself*”) you learned in Python, and the same reason: if the formula has a typo, you want it to have the typo once, not twice.

2.3.1 What goes wrong

Try running stage 1 with 1 2 5:

```
$ echo "1 2 5" | ./quadratic1
```

Please enter the coefficients a, b, c: The roots are nan and nan.

`nan` stands for *not a number* — `std::sqrt(-16)` returned the IEEE-754 sentinel value for an undefined operation, and we then propagated it through the arithmetic. The program ran to completion, but the answer is wrong. Stage 2 makes it notice.

2.4 Stage 2: selection — the `if` statement

```
// quadratic2.cpp -- new bits only
double disc = b * b - 4 * a * c;

if (disc < 0) {
    std::cout << "Sorry, the roots are complex. Try stage 3 of this program.\n";
    return 1;
}

double root1 = (-b + std::sqrt(disc)) / (2 * a);
double root2 = (-b - std::sqrt(disc)) / (2 * a);
```

Two things changed.

The if statement. The shape is `if (condition) { body }`. The *condition* is an expression that evaluates to either `true` or `false`. If it's `true`, the body runs; if `false`, the body is skipped and execution continues after the closing brace.

In `if (disc < 0)`, the condition is the expression `disc < 0`. The `<` operator compares two numbers and produces a `bool` — either `true` or `false`. C++ has six comparison operators:

Operator	Meaning
<code>==</code>	equal to
<code>!=</code>	not equal to
<code><</code>	less than
<code><=</code>	less than or equal to
<code>></code>	greater than
<code>>=</code>	greater than or equal to

 `=` is assignment; `==` is comparison

`if (a = 0) { ... }` is a *bug*, not a syntax error. It assigns 0 to `a` (throwing away whatever `a` had) and then tests whether the value of that assignment, which is 0, is “truthy” — it isn't, so the body never runs. `g++` with `-Wall` flags this with **suggest parentheses around assignment used as truth value**. Heed the warning. We will revisit at length in the Common Errors appendix.

The non-zero return. Stage 2 returns 1 (not 0) in the failure case. A non-zero exit code is the conventional way for a Unix program to tell its caller that something went wrong. You can inspect the exit code of the last program you ran with `echo $?`:

```
$ echo "1 2 5" | ./quadratic2
Please enter the coefficients a, b, c: Sorry, the roots are complex. Try stage 3 of this pro
$ echo $?
1
```

The shell, makefile rules, CI systems, and other programs that compose yours all look at the exit code to decide whether to keep going.

2.5 Stage 3: three-way selection and the `and` keyword

```
// quadratic3.cpp -- the interesting bits only
if (a == 0 and b == 0) {
    std::cout << "a and b are both zero; this is not a quadratic.\n";
    return 1;
}
```

```

}

if (a == 0) {
    // Degenerate linear case:  $bx + c = 0 \Rightarrow x = -c/b$ .
    std::cout << "Linear (a is zero). The root is " << -c / b << ".\n";
    return 0;
}

double disc = b * b - 4 * a * c;

if (disc > 0) {
    double root1 = (-b + std::sqrt(disc)) / (2 * a);
    double root2 = (-b - std::sqrt(disc)) / (2 * a);
    std::cout << "Two real roots: " << root1 << " and " << root2 << ".\n";
} else if (disc == 0) {
    double root = -b / (2 * a);
    std::cout << "One repeated real root: " << root << ".\n";
} else {
    double real_part = -b / (2 * a);
    double imag_part = std::sqrt(-disc) / (2 * a);
    std::cout << "Two complex roots: "
                << real_part << " + " << imag_part << "i and "
                << real_part << " - " << imag_part << "i.\n";
}

```

This is the full program. Three more language pieces appear.

if / else if / else. C++ does not have a dedicated **elif** keyword the way Python does. Instead, **else if** is just an **if** placed inside the **else** branch of an earlier **if**, with the braces left off because the body is itself a single statement. The chain runs *top to bottom* and stops at the first branch whose condition is **true**. The final **else** (no condition) runs if none of the earlier branches matched. Three-way selection — *bigger / equal / smaller*, *positive / zero / negative*, *win / draw / lose* — is common enough that this idiom shows up constantly.

Logical operators with their keyword spellings. The condition `a == 0 and b == 0` uses the **and** keyword to combine two comparisons. C++ has had the spellings **and**, **or**, and **not** as full keywords since C++11, semantically identical to **&&**, **||**, and **!**:

Keyword	Symbolic	Meaning
and	&&	logical AND
or	 	logical OR
not	!	logical NOT

This book writes **and**, **or**, **not** as the **primary spelling** because for this course’s audience — students who already know Python — `if (finished or not found)` reads more transparently than `if (finished || !found)` does. Both forms compile to identical machine code. Neither is enforced or banned by the course’s linter. You will see the symbolic forms `&& || !` in *every* C++ library, AI-generated suggestion, Stack Overflow answer, and textbook other than this one — **you must remain fluent in reading both**.

Why this book made this choice

Two reasons. First, the visual one: `&&` and `&` differ by one character, in the same family — typing one when you meant the other is a real bug class (`if (a & b)` and `if (a && b)` both compile and both do something, but they do very different things). **and** and `&` are *not* visually confusable. Second, the pedagogical one: in a Python-expat audience the keyword forms read like a natural-language sentence on first encounter, which buys back some attention for the actual logic. Stroustrup himself uses the symbolic forms throughout *Programming: Principles and Practice*; we are deliberately diverging on this one point.

Compound statements (blocks). A pair of braces `{ ... }` groups a sequence of statements into a single *compound statement* (a “block”). Any single statement can be replaced by a block. The body of an `if` is a single statement; if you want two statements there, wrap them in braces. This is *how* the chain `if (...) { ... } else if (...) { ... } else { ... }` works mechanically: each `{ ... }` is a single compound statement playing the role of `if`’s body.

Always brace your `if` bodies

You *can* leave the braces off a single-statement body: `if (a < 0) return 1;` is legal. The course style is to always brace anyway:

```
if (a < 0) {  
    return 1;  
}
```

The reason is one of the most expensive bugs in C history — the Apple SSL “*goto fail*” bug of 2014, where a misindented unbraced `if` body silently turned off all certificate verification on iPhones for months. Always-brace is the discipline that prevents whole categories of edit-time mistakes. `clang-format` will adjust the spacing for you; it cannot insert the braces you forgot to type.

2.6 A second worked example: a grade calculator

The quadratic series is the right *first* exercise because it forces you to confront `<cmath>`, comparison, and a three-way branch in a context that is unmistakably mathematics. But selection shows up

everywhere in real code, not just in the math library. The second worked example takes a problem familiar to every undergraduate — *turning a numerical score into a letter grade* — and walks through the same selection ideas in a fresh setting.

The full progression of this example has nine stages (`grade_v1.cpp` through `grade_v9.cpp`); each later stage replaces the selection logic with a more sophisticated data-structure technique. The later stages need ideas from Chapter 3 and beyond, so we stop at stage 2 here. The progression picks up again in chapter 3.

2.6.1 Stage 1: a sequence of independent ifs

```
// grade_v1.cpp -- the interesting part only
if (g < 50.0) { let = 'F'; }
if (g >= 50.0 and g < 60.0) { let = 'D'; }
if (g >= 60.0 and g < 70.0) { let = 'C'; }
if (g >= 70.0 and g < 80.0) { let = 'B'; }
if (g > 80.0) { let = 'A'; }    // BUG: should be g >= 80.0
```

Read this carefully before reading the rest of this paragraph. *What is the shape of the bug?* Try the program on input 80:

```
$ echo 80 | ./grade_v1
Enter the student's grade: The letter grade is  .
```

There is a blank where the letter should be. The student got a perfect score of 80.0. None of the five conditions matched (80 < 50 no, 50 ≤ 80 < 60 no, ..., 80 > 80 no), so `let` was never assigned. We *did* default-init `let` to `'\0'` with the empty `{}`, so the program prints what looks like a space — undefined-behavior-by-strict-reading was avoided by the default-initialization, but the *output* is still wrong.

There is also a more insidious bug in this version that the input 80 does not expose: every condition is tested independently. If two conditions both match (because we wrote the boundaries inconsistently), the *later* one wins. We do not have that problem here, but a future maintainer who adds a new range without thinking carefully *will*.

2.6.2 Stage 2: a single if / else if / else chain

```
// grade_v2.cpp -- the interesting part only
if (g < 50.0) {
    let = 'F';
} else if (g < 60.0) {
```

```
    let = 'D';
} else if (g < 70.0) {
    let = 'C';
} else if (g < 80.0) {
    let = 'B';
} else {
    let = 'A';
}
```

Two things happened.

The boundary bug is gone. Every branch except the last uses a strict `<`. A score of exactly 79.999 is a B; a score of exactly 80.0 falls through to the final `else` and is an A. We did not even need to write `>= 80.0` anywhere — the unconditional `else` says “*everything that didn’t match above*”. That is a real engineering virtue: the partitioning logic is expressed *once*, in one place, and is exhaustive by construction.

The “let was never assigned” failure mode is gone. The chain is exhaustive — for any double value of `g`, exactly one branch runs. `let` is always set.

```
$ echo 80 | ./grade_v2
Enter the student's grade: The letter grade is A.
```

The pattern, named

Whenever you find yourself writing several `ifs` in a row to handle disjoint cases of the same value, the rewrite to `if / else if / else` is almost always the right move. The cases become *visibly* mutually exclusive, the partitioning is forced to be exhaustive (or you write a final `else` that explicitly handles the “I forgot a case” path), and a future reader does not have to mentally re-check whether any two conditions could overlap.

2.6.3 Where the progression goes next

`grade_v3.cpp` notices that the `if/else if/else` chain is itself a repeated pattern (compare-against-threshold, assign-corresponding-letter) and replaces it with two parallel arrays and a `while` loop. `grade_v4.cpp` swaps the C-style arrays for `std::vector`. `grade_v5.cpp` introduces a `struct Grade` and replaces the parallel-vectors-with-an-index pattern with a single `vector<Grade>` and a range-based `for`. The arc continues through `grade_v9.cpp`, picking up early-break, enums, and a small refactor into a free function. **We do not need any of that yet** — it is the job of Chapter 3 and the Functions chapter to introduce those pieces. The point here is that you can already see the *shape* of the refactor: a problem that looked like five separate branches turns out to be a *table*,

and once it is a table the language gives you better ways to express it. That is one of the recurring rhythms of the book.

2.7 The compile/link cycle, revisited

Look again at the command from Chapter 1:

```
$ g++ -std=c++20 -Wall -Wextra -o quadratic3 quadratic3.cpp
$ ./quadratic3
```

You now have the vocabulary to read it more carefully.

1. **Preprocess.** Every `#include` line is a directive to the *preprocessor* to splice the contents of that header into the file. After preprocessing, `quadratic3.cpp` is no longer 50 lines; it is tens of thousands of lines, because `<iostream>` and `<cmath>` brought in everything they declare. The output is a single *translation unit*.
2. **Compile.** The compiler reads the translation unit, type-checks it (catching, for example, `std::sqrt("hello")` before the program ever runs), and translates it into machine code for your CPU. The output is an *object file*.
3. **Link.** The linker stitches your object file together with the standard library's pre-compiled object files (so `std::cout` and `std::sqrt` resolve to real code) and produces the *executable*.

C++ does all three in this one `g++` command. They are conceptually distinct and you will eventually want to invoke them separately (e.g. to compile two `.cpp` files into one program — see the Functions chapter and the Build Systems appendix).

Why this matters, and what it's costing you. Each step adds *time*. For a 50-line program this is unnoticeable; for a 50,000-line project it can be minutes. But for a 50-line program you also get something Python cannot give: a guarantee, before the program ever runs, that every type matches up, every name resolves, every function you called exists and has the right signature. The compiler is your first and most relentless reviewer. **Listen to its warnings.** `-Wall -Wextra` is the bare minimum; we will add more in the Build Systems appendix.

2.8 Interpreted vs compiled

Why does C++ do it this way and Python does not? The short answer is *when the work happens*.

- **Python** ships your `.py` file to an *interpreter* (the `python` program). The interpreter reads each line as the program runs, figures out at that moment what type each variable has, looks up each method by name in a dictionary, and dispatches. Almost all of the work is done *at run time*.

2 Types, operators, selection, and the compile cycle

- C++ does most of the work *at compile time*. The compiler resolves types, selects the right overload of `std::sqrt`, lays out the memory for each variable, and emits CPU instructions that the CPU executes directly. At run time there is no interpreter, no dictionary lookup per call, no type check.

Each choice has tradeoffs. Python is faster to *write* and slower to *run*; C++ is the reverse. Python lets you change a function's signature in one place and try it immediately; C++ asks you to recompile every file that depended on that signature, but in exchange catches the call sites you forgot to update.

For the kind of code most software engineers ship — a programs that runs on a million phones, a service that handles a thousand requests per second, a chunk of firmware that boots an aircraft — that tradeoff is overwhelmingly in C++'s favor on the *run* side, and the engineering practices in the rest of this book are largely about making the *write* side bearable enough that the tradeoff is worth it.

2.9 Whitespace is not significant

If you came from Python, this is the most jarring difference in the language.

In Python, indentation *defines* a block. In C++, indentation is purely a visual hint for humans; *braces* define the block, and *semicolons* terminate statements. All four of these compile to the same program:

```
if (x < 0) { std::cout << "negative\n"; }

if (x < 0) {
    std::cout << "negative\n";
}

if    (x<0)    {std::cout<<"negative\n";}

if
(
    x
    <
    0
)
{ std::cout << "negative\n"; }
```

This is a freedom that gets abused. The course response is: **clang-format on save**. The starter repo ships a `.clang-format` config (Google baseline, two-space indent, 80-column lines, attached

braces, left-align `*` and `&`). Configure your editor to run it on save and the question of “where does the brace go” becomes uninteresting. The course will not award or deduct points for formatting; the linter does that mechanically.

We will revisit `clang-format` in the Sanitizers appendix when we wire it into the build.

2.10 Practice: ways to break each stage

The point of having four stages is that *each one is broken*. Before moving on, spend ten minutes trying to break them on purpose. For each stage, ask:

1. What input produces wrong output without an obvious error?
2. What input crashes the program (or makes it loop forever)?
3. What input does the program *refuse* to handle, with a clear message?

Stage 0 is broken because it lies (it always says “3 and 5”). Stage 1 is broken because it returns `nan` silently for the complex case. Stage 2 is broken because it returns an error message instead of the complex roots that do exist. Stage 3 is *less* broken — it handles every input we have discussed — but it has a subtle bug: `if (a == 0)` is comparing a floating-point number to exactly zero. For an input like `0.00000001 1 1`, stage 3 thinks `a` is not zero and solves the quadratic correctly; for `0.0 1 1` it solves the linear case correctly; for something between, the answer can be numerically terrible. We do not fix this here. It is a real bug, and one of the standing themes of the book: every program has bugs, and the discipline is *knowing which bugs you decided not to fix*.

AI touchpoint — explain a compiler error

Open `quadratic1.cpp` and delete the `#include <cmath>` line. Compile it. You will get an error something like `'sqrt' is not a member of 'std'`. Paste the entire error message into an AI assistant and ask: “*What does this error mean and how do I fix it?*” Read the answer. Then ask: “*What other common errors look like this one?*” This is the **AI as IT aide** role from the AI Tools chapter, and it is the most useful single habit you can pick up in week 1.

2.11 What you should understand by the end of this chapter

- Declaring a typed variable and initializing it with `{}`
- The five arithmetic operators, integer vs. floating division, and when to use parentheses
- Reading input with `std::cin >>` and writing output with `std::cout <<`
- The six comparison operators and the difference between `=` and `==`
- The three logical operators in both their keyword (`and/or/not`) and symbolic (`&&/||/!`) forms
- Selection with `if`, `if/else`, and `if/else if/else`
- Why blocks are always braced in this course

2 Types, operators, selection, and the compile cycle

- The three steps of the compile/link cycle and why C++ does it this way
- That whitespace is not significant and `clang-format` settles the question

If you can read all four stages of the quadratic solver out loud and explain what each line is doing, you are ready for Chapter 3.

3 `std::vector`, `range-for`, and your first struct

Chapter 2 left you at a place where you can write programs that handle *one* value at a time. Read a number; classify it; print a letter. The world that matters to engineers is full of *collections* of values — students in a class, samples in a signal, pixels in an image, points along a trajectory — and the language tools for working with collections are different enough from the single-value tools that they deserve their own chapter.

This chapter introduces three intertwined ideas:

1. **`std::vector<T>`** — C++’s default sequence container, the workhorse you will reach for in 80% of programs you write this term.
2. **Iteration**, in the three idioms you should know on sight: the range-based `for`, the indexed `for`, and a brief look at iterators.
3. **`struct`** — your first user-defined type, the language piece that lets you say “*these N pieces of data go together*” once and treat the group as a single named thing.

The running example is the **grade-calculator progression you started in Section 2.6**. We pick it up at stage 3 and walk through stage 5. Each stage introduces exactly one new idea, and each stage’s last bug is what motivates the next stage.

3.1 Stage 3: parallel arrays and the `while` loop

`grade_v2` ended with a five-branch `if / else if / else` chain. The *structure* of those five branches is identical — compare `g` against a threshold, assign the corresponding letter. When the structure is identical the code is asking to be replaced by *data* (the table of thresholds and letters) plus a *loop* that walks the data.

```
// grade_v3.cpp -- the interesting bits only
double thresholds[5] = {50.0, 60.0, 70.0, 80.0, 100.0};
char letters[5]      = {'F', 'D', 'C', 'B', 'A'};

double g{};
std::cin >> g;

int i = 0;
```

3 `std::vector`, range-for, and your first `struct`

```
while (g >= thresholds[i]) {  
    i++;  
}  
char let = letters[i];
```

Three things appear for the first time.

C-style arrays. `double thresholds[5] = { ... };` declares a *fixed-size* array of five doubles. The `[5]` is the *length* (not a maximum — the length is part of the type), and the values inside the braces are the elements in order. You access an element by *subscripting* with `[i]`, where `i` is an integer index in the range `0..4` (zero-based, like Python).

The while loop. `while (condition) { body }` runs `body` repeatedly *while* `condition` is true. Check the condition; if true, run the body; check the condition again; if true again, run the body again; eventually the condition becomes false and the loop exits. The condition `g >= thresholds[i]` says “as long as my grade is still at or above the threshold at position `i`, move on to the next position.”

The index variable pattern. `int i = 0;` outside the loop, `i++` inside. This is the canonical *visit every position from 0 upward* idiom; we will see a tighter spelling of it (`for`) in a moment.

What goes wrong at the boundary

Run `grade_v3` with input 100:

```
$ echo 100 | ./grade_v3  
Enter the student's grade: The letter grade is .
```

Garbage in the output. Trace it: `i` starts at 0. `100 >= 50` is true, so `i` becomes 1. `100 >= 60` is true, so `i` becomes 2. `100 >= 70`, true, `i = 3`. `100 >= 80`, true, `i = 4`. `100 >= 100`, true, `i = 5`. Now the loop body has set `i = 5`, the condition re-checks `g >= thresholds[5]`, and we read the *sixth* element of a five-element array. There is no sixth element. The C-style array does not check the bounds; the program just reads whatever bits sit beyond the end and uses *that* as the threshold and the letter.

This is *undefined behavior* — the most feared phrase in the C++ vocabulary. The program does not error out; it does not stop; it produces *some* answer. The answer is wrong. Worse, the answer may be different on a different machine, with a different compiler, at a different time of day, in a different build configuration. We will name this category of bug repeatedly throughout the book.

3.2 Why we do not write C-style arrays in 2026

C-style arrays predate C++ entirely — they are 1972 Kernighan-and-Ritchie C, unchanged in essentials for half a century. They have four properties that modern C++ deliberately works around:

1. **They do not carry their size with them.** `thresholds[5]` is a `double[5]` at the point of declaration, but the moment you pass it to a function it *decays* to a bare pointer that has forgotten how long it is. Bounds checking becomes the caller's manual problem.
2. **They have no bounds checking at the subscript site.** `thresholds[5]` compiles fine even though there is no element 5.
3. **They cannot grow.** A `double[5]` is always exactly 5 elements.
4. **They cannot easily be assigned, returned from a function, or stored in other containers.**

Modern C++ gives you two replacements:

- `std::array<T, N>` — fixed size like the C-style array, but carries its size, lives properly in the type system, and supports `.size()`, `.at()`, range-for, and friends.
- `std::vector<T>` — variable-size, grows as you push elements onto it, the default everyday container. *This is the one you reach for first.*

Stage 4 swaps the C-style arrays for `std::vector`.

3.3 Stage 4: `std::vector` replaces the C-style array

```
// grade_v4.cpp -- the interesting bits only
#include <vector>

std::vector<double> thresholds{50.0, 60.0, 70.0, 80.0, 100.0};
std::vector<char>   letters   {'F', 'D', 'C', 'B', 'A'};

double g{};
std::cin >> g;

std::size_t i = 0;
while (g >= thresholds[i]) {
    i++;
}
char let = letters[i];
```

The *logic* is unchanged. The container is different.

3 `std::vector`, range-for, and your first `struct`

`std::vector<T>` and the `<vector>` header. `std::vector` is in the standard library; you `#include <vector>` to get it. The `<T>` is a *template parameter*: `std::vector<double>` is “a vector of doubles”, `std::vector<char>` is “a vector of chars”, `std::vector<std::string>` is “a vector of strings”. We will say more about templates in the Templates chapter; for now, treat the `<T>` like the `[T]` in Python’s `list[int]` type hint.

Brace initialization. `std::vector<double> thresholds{50.0, 60.0, ...};` uses the same `{ ... }` uniform-initialization syntax you saw with `double a{};` in ch02. With braces full of values, it initializes the vector with those elements in order. The vector’s size is whatever you put inside the braces — five, in this case.

`std::size_t` instead of `int`. Container sizes and indices in C++ are *unsigned* — they cannot be negative. The standard type for them is `std::size_t`. Using `int` here would technically work but would trigger a compiler warning about sign-compare in `i < thresholds.size()`-style expressions you will write five seconds from now. Start using `std::size_t` for indices early; it is one less warning to wade through later.

 `vector::operator[]` is *also* unchecked

The 100-input bug from `grade_v3` is not gone. `std::vector::operator[]` — the `[i]` notation — does **not** bounds-check either. The reason is performance: `operator[]` is designed to compile down to a single memory load, the same as the C-style array does, so that a tight inner loop has no hidden cost. If you want bounds checking, use `thresholds.at(i)` instead — it throws `std::out_of_range` on a bad index. We will dig into the tradeoff in the Debugger chapter. For now: the 100-input bug from stage 3 is *still here*. Stage 5 fixes it properly by changing the loop shape so the out-of-bounds case never happens.

3.4 Iterating a vector — three idioms you should know

Before we get to stage 5, take a slight detour. Once you have a vector, the single most common thing you do with it is *visit every element*. C++ has several spellings of this; three of them you should recognize on sight.

The running example: a vector of ints, printed.

```
std::vector<int> v{4, 5, -4, 1, 2, 7, 8, -9, 42, 10};
```


3.4.1 Idiom 1: range-based for (*the default you reach for first*)

```
for (auto x : v) {
    std::cout << x << ' ';
}
```

Read it as English: “*for each x in v , do...*”. The variable x takes on each element of v in order. The type is `auto`, meaning *the compiler figures out from v what x ’s type should be* — here, `int`. You do not say `int`, `size_t`, or anything about the iteration mechanics; the loop is about what you do with each element, not about how you get there.

When you need to *modify* the elements as you go, write `auto&` (reference to the element). When you only need to read them and they are expensive to copy (like a `std::string`), write `const auto&`.

3.4.2 Idiom 2: indexed for (*when the position matters*)

```
for (std::size_t i = 0; i < v.size(); i++) {
    std::cout << "v[" << i << "] = " << v[i] << '\n';
}
```

This is the C-style `for` loop you have probably written in Java or JavaScript. It has three semicolon-separated pieces: an *initializer* (`std::size_t i = 0`), a *test* (`i < v.size()`), and a *step* (`i++`). Use this when the *index* itself is part of what you are printing or computing, or when you need to skip elements (`i += 2`).

`v.size()` returns the current number of elements (as a `std::size_t`).

3.4.3 Idiom 3: iterator-based (*seen everywhere, written rarely*)

```
for (auto it = v.begin(); it != v.end(); ++it) {
    std::cout << *it << ' ';
}
```

`v.begin()` returns an *iterator* pointing at the first element; `v.end()` returns an iterator pointing *one past* the last element. The `*it` syntax *dereferences* the iterator to give you the element it is currently pointing at. `++it` advances to the next position.

You will not write iterators by hand often — the range-based `for` is almost always shorter. But every STL algorithm in the STL Algorithms chapter takes a pair of iterators (`begin`, `end`), so you need to know what they are.

💡 Which idiom to use, by default

- **Range-based for** for “do something with each element”.
- **Indexed for** when the index is part of the computation.
- **Iterator-based** only when an STL algorithm gives you iterators back, or when you need fine-grained control (skip elements conditionally, iterate partway through). Otherwise: range-based.

3.5 Your first struct: putting fields together

In `grade_v4` we had two parallel vectors: a vector of thresholds and a vector of letters. The two vectors had to stay the same length and the same *order*. If anyone ever edited one without editing the other, the program breaks silently — the letter for B would suddenly point at the threshold for C. That kind of constraint, where two *logically connected* pieces of data are held apart by the language, is exactly what a `struct` exists to fix.

```
struct Grade {  
    double threshold;  
    char letter;  
};
```

`struct` is the keyword. `Grade` is the new *type name*. The body is a list of *member declarations* — fields, in Python terms. After this declaration, `Grade` is a usable type, just like `int` or `std::string`.

You construct a `Grade` value with brace initialization in field-declaration order:

```
Grade g1{50.0, 'F'};           // threshold 50.0, letter 'F'  
Grade g2{60.0, 'D'};
```

You access fields with the `.` operator:

```
std::cout << g1.letter << " starts at " << g1.threshold << "\n";
```

And — the punch line — you can put `Grades` inside a `std::vector`:

```
std::vector<Grade> scale{  
    {50.0, 'F'},  
    {60.0, 'D'},  
    {70.0, 'C'},  
    {80.0, 'B'},
```

```
{100.0, 'A'},
};
```

Now the threshold-and-letter pairing is enforced by the type system. There is no way to mis-align the two vectors because there is only one vector.

i struct vs. class

Both keywords define a new user-defined type. In C++ the only language-level difference between `struct` and `class` is the *default access* of members (`struct` defaults to public, `class` defaults to private). The conventional distinction is *cultural*:

- Use **struct** when the type is a *bag of fields* with no real invariants beyond “these belong together” — `Grade`, `Point`, `Color`, `Rect`.
- Use **class** when the type *enforces something* — `BankAccount` (whose balance must be consistent), `LinkedList` (whose nodes must be reachable from the head), `File` (whose handle must be closed).

Chapter 5 takes up `class` in earnest. For this chapter, every user-defined type we write is a `struct`.

3.6 Stage 5: `vector<Grade>` + `range-for` + `early break`

```
// grade_v5.cpp -- the interesting bits only
struct Grade {
    double threshold;
    char letter;
};

std::vector<Grade> scale{
    {50.0, 'F'}, {60.0, 'D'}, {70.0, 'C'}, {80.0, 'B'}, {100.0, 'A'},
};

double g{};
std::cin >> g;

char let = '?';
for (const auto& row : scale) {
    let = row.letter;
    if (g < row.threshold) {
```

3 `std::vector`, range-for, and your first `struct`

```
        break;
    }
}
```

This is the version that finally *looks like* what the problem is asking for. Read the loop aloud: “*for each row in the scale, tentatively pick the row’s letter; if my grade is below the row’s threshold, that’s the answer.*”

Three small ideas pulled together.

const auto& row — for each iteration, **row** is a *const reference* to the current **Grade** in the vector. **const** because we are reading the row, not modifying it. **&** because we want to refer to the actual element without copying it (a **Grade** is small here, but the habit is the one you want when the elements are big).

Tentative assignment + check. We assign **let** *before* checking whether to stop. So after the loop runs to completion — even if no threshold triggered the **break** — **let** is the letter from the *last* row, which is 'A'. The 100-bug from stages 3 and 4 is gone: when **g** = 100, we walk all five rows, and the last **let** = 'A' is the answer.

break. The **break** statement exits the *innermost enclosing loop* immediately. It is the right tool for “I have found the answer, stop looking.” Like every selection or jump statement, it should be used sparingly enough that a reader is not surprised by it; one **break** in a loop, used at the natural exit point, is fine.

```
$ echo 100 | ./grade_v5
Enter the student's grade: The letter grade is A.
$ echo 49 | ./grade_v5
Enter the student's grade: The letter grade is F.
$ echo 75 | ./grade_v5
Enter the student's grade: The letter grade is B.
```

3.7 Structured bindings: peeking inside a row

A modern C++ shorthand worth knowing now even though you will use it rarely in week 1: you can destructure a **struct** (or a **std::pair**, or a **std::tuple**, or a fixed-size array) into named variables at the point of access:

```
for (const auto& [thr, let] : scale) {
    std::cout << let << " starts at " << thr << '\n';
}
```

The `[thr, let]` says “this row has two fields; bind them locally to `thr` and `let`”. The names you choose are local to the loop body. This is the same destructuring idea as Python’s `for k, v in d.items():`. We will use it again in the File I/O chapter when we are reading two-field records from a file.

3.8 Two-dimensional structures

Real engineering data often does not fit in a single sequence. An image is a 2D grid of pixels; a chessboard is an 8×8 grid of squares; a matrix is a 2D grid of numbers. C++ gives you three ways to represent a 2D grid, and which you pick affects performance and clarity.

3.8.1 Option 1: `vector<vector<T>>` (*the flexible default*)

```
std::size_t rows = 4, cols = 5;
std::vector<std::vector<int>> grid(rows, std::vector<int>(cols, 0));
grid[2][3] = 42;
```

Rows can be different lengths if you want (a *ragged* array). Either dimension can be resized at runtime. Indexing is the natural `grid[r][c]`. Memory is allocated in pieces — one allocation for the outer vector of vectors, plus one per row — which is *fine* for most programs but not ideal for tight numerical inner loops.

3.8.2 Option 2: `array<array<T, N>, M>` (*when the size is fixed and small*)

```
#include <array>
std::array<std::array<int, 5>, 4> grid{}; // 4 rows, 5 cols, zero-init
```

Same `grid[r][c]` indexing. The dimensions are *part of the type*, so they are checked at compile time. Memory is one contiguous block; no allocation, no extra layer of indirection. This is what you want when the dimensions are known ahead of time and small (a 3D rotation matrix, a chessboard).

3.8.3 Option 3: flat row-major (*when performance is the brief*)

```
std::vector<int> grid(rows * cols, 0);
auto at = [&](std::size_t r, std::size_t c) -> int& {
    return grid[r * cols + c];
};
at(2, 3) = 42;
```

3 `std::vector`, range-for, and your first `struct`

One contiguous block of memory, addressed by hand. This is the layout that your CPU's cache prefetcher loves; large numerical libraries like Eigen and NumPy use it underneath. The cost is that your code now has to know about the *stride* (`cols`). The tiny `at` lambda hides that arithmetic at the call sites.

💡 Which 2D representation, by default

- `vector<vector<T>>` when the sizes are dynamic and you do not care about the last 20% of performance.
- `array<array<T, N>, M>` when the sizes are known at compile time and small.
- **Flat row-major** when an inner numerical loop is the hot path. We will revisit this in the Complexity chapter.

A worked example: `box_of_stars` from the priors. Two nested `for` loops, no 2D container needed because we never store the grid — we just print it.

```
// box_of_stars.cpp
#include <iostream>
int main() {
    int X{}, Y{};
    std::cin >> X >> Y;
    for (int i = 0; i < X; i++) {
        for (int j = 0; j < Y; j++) {
            std::cout << '*';
        }
        std::cout << '\n';
    }
    return 0;
}
```

Read it twice. The *outer* loop runs `X` times, once per row. Inside each row, the *inner* loop runs `Y` times, once per column, printing one `'*'` each time. After the inner loop finishes, the outer loop's body prints a newline and moves on to the next row. This is the canonical shape of a nested `for`; you will see it in every image-processing loop you ever write, even when the *grid* lives somewhere else than on the screen.

3.9 What changes in stages 6 through 9 (a preview)

The grade-calculator progression keeps going, but the next changes are about *organization*, not iteration:

- `grade_v6` reorganizes the loop condition (testing `>` against the row threshold rather than `<`).

- `grade_v7` factors the loop out into a free function `char get_grade(double g)` — covered in the Functions chapter.
- `grade_v8/v9` lift the `vector<Grade>` and the lookup logic into a small class, `GradeScheme` — covered in the two Classes chapters.

The recurring point is that the *physics* of the problem (turn a grade into a letter) has not changed since stage 1. Every iteration of the program moves the burden of *expressing* that physics from the *code* (handwritten branches) into the *data* (a table) and then into the *type system* (a struct, a class). That trajectory — push work from code into data into types — is one of the central rhythms of good engineering, and we will return to it.

3.10 Practice: try to break each stage

As with chapter 2, every stage of the progression is broken in a way the next stage fixes. For each of `grade_v3`, `v4`, `v5`, ask:

1. What input crashes it or produces wrong output?
2. What input would you *expect* to crash but does not?
3. Which fix does the next version apply, and would a *different* fix also work?

For example: `grade_v4` could be fixed *without* introducing a struct, by using `.at()` instead of `[]` — that would throw a clean exception at the 100-input case rather than silently reading garbage. Is that a better fix than what `v5` does? Why or why not?

 AI touchpoint — generate alternative implementations

Open `grade_v4.cpp` and ask an AI assistant: “Show me three different ways to handle the bounds-check problem on input 100, ranging from minimal to idiomatic.” Read the answers critically. Which would you ship? Which would you reject as over-engineering? This is the **AI as test-case generator + reviewer** role from the AI Tools chapter, applied to a tiny problem you already understand.

3.11 What you should understand by the end of this chapter

- Why we do not write C-style arrays in 2026 (and what we write instead)
- Declaring a `std::vector<T>` with brace initialization
- That `vector::operator[]` is unchecked and `.at()` is the safe alternative
- The three iteration idioms (range-for, indexed-for, iterator) and when to reach for each
- Defining a `struct`, accessing its fields with `.`, putting structs in a vector
- The `const auto&` pattern in a range-for
- The `break` statement and when it is the right tool

3 `std::vector`, *range-for*, and your first *struct*

- Structured bindings (`auto& [a, b] = pair`) at sight-recognition level
- The three 2D representations and how to choose between them

If `grade_v3` through `grade_v5` make sense end to end — including *why* the 100-input bug exists in `v3` and `v4` and *why* `v5` does not have it — you are ready for chapter 4.

Part II

Week 2 — Functions, recursion, classes

The second week introduces the two core *units of organization* in C++ — functions and classes — and treats recursion as a first-class technique, not an afterthought. The week ends with a payoff: the loop-hygiene discipline you have been building since week 1 turns out to be the same discipline behind the STL algorithms, and most well-shaped loops you ever write are an STL algorithm in disguise.

The chapters of this part:

- Chapter 4 — bridging from Python’s `def`, headers and translation units, pass-by-value vs `const&`, `<algorithm>` and `<numeric>`, recursion
- Chapter 5 — from `struct` to `class`, constructors and member functions, destructors, RAII as the central idea, `const`-correctness, and what makes a class earn its existence (taught in one combined session in 2026)
- Chapter 6 — loop hygiene as the spine, “name your loop”, STL algorithms
 - lambdas as the customization point

4 Functions, headers, and recursion

You have been writing functions since your first week of Python. `def f(x):`, a body, a `return` — the idea is not new. What *is* new in C++ is that a function is a **typed contract**: the parameter types and the return type are written down, checked by the compiler, and enforced at every call site. That one change ripples outward into a set of decisions Python never asked you to make — *how* to pass an argument, *what* to return when there might be no answer, *where* the function lives when your program outgrows a single file — and into a vocabulary of judgment about what makes a function *good*.

This chapter has two halves, mirroring the two days the course spends on it. The first half is **mechanics and judgment**: the signature as a contract, pass-by-value vs `const&`, returning by value, pure vs effectful functions, the rule about globals, and a gallery of good and bad functions. The second half is **recursion** and the move to **multiple files** — splitting a reusable thing into a `.h/.cpp` pair — closing with the *shared-parameter smell* that motivates classes in Chapter 5.

The running example is the **paycheck program** you have been calling library functions inside since Chapter 3. In this chapter you finally write those functions yourself.

4.1 From Python `def` to a typed C++ function

Here is the same function in both languages. In Python:

```
def is_leap_year(year):  
    return year % 4 == 0 and (year % 100 != 0 or year % 400 == 0)
```

And in C++:

```
bool is_leap_year(int year) {  
    return year % 4 == 0 and (year % 100 != 0 or year % 400 == 0);  
}
```

The logic is line-for-line identical — including the `and` and `or` keywords, which (as established in Chapter 2) this book spells out rather than writing `&&` and `||`. Three things are new in the C++ version, and all three come from the same source: **types are written down and checked**.

- The **return type** is named: `bool`. The compiler verifies that every **return** statement in the body actually produces a `bool`.
- The **parameter type** is named: `int`. The compiler verifies that every *caller* passes something that is (or converts to) an `int`.
- The function is **declared once**, and the compiler enforces that one signature everywhere. Call `is_leap_year(2026.5)` and the `double` silently truncates to `int` (a conversion, with a possible warning). Call `is_leap_year("2026")` and you get a hard compile error before the program ever runs.

One more difference, easy to miss: the C++ function **does not capture surrounding variables**. A Python function can close over names in the enclosing scope; this C++ function is *exactly* its parameter `year` and its return value, nothing else. (C++ does have closures — lambdas — but they are explicit about what they capture, and they wait until Chapter 6.) For now, hold this framing in your head, because the rest of the chapter is built on it:

The signature is the contract.

4.2 The signature as a contract

Read this signature *without* the body:

```
double interest(double principal, double rate, int years);
```

What does it do? You cannot know the exact formula, but you can read an enormous amount from one line:

- It takes three values and returns one `double`.
- It does **not** modify `principal`, `rate`, or `years` — they are passed by value, so the function works on copies.
- It does **not** print anything, fail, or touch a file — no side effect is named in the type. (We will make “named in the type” precise below.)
- It **always returns something** — it does not say `std::optional`, so there is no “no answer” case to handle.

That is a lot of information from a line you have not even finished reading. The **names** carry the rest. `interest`, not `f` or `do_it` or `calc`. `principal`, not `p`. `years`, not `t` or `n`. A good name plus a good type signature lets a reader **skip the body** most of the time — and that is the whole point.

The 80% heuristic

A function’s name and signature should let a reader predict what it does, correctly, about 80% of the time *without opening the body*. If a reader has to read the implementation to know what

the function does, the signature has failed at its primary job. This is not a style nicety; it is what makes a codebase of ten thousand functions navigable at all.

4.3 Passing arguments: value vs *const&* vs reference

In Python you never thought about this. You pass an object, the function gets a reference to it, and whether your change is visible to the caller depends on whether the object is mutable. C++ makes the choice explicit, and the choice matters for both **correctness** and **performance**. There are three options.

4.3.1 Pass-by-value — for small things

```
int twice(int n) { return n * 2; }
```

`n` is a **fresh local copy** of whatever the caller passed. Mutating `n` inside `twice` does nothing to the caller's variable. For an `int`, `double`, `char`, `bool`, or a small `struct`, copying is cheap — often free, because the compiler optimizes the copy away entirely. This is the default for small types.

4.3.2 Pass-by-*const&* — for big things you only read

```
int total_length(const std::vector<std::string>& names);
```

Recall the type spelling from Chapter 3: `const std::string&` is “a read-only reference to a string.” The `&` says *no copy is made* — `names` refers directly to the caller's vector. The `const` says *and the function promises not to modify it*. Together they buy you both speed (no copy of a potentially huge vector) and safety (the compiler rejects any line in the body that tries to change `names`). The signature still reads as a clean contract: data flows **in**, and the only thing that flows back **out** is the return value.

4.3.3 Pass-by-non-*const* reference — almost never

```
void bigify(int& x, int& y, int n);    // modifies x and y in place
```

A plain `int&` (no `const`) is a reference the function is *allowed to write through*. After `bigify(a, b, 3)` returns, `a` and `b` may have changed. This is occasionally the right tool — but it hides a write behind a call, which is exactly the kind of surprise that produces long debugging sessions.

 The non-const reference rule

If a function has a non-`const` reference parameter, the parameter's *name* must make the modification obvious — `out_count`, `result`, `dest` — or you should redesign the function to **return** the value instead. Modern C++ returns; it does not fill out-parameters. The narrow exception is a value genuinely expensive to construct where the compiler's copy-elision cannot help, which is rare in this course.

4.3.4 The decision, as a tree

Is the argument an `int`, `double`, `char`, `bool`, or small `struct` ($\leq \sim 16$ bytes)?

yes -> pass by value.

Is it a `std::string`, `std::vector`, `std::map`, or other heap-backed type?

function reads it but does not change it -> `const&`.

function must modify the caller's value -> `&` (and name it so).

function needs its own copy to mutate -> by value.

 Live refactor: paycheck stage 2

Open `paycheck1.cpp` and `paycheck2.cpp` side by side. Stage 1 has a wall of inline month-arithmetic sitting in the middle of `main`. Stage 2 lifts that arithmetic into two functions:

```
int month_of_day(int day);    // which month is day-of-year `day` in?
int month_length(int month); // how many days in `month`?
```

Both take small `int` arguments by value. Compile both stages; feed both the same input. The output is **identical**. The refactor changed no behavior and bought two named ideas — and that is precisely what a good refactor looks like: same output, more legible code.

4.4 Returning values

4.4.1 Return by value is the default

```
std::vector<int> three_paycheck_months(int year, int anchor);
```

Yes — that returns an entire `std::vector` *by value*, and yes, it is fast. Modern C++ either *moves* the vector out (handing over its internal buffer rather than copying it) or *elides* the copy entirely. You do not pay for a deep copy of the elements. So **stop reaching for output parameters**. This signature tells the reader everything; the out-parameter version


```
void three_paycheck_months(int year, int anchor, std::vector<int>& out);
```

makes the caller declare a vector first, hides the real result behind a reference, and reads worse for no benefit.

4.4.2 Two return values? A struct or a std::tuple

When a function naturally produces two things, do not pick one to return and shove the other into a reference parameter. Return both. The quick way uses `std::tuple` and the structured bindings you met in Chapter 3:

```
#include <tuple>

std::tuple<int, int> minmax(const std::vector<int>& v);

auto [lo, hi] = minmax(scores); // structured bindings unpack the tuple
```

The `auto [lo, hi]` destructures the returned tuple into two named variables, exactly like Python's `lo, hi = minmax(scores)`. When the pair has a *meaning* that recurs across your program, give it a named struct instead — it reads better at every call site:

```
struct PaycheckSummary {
    int total_paychecks;
    std::vector<int> three_paycheck_months;
};

PaycheckSummary summarize(int year, int anchor);
```

A tuple is fine for a one-off internal pair; a named struct earns its keep when the same pair appears in many signatures.

4.4.3 void returns exist for a side effect

```
void print_summary(const PaycheckSummary& s);
```

A void function returns nothing, so it can only justify its existence by *doing* something — printing, writing a file, mutating a parameter. The name must declare that effect: `print_summary`, not `summary`. If you catch yourself giving a void function a *noun* for a name, you almost certainly wanted it to **return** that noun instead.

4.4.4 “I might not have an answer” — `std::optional`

What does a function return when the answer might legitimately not exist?

```
#include <optional>

std::optional<int> first_three_paycheck_month(int year, int anchor);
```

`std::optional<int>` is “an `int`, or nothing.” It beats the old habit of returning a sentinel like `-1` or `0` to mean “not found,” because the type **forces the caller to check** before using the value — there is no way to accidentally treat the “nothing” case as a real month number. We return to `std::optional` in the error-signaling section below, and again later in the course.

4.5 Pure vs effectful functions

A **pure function** returns the same output for the same input and changes nothing else in the world. `is_leap_year`, `month_length`, `twice`, `std::abs`, `std::sqrt` — all pure. You can test a pure function with one input and one expected output; you can call it from anywhere; you can run it at compile time (that is what `constexpr` does); you can reason about it in isolation forever.

An **effectful function** does something beyond computing its return value — it prints, writes a file, mutates a parameter, updates a global, or talks to the network. `std::cout << x`, `std::sort` (it reorders its argument), `print_summary` — all effectful.

The discipline is simple to state and hard to keep: **prefer pure functions, and when a function must have an effect, make that effect its whole declared purpose, not a hidden extra.** Effects you cannot see from the signature — the `int compute(int n)` that secretly prints, the getter that secretly mutates — are the single most common source of “but *why* isn’t this working” sessions.

4.6 Globals: don’t (with one exception)

Here is a small program kept verbatim from a prior offering, because it lands every year:

```
// DO NOT FOLLOW THIS MODEL
int x;                                // a global variable
void twice() { x = x * 2; }           // operates on the global

int main() {
    x = 21;
    twice();
}
```

```
std::cout << x;           // prints 42
}
```

Walk through what is wrong with `twicex`:

- **Over-specialized.** It only works on the one global named `x`. You cannot use it to double anything else.
- **Hides code without adding clarity.** The name does not say what it does or what it touches.
- **Has an invisible side effect.** The signature `void twicex()` promises nothing and delivers a mutation of `x`.
- **Untestable in isolation.** You cannot ask “what does `twicex` return for input 7?” — it takes no argument and returns nothing.

The pure version fixes every one of these at once:

```
int twice(int n) { return n * 2; }

int main() {
    int x = 21;
    x = twice(x);
    std::cout << x;
}
```

Same behavior at the call site. The function is now pure, named for what it does, reusable on any `int`, and trivially testable.

i When *is* a global acceptable?

For genuinely program-wide **constant configuration** — and prefer `constexpr` so the value is fixed at compile time:

```
constexpr int kBoardSize = 8;           // a chessboard is 8x8, always
constexpr double kPi = 3.14159265;
```

Even then, tuck it inside a namespace or a class when you can. Almost nothing else justifies a global. Note that `std::cout`, `std::cin`, and `std::cerr` are globals — they are the unavoidable side-effect surface of the standard library — but you name them explicitly at every single use (`std::cout << ...`), never hidden. That visibility is the discipline.

4.7 A gallery of good and bad functions

Read each of these as a signature only. Decide what is good or bad *before* reading the verdict.

4.7.1 Good

```
double euclidean(const Point& p);           // pure, const&, returns
std::optional<int> parse_int(const std::string& s); // failure is in the type
bool is_leap_year(int year);               // the signature IS the docs
std::vector<int> three_paycheck_months(int year, int anchor);
```

Each one tells you what it takes, what it gives back, and whether it can fail — all without a body.

4.7.2 Bad

```
void process(Data& d);           // "process"? what does it DO?
int do_it(int x, int y, int z, int w, int q); // do what? what are these five?
void save_and_email_and_log(const Report& r); // three jobs, one name
int compute(int n);              // compute *what*?
void update();                   // update *what*?
```

4.7.3 The loudest smell: “and” in the name

`save_and_email_and_log` is three functions wearing one hat. The fix is not a better name — it is **three functions**:

```
void save(const Report& r);
void email(const Report& r);
void log(const Report& r);
```

Now each is independently testable, independently reusable, and can fail independently. The caller composes them:

```
save(r);
email(r);
log(r);
```

— and only wraps them under one name if the *composition itself* is a meaningful operation worth naming.

This is the **do-one-thing** principle, and “and” in a function name is its most visible violation. The other three smells to watch for: a function that is **too long** (more than ~20 lines is a warning sign — we make this precise in Chapter 17), a function with **hidden side effects** (covered above), and **the same parameter dragged through many functions** (the shared-parameter smell, which closes this chapter).

4.8 The library-check reflex

Before you write a loop that summarizes a container, stop and ask: **is this already in the standard library?** It very often is — written, tested, and faster than what you would write by hand.

You catch yourself writing...	The library already has
Sum a <code>vector<double></code>	<code>std::accumulate (<numeric>)</code>
Find the biggest element	<code>std::max_element (<algorithm>)</code>
Count elements matching a condition	<code>std::count_if (<algorithm>)</code>
Check if <i>any</i> element matches	<code>std::any_of (<algorithm>)</code>
Sort	<code>std::sort (<algorithm>)</code>
Find a value	<code>std::find, std::find_if</code>

The contrast, concretely:

```
#include <numeric>

// Hand-rolled.
int total = 0;
for (int p : paydays) {
    total += p;
}

// Library version.
int total = std::accumulate(paydays.begin(), paydays.end(), 0);
```

Both are correct. The hand-rolled loop is fine — *building* one yourself is how you learn what `std::accumulate` even does. But the library version is what a working C++ engineer reaches for, and recognizing when your loop is really a named algorithm in disguise is a genuine skill. We make algorithms the spine of Chapter 6; for now, just plant the reflex. **Build it to learn it, use it to ship it.**

💡 AI touchpoint — name and critique a function

Two low-risk, high-value uses of an LLM live right here in the function-design loop.

First, *naming*: paste a function body and ask, “*This function does X. Suggest three clearer names and say which is best and why.*” You stay the judge; the assistant widens the menu. This is the **AI as pair programmer** role.

Second, *spec clarification*: paste only the signature — `std::vector<int> three_paycheck_months(int year, int anchor)` — and ask, “*What edge cases should I test for this function?*” Then **you** write the tests it suggests. (Leap years? `anchor` past the end of the year? An `anchor` of zero?) This is the **AI as spec clarifier** role, and it is exactly the kind of thing the course’s AI policy expects you to use openly and disclose.

A worthwhile live exercise: hand an assistant one of the *bad* functions from the gallery above and ask it to refactor. Then critique what it produces — it will often improve the name and split the responsibilities, but you should be able to say *whether it went far enough*.

4.9 Recursion

A **recursive function** is one that calls itself. The textbook first example:

```
int factorial(int n) {
    if (n <= 1) {
        return 1;           // base case
    }
    return n * factorial(n - 1); // recursive case
}
```

Every recursive function has the same two pieces, and you should be able to point at both before you trust the code:

- A **base case** — a value of the argument for which the answer is immediate, with no further recursion. `factorial(1)` is 1, full stop. Without a base case the recursion never stops, the calls pile up forever, and the program dies with a **stack overflow**.
- A **recursive case** — reduce the problem to a *smaller* instance of itself and combine. `factorial(n)` is `n` times `factorial(n - 1)`, a strictly smaller subproblem, guaranteed to march toward the base case.

What is physically happening: each call gets its own private copy of the parameters and local variables — its own **stack frame**. Call `factorial(4)` and four frames stack up before any of them return:

`factorial(4)`

```

|- 4 * factorial(3)
|   |- 3 * factorial(2)
|   |   |- 2 * factorial(1)
|   |   |   \- 1                <- base case, returns immediately
|   |   \- 2 * 1 = 2
|   \- 3 * 2 = 6
\ - 4 * 6 = 24

```

The frames build up on the way *down* to the base case and collapse on the way *back up*, each multiplying its *n* into the result handed back from below. You will watch this stack grow and shrink, frame by frame, under the debugger in Appendix C. For now, draw the tree and trust the mechanism.

4.9.1 Fibonacci and the cost of recursion

The Fibonacci sequence is the classic cautionary tale:

```

int fib(int n) {
    if (n < 2) {
        return n;
    }
    return fib(n - 1) + fib(n - 2);
}

```

It looks just as clean as `factorial`, and for small *n* it works fine. Then try `fib(40)`: it takes a couple of seconds. Try `fib(50)`: go to lunch and it still will not be done. Something has gone badly wrong, and the structure of the recursion is why.

Each call to `fib` spawns *two* more calls, which each spawn two more. The recursion tree branches, and the total number of calls is roughly 2^n — the work doubles with every increment of *n*. Worse, it is *wasted* work: computing `fib(5)` recomputes `fib(3)` from scratch several times over, each time walking all the way back down to the base case. (This is the “Schlemiel the painter” pattern — each step starts over from the beginning instead of remembering where it left off.)

There are three ways out, in increasing sophistication:

```

// 1. Iteration -- carry the last two values forward in a loop.
int fib(int n) {
    int a = 0;
    int b = 1;
    for (int i = 0; i < n; i++) {
        int next = a + b;

```

```

    a = b;
    b = next;
}
return a;
}

```

This is *linear* — one pass, n additions, no re-computation — and it is the right answer here.

```

// 2. Memoized recursion -- cache each fib(k) the first time you compute it,
//    so the second request is a lookup. Needs std::unordered_map (later).
// 3. Closed form -- Binet's formula. That is mathematics, not programming.

```

Recursion is a tool, not a virtue

The lesson of `fib` is not “recursion is slow.” It is that **recursion fits some problems and fights others**. Reach for it when the problem is *naturally* recursive — walking a tree, traversing the linked structures of Chapter 11, the binary search tree of Chapter 14 — where the recursive shape of the code mirrors the recursive shape of the data. Do not reach for it just because you can. For a flat sequence like a year of days, **iteration wins**, and the hand-rolled loop is the right tool.

4.9.2 Recursion and iteration are duals

Every recursive function can be rewritten as a loop, and every loop can be rewritten as a recursion. The choice is one of style and cost:

Recursion wins	Iteration wins
Trees and recursive data structures	Linear, flat traversals
The recursive code mirrors the problem (BST insert)	Performance matters and you can’t afford stack growth
The recursive version simply reads more clearly	The recursion would get very deep (>~10,000 frames risks overflow)

For the paycheck program — a year is a flat run of days — iteration is the clear choice, and the stage-2 loop you wrote earlier is exactly right. We meet recursion again where it genuinely belongs, in Chapter 11 and Chapter 14.

4.10 Splitting across files: headers and translation units

So far: one `.cpp`, one program. As soon as you want to **reuse** a function across more than one program — without copy-pasting it — you need to split your code across files. The C++ answer is a pair: a **header file** (`.h`) holding declarations, and an **implementation file** (`.cpp`) holding the bodies.

The worked example is the `Point` struct from Chapter 3, extracted into its own reusable unit.

4.10.1 `Point.h` — the declaration (the *what*)

```
#pragma once    // include guard: "splice this header in at most once per file"

struct Point {
    double x;
    double y;
};

double distance(const Point& a, const Point& b);
```

Notice the header declares the function but does **not** define it — that trailing semicolon, with no body, is a *declaration*. It is a promise: “somewhere, a function with exactly this signature exists.”

4.10.2 `Point.cpp` — the definition (the *how*)

```
#include "Point.h"
#include <cmath>

double distance(const Point& a, const Point& b) {
    double dx = a.x - b.x;
    double dy = a.y - b.y;
    return std::sqrt(dx * dx + dy * dy);
}
```

This file `#includes` its own header (so the compiler can check the definition against the declaration) and then supplies the body that keeps the promise.

4.10.3 main.cpp — the caller

```
#include "Point.h"
#include <iostream>

int main() {
    Point home{0.0, 0.0};
    Point bu{2.0, 3.0};
    std::cout << distance(home, bu) << "\n";
}
```

main.cpp includes only the *header* — it needs the contract, not the implementation. It never sees Point.cpp's source at all.

4.10.4 Building it: three steps, by hand

```
$ g++ -std=c++20 -c Point.cpp -o Point.o    # compile Point.cpp -> Point.o
$ g++ -std=c++20 -c main.cpp -o main.o      # compile main.cpp -> main.o
$ g++ Point.o main.o -o demo                # link both .o files into one program
```

The `-c` flag means *compile only, do not link* — produce an object file (`.o`) and stop. Three observations make this whole machinery click:

1. The compiler compiles **each .cpp independently**. Each one is a **translation unit** (the same word from Chapter 2). Point.cpp never sees main.cpp's source and vice versa. They agree with each other *only* through the shared header.
2. The header is the **shared contract**. Both files `#include "Point.h"`, and that is exactly how the call to `distance` in `main` is guaranteed to match the definition in `Point.cpp`. The linker resolves the call to the definition at the final step.
3. `#pragma once` prevents the header from being spliced in twice within a single translation unit (which would define `struct Point` twice — an error). Older code uses `#ifndef POINT_H / #define POINT_H / ... / #endif` *include guards* to the same end; both work, `#pragma once` is shorter, and the course allows either.

i What goes where

Header (<code>.h</code>) — the public face	Implementation (<code>.cpp</code>) — kept private
<code>struct</code> / <code>class</code> definitions	function bodies
function declarations (signatures)	helpers used only inside this file
shared type aliases	<code>static</code> file-local functions

templates (whole body — Chapter 15)

anything that should not be public

The rule of thumb: **the header is your set of promises; the .cpp is how you kept them.** Anyone who `#includes` your header is reading the public API and nothing more. This is the team-scale discipline of Chapter 13 in seed form — it is what lets three people work on one program without colliding.

⚠ Every include and link is a trust decision

A small security through-line worth one sentence here: every `#include "foo.h"` splices someone else's code into *your* translation unit, and every linked `.o` runs inside *your* process. Trusting a header or a library is the same kind of decision as trusting a `pip install` — you are running their code with your privileges. We return to this in Chapter 13.

We do the build mechanics here by hand to make the three steps visible. In real projects you drive them with a build system rather than typing `g++` three times; that is the job of Appendix A.

4.11 Overloading and default arguments, briefly

Two smaller language features round out the function toolkit.

Overloading lets several functions share one name as long as their parameter lists differ:

```
int    euclidean(int x, int y);
double euclidean(double x, double y);
double euclidean(const Point& p);
```

The compiler picks the best match at each call site based on the argument types. Use it when the *operation is conceptually the same* and only the types differ — the standard library's `std::sin` is overloaded for `float`, `double`, and `long double` precisely so callers do not have to care. Do **not** overload functions that actually do *different* things; give those different names.

Default arguments let a parameter be omitted, falling back to a default:

```
void shift(int& value, int n = 1);

shift(score);           // n defaults to 1
shift(score, 5);        // n is 5
```

This lets you grow a function's API without breaking existing callers — add a new defaulted parameter, and old call sites keep compiling. Use it sparingly, though: a function with five defaulted arguments has five hidden behaviors, and that is its own kind of signature smell.

4.12 Signaling errors: three tools

When a function might not be able to produce an answer, you have three ways to say so, in rough order from worst to best for most situations:

1. **A sentinel return value** — `-1`, `0`, `false`, `""`. The old-school approach, and error-prone: nothing forces the caller to check, so the sentinel quietly flows downstream as if it were a real value.
2. **`std::optional<T>`** — “I will give you a `T`, or nothing.” The modern default. The type *forces* the caller to ask whether there is a value before using it.
3. **Throw an exception** — reserved for genuinely *exceptional* conditions, not ordinary control flow. It is heavyweight, and in this course you mostly meet it when a library function throws at *you* (`std::vector::at` throwing `std::out_of_range`, from Chapter 3).

`std::optional` is the one to reach for by default:

```
#include <optional>

std::optional<int> parse_int(const std::string& s);

auto maybe_n = parse_int(line);
if (maybe_n) {                                // is there a value?
    std::cout << "got " << *maybe_n << "\n";    // *maybe_n unwraps it
} else {
    std::cout << "not a number\n";
}
```

The `if (maybe_n)` asks “did we get a value?”; the `*maybe_n` unwraps it once you know it is there. The compiler will not let you skip the check and reach straight for the value, which is the entire point — the “no answer” case cannot be silently forgotten.

4.13 The shared-parameter smell — a bridge to classes

Step back and look at the four paycheck functions you wrote in this chapter, lined up:

```
bool        is_leap_year(int year);
int         month_length(int month, int year);
int         month_of_day(int day,   int year);
std::vector<int> three_paycheck_months(int year, int anchor);
```

Three of the four take `int year`. The fourth takes `year` plus an `anchor`, and inside its body it passes `year` along to `month_length` and `month_of_day` on every iteration:

```

for (int day = anchor; day <= (is_leap_year(year) ? 366 : 365); day += 14) {
    int month = month_of_day(day, year);
    // ... year, year, year ...
}

```

`year` is dragged through every call. That repeated parameter is a **smell** — the last of the four function smells from the gallery. It is the data trying to tell you something: *I belong to something*. The something is a **year’s calendar**, and a year’s calendar is a real thing in the world — it knows its own leap-ness, its own month lengths, its own day-to-month lookup. Right now that knowledge is scattered across four free functions that have to be *told* the year over and over.

The fix is to give that shared data a home. In Chapter 5 you will write:

```

class Calendar {
public:
    explicit Calendar(int year);
    bool is_leap() const;
    int month_length(int month) const;
    int month_of_day(int day) const;

private:
    int year_;
};

```

Now `year` *disappears* from every public function — it lives inside the `Calendar`, and the functions become **member functions** that already know which year they belong to. The caller writes:

```

Calendar c{2026};
if (c.is_leap()) { /* ... */ }
int m = c.month_of_day(105);

```

That is the punch line, and it is the central idea of the next chapter:

A class is what a group of related functions becomes when they share state.

The paycheck program — which started as month-arithmetic crammed into `main`, then became a handful of free functions in this chapter — becomes one of your first real classes in Chapter 5. The functions you wrote today did not go to waste; they are exactly the methods the `Calendar` will absorb.

4.14 What you should understand by the end of this chapter

- Reading a function **signature as a contract** — name, parameter types, return type, **const**-ness — and predicting behavior without the body
- Choosing the parameter-passing convention: **value** for small types, **const&** for large read-only ones, **non-const reference** only when the job is to modify the caller’s value and the name says so
- Returning **by value** as the default; using a **struct** or **std::tuple** (with structured bindings) for two results; using **std::optional** for “maybe”
- The distinction between **pure** and **effectful** functions, and why pure ones are cheaper to reason about and test
- The rule for **globals**: don’t, except **constexpr** program-wide constants
- The four function smells — too long, hidden side effects, repeated parameters, and “**and**” in the name — and the do-one-thing fix
- The **library-check reflex**: reach for **std::accumulate**, **std::max_element**, **std::count_if**, **std::any_of** before hand-rolling a summarizing loop
- Writing a **recursive** function (base case + recursive case), tracing its stack frames, knowing why naive **fib** is exponential, and rewriting it iteratively
- Splitting a reusable thing into a **.h/.cpp pair** with **#pragma once**, and compiling it as two translation units linked against a **main**
- **Overloading** and **default arguments** at recognition level
- The three error-signaling tools, and why **std::optional** beats a sentinel
- The **shared-parameter smell** — and why it is the data asking to become a class in Chapter 5

If you can read each of the four paycheck function signatures aloud and say what it takes, what it returns, and whether it can fail — and explain why **year** showing up in three of them is a smell — you are ready for Chapter 5.

5 Classes: encapsulation, RAII, and earning a class's existence

In Chapter 3 you wrote your first `struct` — a `Grade` that bundled a threshold and a letter so the two could not drift apart. That was the gentle introduction. A `struct` is a *bag of fields*: it solves the “these belong together” problem and nothing more. Anyone can still reach in and put nonsense into any field.

This chapter is where you write your first real `class`. It is, for most students in this course, the first time you have ever *defined* a class yourself — and that is true even if you have used Python classes for years, because Python classes and C++ classes solve overlapping problems with very different machinery. We will bridge from Python explicitly and repeatedly, and the bridge has some sharp drops: Python classes have no destructors, no real `const`, and a garbage collector that hides the entire question of *ownership* that this chapter is mostly about.

The course taught **Classes I** (`struct` → `class`, member functions, constructors, `this`) and **Classes II** (destructors, RAII, `const`-correctness, what makes a good class) in one combined session. This chapter follows suit and runs the whole arc in one flow, with one organizing question underneath all of it:

A class is a struct that earns the right to have rules. When does it earn it?

The answer comes at the end, in three parts. Everything before it is the machinery you need to understand the answer.

5.1 The smell that motivates a class

Recall the last note in Chapter 4: when you find yourself passing the *same cluster of parameters* into function after function — `withdraw(balance, account_number, ...)`, `deposit(balance, account_number, ...)`, `transfer(balance, account_number, ...)` — that repetition is a *smell*. The cluster of data wants to be one thing, and the functions that operate on it want to live next to it. A `struct` groups the data. A `class` groups the data *and* the operations *and* the rules that keep the data sensible. This chapter is the cure for that smell.

Here is the starting point — the `Point` type from a week-2 in-class demo, written as a plain `struct`:

```
// Point.h
#pragma once

struct Point {
    double x;
    double y;
};

double distance(const Point& a, const Point& b);
```

```
// Point.cpp
#include "Point.h"
#include <cmath>

double distance(const Point& a, const Point& b) {
    double dx = a.x - b.x;
    double dy = a.y - b.y;
    return std::sqrt(dx * dx + dy * dy);
}
```

This compiles. It runs. It is *fine* — for what it is. But read it, then break it on purpose:

```
Point p;           // p.x and p.y are uninitialized garbage
p.x = 1e30;
p.y = NAN;         // "not a number" - a perfectly legal double
double d = distance(p, p); // returns nan; nothing complained
```

If `Point` is supposed to *mean* “a real, finite location in the plane,” the struct cannot enforce that meaning. Nothing stops a client from default-constructing garbage, from storing `NAN`, from leaving a field unwritten. The struct has no opinion about what a valid `Point` is, because a struct cannot *have* opinions. That is the gap a class fills.

5.2 The one-sentence difference between struct and class

In Chapter 3 we said the only *language-level* difference between `struct` and `class` is the default access of members. That is literally true and worth restating precisely:

A class is a struct where the members are private by default — so the only way in is through functions you wrote.

Everything else in this chapter — member functions, constructors, **this**, **const** methods, destructors — is a *consequence* of that one fact. Once the data is private, you must provide a controlled way in (constructors and member functions), and once you control the way in, you can finally enforce what a valid object looks like.

Here is **Point** converted to a class. Read it against the struct above:

```
// Point.h (after conversion)
#pragma once

class Point {
public:
    Point(double x, double y);           // the only way to make one
    double distance_to(const Point& other) const;
    double x() const { return x_; }      // controlled read
    double y() const { return y_; }

private:
    double x_;
    double y_;
};
```

```
// Point.cpp (after conversion)
#include "Point.h"
#include <cmath>

Point::Point(double x, double y) : x_{x}, y_{y} {}

double Point::distance_to(const Point& other) const {
    double dx = x_ - other.x_;
    double dy = y_ - other.y_;
    return std::sqrt(dx * dx + dy * dy);
}
```

Four moves happened, and each one is a section of this chapter:

1. **private: data, public: interface** — the doorway is now controlled. The `x_` and `y_` members are private; only code *inside* the class can touch them. (The trailing underscore is a course convention for “this is a private member,” nothing more — the compiler does not care.)
2. **A constructor** — `Point p{3.0, 4.0};` is now the *only* way to make a `Point`. Default-construction of garbage is gone.
3. **Member functions** — `distance_to` belongs to the type and is called as `a.distance_to(b)`. Inside it, `x_` and `other.x_` are both reachable because we are *inside the class* — *we own the*

keys.

4. **const member functions** — the trailing **const** on `distance_to`, `x()`, and `y()` is a compiler-enforced promise: *calling me does not change the object.*

i **public:** and **private:** are sections, not per-member labels

The labels **public:** and **private:** introduce a *section* that runs until the next label — everything between them is public, everything after **private:** is private. This differs from Java, where each member carries its own modifier. By convention C++ classes put the **public interface first** (it is what a reader cares about) and the private data last. A **struct** with an explicit **private:** section and a **class** with an explicit **public:** section are the same thing built from opposite defaults.

5.2.1 How this maps to Python — and where it stops mapping

If you have written Python classes, three things will feel familiar and one will feel alien.

Familiar: a class groups data with the functions that act on it. Familiar: you call a method as `object.method(args)`. Familiar: there is a special function that runs when you make an object (`__init__` in Python; the constructor here).

Alien: **Python's private is a polite fiction.** A leading underscore in Python (`self._x`) is a *convention* that says “please don't touch this,” and nothing enforces it — any caller can still write `obj._x = 99`. C++'s **private** is enforced by the compiler: code outside the class that names a private member *does not compile*. This is the first of several places in this chapter where C++ turns a Python convention into a compiler-checked guarantee.

5.3 Constructors: the only way in

A **constructor** is a member function with the same name as the class and no return type. Its job is to take an object from “raw memory the compiler just set aside” to “a valid object that satisfies the class's rules.” It is the gatekeeper: after the conversion above, you cannot make a **Point** *without* running a constructor, and the constructor gets to decide what counts as a valid starting state.

```
Point::Point(double x, double y) : x_{x}, y_{y} {}
```

Read the pieces:

- **Point::Point** — “the constructor of class **Point**.” The **Point::** prefix says this definition lives in the **Point** class; the second **Point** is the constructor's name (same as the class).
- **(double x, double y)** — the constructor's parameters, exactly like any function.

- `: x_{x}, y_{y}` — the **member initializer list**. This is the part with no Python analogue and it matters. It says “initialize member `x_` from parameter `x`, and member `y_` from parameter `y`.” It runs *before* the constructor body.
- `{}` — the (here empty) constructor body. Any code here runs *after* the members are already initialized.

💡 Prefer the initializer list to assignment in the body

You could instead write:

```
Point::Point(double x, double y) {
    x_ = x;    // assignment in the body
    y_ = y;
}
```

For two `doubles` the difference is invisible. But the initializer-list form *constructs* each member directly with its value, while the body form *default-constructs* the member first and then *assigns* over it — two steps where one would do. For members that are expensive to default-construct (a `std::string`, a `std::vector`, another class), this is a real cost, and for members that *cannot* be default-constructed (a `const` member, a reference member, a class with no default constructor) the body form does not compile at all. The initializer list is the habit; reach for it by default. `clang-tidy`’s `cppcoreguidelines-prefer-member-initializer` check will nudge you.

The payoff is that the garbage you could create with the struct is now unreachable:

```
Point p;                // ERROR: no matching constructor (no default ctor)
Point p{3.0, 4.0};      // OK: x_ = 3.0, y_ = 4.0, both initialized
```

`Point p`; no longer compiles, because we declared a constructor that takes two arguments and never declared one that takes none. Declaring *any* constructor suppresses the compiler’s free default constructor. If you *want* a no-argument `Point` (at the origin, say), you say so explicitly:

```
Point() : x_{0.0}, y_{0.0} {}    // an explicit default constructor
```

5.3.1 Establishing an invariant in the constructor

So far the constructor just copies its arguments into members. Its real power is to *reject* arguments that would make an invalid object. Suppose we decide a `Point` must be finite — no NaN, no infinity. The constructor is where we enforce that:

```

#include <cmath>
#include <stdexcept>

Point::Point(double x, double y) : x_{x}, y_{y} {
    if (not std::isfinite(x_) or not std::isfinite(y_)) {
        throw std::invalid_argument{"Point coordinates must be finite"};
    }
}

```

Now there is *no way* to obtain a `Point` whose coordinates are not finite — the only door into the type runs this check. (We are using `throw` informally here; exceptions get their proper treatment later. For a course that has not met exceptions yet, `assert(std::isfinite(x_) and std::isfinite(y_))`; from `<cassert>` is the lighter-weight alternative, and `std::optional<Point>` returned from a static `make()` function is a third. The activity block asks you to compare these three with a neighbor — there is no single right answer, only tradeoffs.)

That word — **invariant** — is the heart of the chapter, and we return to it in Section 5.8 once we have the rest of the machinery.

5.4 Member functions and the implicit `this`

A **member function** is a function declared inside the class. It is called on a particular object with the `.` operator, and inside it the object's members are directly nameable:

```

double Point::distance_to(const Point& other) const {
    double dx = x_ - other.x_;           // x_ means this->x_
    double dy = y_ - other.y_;
    return std::sqrt(dx * dx + dy * dy);
}

```

When you write `x_` inside a member function, the compiler quietly rewrites it as `this->x_`. **this** is a hidden pointer, passed invisibly to every member function, that points at the object the function was called on. In `a.distance_to(b)`, inside the call **this** points at `a`, so `x_` is `a`'s `x_`, and `other` is `b`.

If you have written Python methods, **this** is exactly Python's `self` — with one difference: in Python you write `self` out explicitly as the first parameter and use it on every member access (`self.x`). In C++ **this** is *implicit*: it is not in the parameter list and you do not normally write it. You *can* — `this->x_` is legal and occasionally necessary — but `x_` alone is the idiom.

You can see **this** is a real per-object pointer: a member function that prints **this**, called on two different `Points`, prints two different addresses. Each object is a distinct lump of memory, and **this**

is how one shared copy of the member-function code knows *which* lump it is working on. That is the whole trick: one copy of the function code, a different **this** per call.

i `other.x_` from inside the class — privacy is per-*class*, not per-*object*

Look again at `other.x_` in `distance_to`. We are reaching into the *private* member of a *different* `Point` object. That is allowed: access control in C++ is per-**class**, not per-**object**. Any `Point` member function may touch the private members of any `Point` — your object can read its sibling’s `x_`. The privacy boundary is “is this code part of the `Point` class?”, not “is this the same `Point`?” This is the same rule as Python’s name-mangling vs. convention, except again, here it is enforced.

5.5 *const* member functions and *const*-correctness

The trailing `const` on `distance_to`, `x()`, and `y()` is the single most important piece of C++ discipline in this chapter, and it is the one with the sharpest break from Python — because **Python has no `const` at all**.

A `const` member function is a **promise to the compiler that calling it will not modify the object**. Inside a `const` member function, `this` is a pointer-to-`const`, and any line that tries to change a member is a compile error:

```
double Point::x() const {
    x_ = 0.0;           // ERROR: assignment of member 'x_' in read-only object
    return x_;
}
```

The compiler refuses. That refusal is the feature. A getter that only reports state should always be `const`; a mutator that changes the object must *not* be `const`. Tag each member function as you write it.

5.5.1 Why *const*-correctness is not optional

It would be easy to treat `const` as decoration you sprinkle on for style points. It is not, because **`const` is viral through references**, and the course’s preferred way to pass objects around — by `const&`, the cheap, safe way from the functions chapter — depends on it.

You can only call a `const` member function on a `const` object or a `const&`. So the moment any code holds your object by `const&` and tries to ask it a question, *that question’s member function must be `const`*. Concretely:

```
double total_distance_from_origin(const std::vector<Point>& points) {
    Point origin{0.0, 0.0};
    double total = 0.0;
    for (const auto& p : points) {        // p is a const reference to each Point
        total = total + p.distance_to(origin);    // requires distance_to to be const
    }
    return total;
}
```

`p` is a `const Point&` because we wrote `const auto&` in the range-for (Chapter 3). If `distance_to` were *not* marked `const`, this loop would not compile — the compiler would say something like “passing ‘const Point’ as ‘this’ argument discards qualifiers.” That error message is one of the most common a beginner hits, and now you know exactly what it means: you called a non-`const` method on a `const` object. The fix is almost always “the method should have been `const` and you forgot the keyword.”

⚠ The `const`-incorrect code you will find online is wrong; the linter is right

A large fraction of casually-written C++ on the internet skips `const` on methods that deserve it, because the code happened to work in a context where no one held the object by `const&`. It still compiles; it is still wrong, in the sense that it cannot be used by `const`-correct callers and will break the moment someone tries. The course `.clang-tidy` enables the checks that flag missing `const`, and `const`-correctness is one of the strongest tools you have for letting the compiler catch your *future* mistakes. Treat a missing-`const` lint warning as a real defect, not a nag.

The same `const` keyword shows up on parameters, for the same reason. `void greet(const Point& p)` says “I will look at `p` but not modify it” — it documents intent *and* lets the caller pass a temporary. You met `const&` parameters in the functions chapter; here it connects to methods: a `const&` parameter can only have its `const` methods called.

5.6 RAII: acquire in the constructor, release in the destructor

`Point` is the easy case: it owns nothing but two doubles. Both doubles live *inside* the `Point`, the `Point` lives on the stack, and when the `Point` goes out of scope its doubles vanish with it. There is nothing to clean up.

The interesting case is when an object **owns something that lives elsewhere** — a chunk of heap memory, an open file, a network connection, a lock on a shared resource. These things are not automatically reclaimed when the owning object disappears. Someone has to release them. The question is *who*, and *when*.

5.6.1 The C answer, and why it is a trap

In C, you release a resource by remembering to call the matching teardown function — `fclose` for an `fopen`, `free` for a `malloc`, `unlock` for a `lock` — on *every* path out of the scope that acquired it. Read that carefully: *every* path. The happy path, yes. But also the path where a check five lines later does an early `return`. Also the path where an exception is thrown. Also the path that some future maintainer adds in 2027 with a new `return` they did not realize needed cleanup. Miss one path and you leak the resource — a file left open, memory never returned, a lock held forever.

Python papers over the memory case with a garbage collector and the file case with `with` blocks, so you may never have *thought* about deterministic cleanup. C++ does something more powerful than either, and it is the central idea of resource management in the language.

5.6.2 The destructor

Every C++ class has a **destructor**, written `~ClassName()`. The compiler calls it *automatically* when an object’s lifetime ends:

- a local (stack) object goes out of scope — the `}` at the end of its block;
- a heap object is `deleted`;
- a `std::vector` element is erased, or the vector reallocates;
- the program ends, for globals and `statics`.

If you write no destructor, the compiler generates one that destroys each member in turn — and for a class like `Point`, made of plain `doubles`, that generated destructor does exactly the right thing (nothing, really — the `doubles` just go away). You write your *own* destructor only when the object owns a resource that will not clean itself up.

This is the Python break worth stating flatly: **Python classes have no destructor you can rely on.** Python’s `__del__` exists but runs whenever the garbage collector gets around to it — which might be much later, might be at interpreter shutdown, might (in a reference cycle) be never. C++’s destructor runs at a *precise, predictable moment*: the closing brace of the scope. That determinism is what makes the next idea work.

5.6.3 RAI, the central idea

RAI — Resource Acquisition Is Initialization. (Stroustrup, 1984.) Acquire the resource in the constructor; release it in the destructor.

Tie the resource’s lifetime to the object’s lifetime. Because the language *guarantees* the destructor runs when the object goes out of scope — on the happy path, on an early `return`, on an exception, on a path nobody has written yet — the release is automatic and happens *exactly once*. You never write the cleanup call. The scope’s closing brace *is* the cleanup call.

5 Classes: encapsulation, RAII, and earning a class's existence

Here is the demo from class — a tiny class that owns an open file:

```
// l6_owningfile.cpp
#include <cstdio>
#include <iostream>

class OwningFile {
public:
    explicit OwningFile(const char* path) : f_{std::fopen(path, "w")} {
        std::cout << "[ctor] opened " << path << " (FILE* = " << f_ << ")\n";
    }

    ~OwningFile() {
        if (f_) {
            std::fclose(f_);
            std::cout << "[dctor] closed (FILE* = " << f_ << ")\n";
        }
    }

    void write_line(const char* s) {
        std::fputs(s, f_);
        std::fputc('\n', f_);
    }

private:
    std::FILE* f_;
};

int main() {
    OwningFile log{"l6_demo.log"};          // ctor runs here: file opened
    log.write_line("hello classes II");
    return 0;                               // dtor runs here: file closed, automatically
}
```

Compile and run it:

```
$ g++ -std=c++20 -Wall -Wextra -o owningfile l6_owningfile.cpp
$ ./owningfile
[ctor] opened l6_demo.log (FILE* = 0x...)
[dctor] closed (FILE* = 0x...)
```

Watch the order: the constructor message, then the work, then — with no `close()` anywhere in

`main` — the destructor message. The file got closed because `log` went out of scope at `return 0;`, and going out of scope *is* the trigger for the destructor.

i `explicit` on a one-argument constructor

The `explicit` keyword on `OwningFile(const char* path)` blocks C++ from *implicitly* converting a `const char*` into an `OwningFile` behind your back — so `OwningFile log = "file.txt";` and silent conversions in function calls are disallowed; you must write `OwningFile log{"file.txt"};` and mean it. Marking single-argument constructors `explicit` is a course default: it prevents a class of surprising conversions. `clang-tidy`'s `google-explicit-constructor` check enforces it.

5.6.4 The destructor cascade on scope exit

Now the part that makes RAII feel like magic rather than bookkeeping. Put two owning objects in nested scopes and watch when each one's destructor fires:

```
int main() {
    OwningFile outer{"outer.log"};
    {
        OwningFile inner{"inner.log"};
        inner.write_line("inside the block");
    } // <-- inner's destructor fires HERE, at the inner }
    outer.write_line("after the block");
    return 0;
} // <-- outer's destructor fires HERE, at the outer }
```

The output:

```
[ctor] opened outer.log
[ctor] opened inner.log
[dtor] closed inner.log    <-- at the inner }
[dtor] closed outer.log    <-- at the outer }, last
```

Two rules to take away:

1. **Destructors fire at the closing brace of the object's scope** — `inner` dies at the inner `}`, well before `main` ends.
2. **Within a scope, destructors run in reverse construction order** — last built, first torn down. If you construct `a` then `b` then `c` in one block, they are destroyed `c`, `b`, `a`. This is the same last-in-first-out discipline as the call stack itself, and it is not an accident: local objects live *on* the stack, and unwinding the stack tears them down in reverse. You will watch this happen frame by frame in a debugger later (Appendix C).

5.6.5 You have been using RAI since week 1

Here is the punchline that recontextualizes everything you have already written. `std::vector` and `std::string` are **RAII classes**. Each one owns a heap buffer: it acquires the buffer as it grows and releases it in its own destructor. When a `std::vector<Point>` goes out of scope, it destroys every `Point` and frees its storage — and you never wrote a line of cleanup.

```
{
    std::vector<Point> points(1000);    // acquires heap storage in its constructor
    // ... use it ...
}
```

// its destructor frees everything, automatically

This is why `std::vector` has no `free()` method and `std::string` has no `close()`: there is nothing for *you* to call, because the destructor already calls it. The same is true of the smart pointer you meet in Chapter 10 — a `std::unique_ptr<T>` is RAI for a single owned heap pointer: it acquires at construction and `deletes` in its destructor, which is precisely why the course says “use `unique_ptr`, never raw `new/delete`.” And it is what makes the linked structures of Chapter 14 clean: a node that holds a `std::unique_ptr<Node>` `next` destroys its whole tail by cascade when it dies, no manual traversal required.

Class	Acquires (in constructor)	Releases (in destructor)
<code>std::vector<T></code>	heap memory as it grows	frees it
<code>std::string</code>	heap memory for characters	frees it
<code>std::unique_ptr<T></code>	a heap pointer	<code>deletes</code> it
<code>std::lock_guard<std::mutex></code>	locks the mutex	unlocks it
<code>std::ofstream</code>	opens the file	closes it

That last row is the kicker. `std::ofstream` is `OwningFile`, written correctly and shipped in the standard library — it owns an output file and closes it in its destructor. We built `OwningFile` to *see* what RAI does; in real code you would never write it, because the library already did.

5.7 The Rule of Zero

The `OwningFile` demo raises an uncomfortable question. We hand-wrote a destructor that closes a `FILE*`. What about *copying* an `OwningFile`? If you copy one, both copies hold the same `FILE*`, both destructors run, and the file gets closed *twice* — a bug. To handle that correctly you would have to write not just the destructor but also a copy constructor, a copy assignment operator, and (in modern C++) a move constructor and move assignment. Five functions, all of which must be correct, all of which are easy to get subtly wrong. That is the **Rule of Five** (historically the **Rule**

of Three, before move semantics): *if you write one of the special resource-management functions, you almost certainly need all of them.*

The way to win this game is to **refuse to play it**:

Rule of Zero. If your class’s members are all self-managing — plain values, `std::vector`, `std::string`, `std::unique_ptr`, every one of them already RAI — then write *no* destructor, no copy constructor, no assignment operator. The compiler-generated defaults are correct, because each member already knows how to destroy, copy, and move itself.

This is the modern default, and it is what almost every class you write in this course should be. `Point` is a Rule-of-Zero class: it owns nothing but two `doubles`, so you write zero special functions and it is *complete*. If you ever find yourself wanting to hand-write a destructor, the first question is not “how do I write the other four?” but “can I replace the raw resource with a RAI member so I need none of them?” — wrap the raw `FILE*` in a `std::ofstream`, wrap the raw `new[]` in a `std::vector`, wrap the raw pointer in a `std::unique_ptr`. Push the resource management down into a library type that already got it right, and your class goes back to Rule of Zero.

The opener for this lecture put it as a one-liner worth remembering: **the best destructor is the one you didn’t have to write.**

💡 When you genuinely must write a destructor

Rule of Zero is the default, not an absolute. You write a hand-rolled destructor (and then the rest of the Rule of Five) exactly when you are *wrapping a raw resource that has no RAI type yet* — building the very `std::ofstream`-like wrapper that everyone else then gets to use under Rule of Zero. That is real, legitimate, careful work, and it is rare. In this course you will do it once or twice, deliberately, with the full Rule of Five in front of you — never by accident because you reached for raw `new` out of habit.

5.8 Invariants: the rule a class keeps

We can now name the idea that has been lurking since the constructor section. An **invariant** is a property of an object that is *always true* between operations — established by the constructor and preserved by every member function. It is the *meaning* the class enforces.

- For a finite `Point`: `x_` and `y_` are always finite. The constructor checks it; no member function ever stores a non-finite value, so it stays true forever.
- For a `Date`: the month is always in `1..12`, the day is always valid for that month. The constructor rejects `Date{2026, 13, 5}`; no setter is allowed to break the rule.
- For an `Email`: the stored string always contains an `@`. Construction validates it; there is no way to end up with an `Email` that has no `@`.

This is the deep payoff of making data **private**. Because the *only* way to change the object is through member functions you wrote, and every one of those functions is written to keep the invariant true, *calling code can rely on the invariant without re-checking it*. A function that receives a **Date** does not have to ask “is the month in range?” — it cannot be otherwise, because no invalid **Date** can exist. The invariant is enforced once, at the boundary, and trusted everywhere inside.

There is a clean way to think about this if you have seen induction or recursion: an invariant is a **base case for the state of your program**. The constructor establishes it (the base case holds); every member function preserves it (each step keeps it holding); so it holds for the object's entire lifetime by induction. The same invariant idea returns with teeth in Chapter 14, where the binary-search-tree property (“everything in the left subtree is smaller, everything in the right is larger”) is an invariant that *every* insert and delete must preserve, and the whole correctness of the structure rides on it.

A **struct** cannot have an invariant in this enforced sense, because its data is public — anyone can break any property at any time. That is the line between the two: a **struct** is a bag of fields with no rules; a **class** is a bag of fields *plus a promise that some rule always holds*.

5.9 What makes a class earn its existence

We can finally answer the organizing question. Not every cluster of data should become a class. A class carries costs — a constructor to write, an interface to design, encapsulation that a caller must go through — and those costs are only worth paying when the class buys you something a **struct** plus free functions cannot. A class **earns its existence** when at least one of these three is true:

1. **It enforces an invariant** that a **struct** cannot. (Finite **Point**, month-in-range **Date**, @-containing **Email**.) The class exists to make a bad state *unrepresentable*.
2. **It owns a resource** that needs deterministic cleanup. (**OwningFile**, **std::vector**, **std::unique_ptr**, **std::lock_guard**, **std::ofstream**.) The class exists to tie a resource's lifetime to an object's lifetime via RAII.
3. **It bundles state with the operations on that state** in a way no free function can match — typically because the operations share a *non-trivial* representation, or because the public interface deliberately *hides* that representation so it can be changed later without touching callers.

If **none** of those apply, the right answer is a **struct** plus free functions — or just a free function. A **Grade** (Chapter 3) is correctly a **struct**: it has no invariant beyond “these two fields belong together,” owns no resource, and the function that uses it (turn a score into a letter) is happier as a free function. Forcing it into a class would be ceremony with no payoff.

⚠ The salt-shaker trap

The lecture opened with xkcd #974, “The General Problem” — a person asked to pass the salt instead builds a general-purpose automated condiment-passing system, and dinner goes cold. Student code is full of this: elaborate class hierarchies, interfaces, and abstractions built for a generality that never arrives. A class that does not enforce an invariant, does not own a resource, and does not hide a non-trivial representation has not earned its existence — it is the salt shaker, automated for a use case that never came. When you reach for `class`, be able to *name* which of the three reasons applies. If you cannot, write the `struct`.

A related discipline, and a deliberate course choice: **the lever for managing complexity in this course is composition plus invariants, not deep inheritance**. You will see `class Derived : public Base` exactly twice all term, in a compact treatment. The overwhelming majority of good design here is *small, single-purpose classes that each earn their existence and combine by containing one another* — a `Game` that holds a `Board` that holds `Cells` — rather than tall trees of base and derived types. When a Python or Java instinct tells you to reach for inheritance, reach for composition first.

5.10 AI touchpoint — “does this class earn its existence?”

This chapter’s judgment call — *is this type pulling its weight, or is it the salt shaker?* — is exactly the kind of design question an AI assistant is good at sharpening, precisely because it has no single correct answer.

Take a class you wrote in the activity block (or `Point` itself) and paste it into an AI assistant with this prompt: *“Here is a small C++ class. For each of these three tests, tell me whether it applies: (1) does it enforce an invariant a struct couldn’t, (2) does it own a resource needing cleanup, (3) does it hide a non-trivial representation? If none apply, rewrite it as a struct plus free functions.”* Read the answer critically — the model will sometimes manufacture an “invariant” that is really just a default value, and you should be able to catch that. Then push back: *“Could this be a plain function instead of a class?”*

A second, sharper use: paste a class that hand-writes a destructor and ask *“does this class obey the Rule of Five correctly, and could it be rewritten under the Rule of Zero?”* The model will usually spot the missing copy constructor or suggest the `std::unique_ptr/std::vector` member that makes the destructor disappear. This is the **AI as reviewer** role from the AI Tools chapter — you keep the judgment; the model widens the set of options you are judging among.

5.11 What you should understand by the end of this chapter

- The single language-level difference between **struct** and **class** (default access) and the cultural convention layered on top (bag-of-fields vs. enforces-a-rule)
- That **private** in C++ is compiler-enforced, unlike Python's underscore convention
- Writing a **constructor**, including the member initializer list (`: x_{x}`) and why it beats assignment in the body
- How a constructor **establishes an invariant** by rejecting bad arguments
- **Member functions** and the implicit **this** (= Python's **self**, but hidden)
- **const member functions** and const-correctness: what the trailing **const** promises, why it is viral through **const&**, and how to read the “discards qualifiers” error
- The **destructor** (`~ClassName()`), when the compiler calls it, and why Python has no reliable equivalent
- **RAII** as the central resource-management idea: acquire in the constructor, release in the destructor, cleanup guaranteed on every path
- The **destructor cascade** on scope exit — at the closing brace, in reverse construction order
- That `std::vector`, `std::string`, `std::unique_ptr`, and `std::ofstream` are all RAII types you have already been relying on
- The **Rule of Zero** (and why the Rule of Five is a trap to avoid by composing RAII members)
- What an **invariant** is and why private data is what lets callers trust it
- The three-part test for whether a class **earns its existence** — and the salt-shaker trap when none of the three apply

If you can take the `Point` struct, convert it to a const-correct class with an invariant-enforcing constructor, *and* explain in one sentence which of the three reasons makes it worth being a class at all, you are ready for the STL work in Chapter 6.

6 Loop hygiene, “name your loop”, and STL algorithms

By now you can write a loop. Chapter 3 gave you the three idioms — range-based `for`, indexed `for`, iterator-based — and you have used them to walk a `std::vector`, sum it, search it, find the biggest thing in it. This chapter is not about new loop *syntax*. It is about the loops you already know how to write, and a claim that sounds strange the first time you hear it:

Most of the loops you are about to write should not exist. They were already written, decades ago, by experts, and they live in the standard library under names like `std::accumulate` and `std::max_element`. Your job is to learn to *recognize* them.

We get there in three moves. First, **loop hygiene** — the habits that make a hand-written loop correct and readable. Second, “**name your loop**” — the observation that every well-shaped loop is a standard-library algorithm wearing a disguise. Third, the **STL algorithms** themselves, with **lambdas** as the customization point. The third move *answers* the questions the first move raised: once you can name the loop, the library version is almost always the one you wanted.

This is also the chapter that puts a name on a reflex first mentioned in Chapter 4: before you write a thing, check whether the library already wrote it.

6.1 Part 1: loop hygiene — the spine

Before we reach for the library, look hard at the loops you already write. Most bugs and most unreadable code in a program live *inside a loop*. There are four habits that keep a loop honest, and they are worth more than any single algorithm, because they apply whether or not you ever call `std::sort`.

6.1.1 Invariants: the sentence that stays true

A **loop invariant** is a statement that is true *before* the loop starts, true *after every iteration*, and therefore true *when the loop ends*. You do not have to write it in the code, but you should be able to say it out loud. It is the thing the loop is *maintaining*.

```
// Find the maximum of a non-empty vector.
double max_so_far = v.at(0);           // invariant: max_so_far is the largest
for (std::size_t i = 1; i < v.size(); ++i) {    // of v[0..i-1] so far
    if (v.at(i) > max_so_far) {
        max_so_far = v.at(i);
    }
}
// invariant at exit: max_so_far is the largest of v[0..size-1] - i.e. all of v.
```

The invariant here is “*max_so_far holds the largest element we have looked at so far.*” It is true before the loop (we have looked at `v[0]`). The body keeps it true (if the new element beats the record, update the record). So when the loop runs out of elements, the invariant — now spanning the whole vector — is exactly the answer.

When a loop confuses you, the cure is almost always to ask: *what is the sentence that this loop keeps true?* If you cannot state it, the loop probably has a bug, because **you cannot maintain an invariant you cannot name.**

i Python had invariants too — you just didn’t have to say “`i < len(v)`”

In Python you would write `for x in v:` and never touch an index. The invariant was still there (`max_so_far` is the running max), but the *iteration mechanics* were hidden. C++’s indexed loop exposes them, which is more rope. The discipline below — and ultimately the STL algorithms — is about handing that rope back to the library so you can think about the invariant instead of the index.

6.1.2 What goes inside the loop, and what goes outside

This is where most sloppy loops go wrong. The body of a loop runs once *per element*. Anything you put there pays that cost — in runtime, and in reader attention — every single iteration. Two anti-patterns:

Anti-pattern A: loop-invariant work computed inside the loop (hoisting). If a value does not change across iterations, computing it *inside* the loop is wasted work — and worse, it is a *lie to the reader*, because its placement says “this might change each time.”

```
// Anti-pattern: recomputed every iteration, and reads as if it varies.
for (std::size_t i = 0; i < v.size(); ++i) {
    double scale = std::sqrt(2.0 * steps);    // <-- constant! hoist it
    total += v.at(i) * scale;
}
```



```
// Clean: the invariant is named once, where the reader expects it.
const double scale = std::sqrt(2.0 * steps);
for (std::size_t i = 0; i < v.size(); ++i) {
    total += v.at(i) * scale;
}
```

Pulling the constant out of the loop is called **hoisting**. Note the second version also marks it `const`, which tells the reader (and the compiler) “this will not be reassigned.” Modern optimizers will often hoist a provably-constant expression for you — but you should hoist it *for the reader* even when the compiler would have done it for the machine.

Anti-pattern B: a once-only decision evaluated every iteration. The classic shape is an `if (n == 0)` *inside* the loop — a test that can only ever be true on the first (or last) iteration, re-checked on every one.

```
// Anti-pattern: this if can only fire on iteration 0, but runs N times.
for (std::size_t i = 0; i < v.size(); ++i) {
    if (i == 0) {
        std::cout << "Results:\n";           // header - belongs before the loop
    }
    std::cout << "  " << v.at(i) << '\n';
}

// Clean: decide before the loop. The body now does exactly one job.
std::cout << "Results:\n";
for (const auto& x : v) {
    std::cout << "  " << x << '\n';
}
```

The fix is one of two moves: **decide before the loop** (as above), or **peel the special iteration out** of the loop entirely and handle it on its own. Both leave the loop body doing exactly one thing, the same thing, every time — which is what makes a loop readable.

⚠ “It still gives the right answer” is not the bar

The `if (i == 0)` version *works*. It prints the right output. That is exactly what makes it dangerous: it passes the test, so nothing forces you to fix it, and it sits there teaching the next reader that header-printing is part of the per-element job. Hygiene is not about correctness alone — it is about a loop whose shape *tells the truth* about what it does. A reviewer (human or AI) who sees `if (i == 0)` inside a loop will flag it on sight.

6.1.3 Init and exit conditions: the two ends of the loop

Every loop has two fragile points, and they are the two places bugs cluster:

- **Initialization** — the state the loop assumes is true *before* the first iteration. In the max example above, the init is `max_so_far = v.at(0)`, and it carries a hidden assumption: **the vector is non-empty**. If `v` is empty, `v.at(0)` throws. (With `v[0]` it would be undefined behavior — see Chapter 3 on why we prefer `.at()` exactly here, at the index that might be out of range.) A loop that assumes non-emptiness must either *guarantee* it or *check* it.
- **The exit condition** — the test that decides when to stop. Off-by-one bugs live here. `i < v.size()` visits indices `0 .. size-1` (correct). `i <= v.size()` visits one element too many (out of bounds). You saw the 100-input version of this bug in Chapter 3, where the `while` loop’s exit condition let the index run one past the end of the array.

Get in the habit of reading both ends of every loop you write *as a pair*: what does this assume going in, and what stops it coming out? The two empty cases — “*what if the container has zero elements?*” and “*what if it has exactly one?*” — catch the large majority of loop bugs before they ship.

6.1.4 DRY the loop body; extract a helper when it grows

The same **Don’t Repeat Yourself** rule you met in Chapter 2 (compute `disc` once, use it twice) applies inside and across loops. Two cases:

Two loops with the same body. If you find yourself writing the same loop body in two places — processing the on-time submissions, then the late ones with identical logic — that body wants a *name*. Lift it into a small helper function and call it from both loops.

One loop whose body has grown. A loop body longer than a handful of lines is usually one extracted function away from being readable. When the body grows a mind of its own — several locals, a nested branch, a sub-calculation — give that work a name:

```
// Before: the loop body is doing two jobs and you have to read all of it
// to find out what they are.
for (const auto& row : records) {
    double a = row.x - mean_x;
    double b = row.y - mean_y;
    double d = std::sqrt(a * a + b * b);
    if (d > cutoff and row.valid) {
        flagged.push_back(row.id);
    }
}
```

```
// After: the body names what it does. The helper is testable on its own.
auto is_outlier = [&](const Record& row) {
    double a = row.x - mean_x;
    double b = row.y - mean_y;
    double d = std::sqrt(a * a + b * b);
    return d > cutoff and row.valid;
};

for (const auto& row : records) {
    if (is_outlier(row)) {
        flagged.push_back(row.id);
    }
}
```

Read the “after” loop aloud: *“for each row, if it is an outlier, flag its id.”* That sentence is the whole loop. The *definition* of “outlier” is one helper away, where you can read it — or test it — in isolation.

And notice what just happened: the loop now has the shape *“for each element, if a condition holds, do something.”* Hold that thought. That shape has a name.

6.2 Part 2: name your loop

Here is the habit that ties the whole chapter together. **Before you write a loop, finish this sentence:** *“this loop ____ the elements.”*

- Sums them. → a **fold**
- Counts the ones that match. → a **count**
- Finds the largest. → a **max**
- Finds the first one matching. → a **search**
- Makes a new thing from each. → a **transform**
- Puts them in order. → a **sort**
- Checks “is there any / are they all”. → a **quantifier**

That verb is the name of a standard-library algorithm. Almost every loop you write over a container is one of a small number of *shapes*, and each shape has a name in `<algorithm>` or `<numeric>`:

If your loop...	...it is	from
adds up / folds the elements into one value	<code>std::accumulate</code>	<code><numeric></code>

If your loop...	...it is	from
finds the largest / smallest element	<code>std::max_element / std::min_element</code>	<code><algorithm></code>
counts elements that satisfy a condition	<code>std::count_if</code>	<code><algorithm></code>
makes a new element from each old one	<code>std::transform</code>	<code><algorithm></code>
checks “is there any / are all”	<code>std::any_of / std::all_of</code>	<code><algorithm></code>
finds the first element matching	<code>std::find / std::find_if</code>	<code><algorithm></code>
puts the elements in order	<code>std::sort</code>	<code><algorithm></code>

This is the “**library check**” reflex from Chapter 4, made concrete: when you catch yourself writing one of these loops by hand, stop and ask “*what is the standard name for this?*” It almost always exists, it was written by experts, it has been tested for decades, and it says what it does in one line instead of five.

The Python bridge is exact, because you already do this. You do not write a `for` loop to add up a list — you write `sum(xs)`. You do not loop to find the biggest — you write `max(xs)`. You do not hand-roll a sort with a key — you write `sorted(xs, key=...)`. C++’s standard algorithms are the same idea; they are just spelled with iterator pairs and lambdas instead of built-in functions and `key=`.

Python	C++
<code>sum(xs)</code>	<code>std::accumulate(xs.begin(), xs.end(), 0.0)</code>
<code>max(xs)</code>	<code>*std::max_element(xs.begin(), xs.end())</code>
<code>sum(1 for x in xs if p(x))</code>	<code>std::count_if(xs.begin(), xs.end(), p)</code>
<code>[f(x) for x in xs]</code>	<code>std::transform(...)</code> into an output
<code>any(p(x) for x in xs)</code>	<code>std::any_of(xs.begin(), xs.end(), p)</code>
<code>next(x for x in xs if p(x))</code>	<code>std::find_if(xs.begin(), xs.end(), p)</code>
<code>xs.sort(key=k)</code>	<code>std::sort(xs.begin(), xs.end(), cmp)</code>

The catch is the right column’s `p`, `cmp`, `key`: most of these algorithms need you to tell them *what counts* — what to add, what makes one element “bigger,” which condition to test. In Python you hand that over as a `lambda` or a `key=`. In C++ you hand it over as a **lambda**, too. So lambdas come next.

6.3 Part 3: lambdas — the customization point

A **lambda** is an anonymous function you write inline, right where you use it. If you have written `key=lambda r: r.score` in Python, you already know the idea; C++ just spells it with more punctuation.

The syntax has three brackets:

```
[capture](parameters) { body; return ...; }
```

- [...] — the **capture list**: which surrounding variables the lambda can see (often empty, []).
- (...) — the **parameters**, exactly like a normal function.
- {...} — the **body**.

The simplest possible lambda, stored in a variable so you can see its shape:

```
auto square = [](double x) { return x * x; };
double nine = square(3.0);      // 9.0 - it's just a function with no name
```

You rarely store one like that. The point is to pass it *directly* to an algorithm as the “what counts” rule:

```
// count how many scores are passing
int passing = std::count_if(scores.begin(), scores.end(),
                           [](double s) { return s >= 60.0; });
```

Read that lambda as: “*given a score s , is it passing?*” The algorithm calls it once per element and tallies the **true**s. When the element is a whole object, take it **by const reference** — `const T&` — so you do not copy it and cannot accidentally modify it. That is the same `const&` pass-by-reference rule you learned for functions in Chapter 4:

```
// count students who passed, given a vector<Student>
int passing = std::count_if(roster.begin(), roster.end(),
                           [](const Student& st) { return st.grade() >= 60.0; });
```

Capture, briefly. [] captures nothing — the lambda uses only its parameters. [cutoff] captures a *copy* of a local variable so the body can use it; [&cutoff] captures it *by reference*. This is the C++ answer to Python’s closures, where a `lambda` silently sees the enclosing scope; in C++ you say explicitly what it may see.

```
double cutoff = 60.0;
int passing = std::count_if(roster.begin(), roster.end(),
                           [cutoff](const Student& st) { // capture by value
                               return st.grade() >= cutoff;
                           });
```

💡 [&] and [=]: capture-all, and why to avoid them

You will see [&] (“capture everything by reference”) and [=] (“capture everything by value”) in real code. They work, but they hide *what* the lambda depends on, and [&] in particular is a footgun: if the lambda outlives the variables it captured by reference (stored in a container, returned from a function), those references dangle — a pointer-style bug we take up properly in Chapter 9. The course habit is to **name your captures** — [cutoff], [&total] — so the dependency list is visible. For the algorithms in this chapter, [] is usually all you need.

6.4 Part 4: the algorithms, by shape

Now the payoff. Each subsection takes one of the loop shapes from Section 6.2, writes it by hand, and then replaces it with the named algorithm. The library version is shorter, harder to get wrong, and — the part that matters most — it *announces its shape* so the next reader does not have to trace the loop to discover what it was.

The running data is a `std::vector<double> xs` of measurements, plus, where an object is needed, a `std::vector<Student>` with a `.grade()` member.

6.4.1 `std::accumulate` — fold a range into one value (<numeric>)

The three-argument form sums a range, starting from an initial value:

```
#include <numeric>
std::vector<double> xs{1.0, 2.0, 3.0};
double sum = std::accumulate(xs.begin(), xs.end(), 0.0); // 6.0
```

⚠️ The initial value sets the type — pass 0.0, not 0

Pass 0.0 (a double), not 0 (an int). If you start from `int 0`, `std::accumulate` does the *entire* sum in integer arithmetic and silently truncates every double along the way: the sum of {1.5, 1.5, 1.5} comes back as 3, not 4.5. This is the single most common `accumulate` bug, and the compiler will not warn you, because integer-plus-double is a perfectly legal expression. The initial value’s type *is* the accumulator’s type.

When the thing you are summing has to be *pulled out of* each element, use the four-argument form: a **binary lambda** (accumulator, element) that returns the new accumulator.

```
// mean grade over a vector<Student>
double total = std::accumulate(
    roster.begin(), roster.end(), 0.0,
    [](double acc, const Student& st) { return acc + st.grade(); });
double mean = total / roster.size();
```

Read it as: “start at 0.0; for each student, add their grade to the running total.” That is exactly the hand-rolled loop — `double total = 0.0; for (...) { total += st.grade(); }` — with the boilerplate deleted and the intent named.

6.4.2 `std::max_element` — find the largest (<algorithm>)

`std::max_element` returns an **iterator** to the largest element — *not the value*. Dereference it (`*it`, or `it->member` for an object) to reach the element.

```
#include <algorithm>
std::vector<double> xs{4.0, 9.0, 2.0, 7.0};
auto it = std::max_element(xs.begin(), xs.end());
double biggest = *it;    // 9.0 - note the dereference
```

For objects, pass a lambda that answers “*is a less than b?*” — and the algorithm returns the element that is greatest by that rule:

```
auto top = std::max_element(
    roster.begin(), roster.end(),
    [](const Student& a, const Student& b) { return a.grade() < b.grade(); });
double top_grade = top->grade();
```

 Two traps: the empty range, and the direction of the comparator

It returns an iterator, and on an empty range it returns `end()` — which you must never dereference. With a non-empty vector you are safe, but get in the habit of noticing the precondition (the same non-emptiness assumption the hand-rolled max had as its *init condition* back in Part 1 — the library did not make the assumption go away, it just made it explicit). **The comparator is always “less than.”** Even though you want the *maximum*, the lambda returns `a < b`. The algorithm uses it to decide ordering and returns the greatest under that ordering. Writing `>` there silently gives you the *minimum* — a quiet bug that compiles, runs, and returns a plausible wrong answer.

6.4.3 `std::count_if` — count what matches (<algorithm>)

The shape you reached at the end of Part 1 — “for each element, if a condition holds, count it” — is `std::count_if`. The condition is a lambda returning `bool` (a **predicate**):

```
int n_passing = std::count_if(roster.begin(), roster.end(),
                             [](const Student& st) { return st.grade() >= 60.0; });
```

This is Python’s `sum(1 for st in roster if st.grade() >= 60.0)`, one line, no running counter to forget to increment. There is also a plain `std::count` for counting a specific *value* (no predicate): `std::count(v.begin(), v.end(), 0)` counts the zeros.

6.4.4 `std::find` and `std::find_if` — the first match (<algorithm>)

`std::find` searches for a value; `std::find_if` searches for the first element satisfying a predicate. Both return an iterator, and both return `end()` when there is no match — so the *test* for “did we find it?” is a comparison against `end()`:

```
auto it = std::find_if(roster.begin(), roster.end(),
                      [](const Student& st) { return st.grade() < 50.0; });
if (it != roster.end()) {
    std::cout << "First failing student: " << it->name() << '\n';
} else {
    std::cout << "Nobody is failing.\n";
}
```

This is Python’s `next((st for st in roster if st.grade() < 50.0), None)`, where the `!= end()` check plays the role of the `None` default. The `find_if` stops at the *first* match — it does not scan the rest of the container — which is the same early-exit you would write with a `break`, handed to the library.

6.4.5 `std::sort` — put them in order (<algorithm>)

`std::sort` orders a range in place. Plain `std::sort(v.begin(), v.end())` sorts ascending by `<`. To sort by a *key* — the Python `sorted(xs, key=...)` move — pass a comparator lambda, again of the “is a before b?” shape:

```
// sort students by grade, highest first
std::sort(roster.begin(), roster.end(),
          [](const Student& a, const Student& b) { return a.grade() > b.grade(); });
```


Here the comparator decides “comes first,” so `>` puts the highest grade at the front — descending order. (Contrast `max_element`, where the comparator means “less than” regardless of which extreme you want. The rule is per-algorithm; read the one line of documentation the first few times.)

i `std::sort` mutates; Python's `sorted` copies

`std::sort` reorders the vector *in place* and returns nothing — it is Python's `xs.sort()`, not `sorted(xs)`. If you need to keep the original order, copy the vector first (`auto copy = xs;`) and sort the copy. And `std::sort` requires **random-access** iterators, which `std::vector` has but a `std::list` (coming in Chapter 11) does not — that container brings its own `.sort()` member for exactly this reason.

6.5 Part 5: the answer to Part 1's questions

Now circle back. Part 1 raised a list of hygiene questions about a hand-written loop:

- *Is the invariant clearly maintained?*
- *Is loop-invariant work hoisted out?*
- *Is there a once-only decision wrongly stuck inside?*
- *Are the init and exit conditions both correct, including the empty case?*
- *Is the body small enough to read?*

When you replace the loop with `std::accumulate`, `std::count_if`, `std::max_element`, or `std::sort`, **every one of those questions is answered by construction**:

- The invariant is the algorithm's *definition* — `accumulate` *is* a running fold, by name.
- There is no place to put loop-invariant work, so you cannot put it in the wrong place. (Hoisting becomes “compute it before the call.”)
- There is no per-iteration `if (i == 0)`, because you do not see the iterations at all.
- The init and exit conditions are the library's, written once and tested for decades. (The empty-range precondition of `max_element` is the one thing it hands *back* to you — and it does so explicitly, by returning `end()`.)
- The body that remains is the lambda — the *one* rule that is genuinely yours, with all the bookkeeping stripped away.

That is the sense in which **the library version is almost always the one you wanted**. Loop hygiene is the set of properties a hand-written loop has to *earn*. A named algorithm has most of them by default; all you supply is the part that was ever specific to your problem.

💡 When *not* to reach for an algorithm

This is not dogma. A few loops are genuinely clearer hand-written:

- A loop that does **two things at once** — say, summing *and* finding the max in a single pass — is two algorithm calls, hence two passes over the data. For a hot inner loop over millions of elements, one hand-written pass can be the right call. (Measure first; Chapter 16 is about not guessing.)
- A loop whose body has **real control flow** — early returns, multiple breaks, side effects on several variables — may not fit any single algorithm’s shape, and forcing it into one produces a worse lambda than an honest loop.
- A genuinely **novel** computation that no standard algorithm names.

The reflex is “*name your loop first.*” If it has a clean name, use the named version. If naming it is a struggle, that is information too — sometimes it means the loop is doing too much and wants splitting; occasionally it means you have a real, irreducible loop. Both are fine. The anti-pattern is never *asking*.

6.6 Build to learn, use to ship

There is a tension a sharp student notices immediately: *if the library already does all this, why did the earlier chapters make me hand-write the max and the sum?*

Because **you built it once to learn it**. You wrote the hand-rolled mean and the hand-rolled max so that you understand what the loop *does* — the invariant, the init condition, the per-element update. You will need that mechanism cold in the Algorithms course, and you need it now to *read* code that predates the library habit (most code does). Knowing the mechanism is also what lets you judge the exceptions in the callout above: you cannot decide a hand-written two-in-one pass is worth it unless you know exactly what the library would have cost.

But now that you know the mechanism, **use the library**. It is written by experts, benchmarked for decades, correct in the corner cases you have not thought of yet, and it says its intent in one line. That is the rule for the rest of the course, and the rest of your career:

Build it once to learn it. Then reach for the standard version to ship.

The same lens returns when we reach containers beyond `std::vector` in Chapter 11: a `std::map` is not just a place to put data, it is a place that comes with algorithms (ordered iteration, logarithmic lookup) you would otherwise hand-write and get wrong. The container-and-algorithm pairing is one of the central ideas of the standard library, and “name your loop” is how you learn to see it.

6.7 AI touchpoint — “what’s the STL function that does this?”

This chapter’s reflex is one of the *best* uses of an AI assistant, and one of the *worst* — depending on which question you ask.

The good question. You have a hand-written loop and you suspect it has a name. Paste the loop and ask: “*Is there a standard-library algorithm that does this? Show me the call, and explain what it assumes about the input.*” This is the **AI as library guide** role — you are using the assistant to do the “library check” faster than grepping cppreference. It is excellent at this: the mapping from loop-shape to algorithm-name is exactly the kind of pattern an LLM has seen ten thousand times.

The bad question. “*Write me a loop that computes the mean and the max.*” Now the assistant hands you code you did not reason about, and you have skipped the part the course is actually grading — your ability to *name the shape* and *explain every line*. If you cannot say why the `accumulate` starts at 0.0 and not 0, or why the `max_element` comparator uses `<` even though you want the max, you have not learned the thing; you have laundered it through a model.

The disciplined move: ask the assistant for the *name*, then write the call yourself, then ask it to *review* your call — “*Did I get the initial-value type and the comparator direction right?*” That keeps you in the loop (so to speak) on the two bugs this chapter warned about, while still using the tool for what it is genuinely good at.

6.8 What you should understand by the end of this chapter

- What a **loop invariant** is, and why being able to *state* it is the test of whether you understand a loop
- The two inside-vs-outside anti-patterns — **un-hoisted invariant work** and the **once-only decision inside the loop** (`if (i == 0)`) — and how to fix each
- Reading a loop’s **init and exit conditions** as a pair, including the empty- and single-element cases where bugs cluster
- **DRYing** a repeated loop body and **extracting a helper** (a named function or a lambda) when the body grows
- The “**name your loop**” reflex: every well-shaped loop is `std::accumulate`, `std::max_element`, `std::count_if`, `std::find_if`, `std::sort`, or one of their siblings in disguise
- **Lambdas** as the customization point — the `[capture](params){body}` syntax, `const T&` parameters for objects, and named captures over `[&]`/`[=]`
- The four workhorse algorithms in detail: `std::accumulate` (and the 0.0-not- 0 trap), `std::max_element` (iterator return, `end()` on empty, `<` comparator), `std::count_if`, and `std::sort` (in-place, key via comparator)
- The Python bridge: these *are* `sum`, `max`, the filtered count, and `sorted(key=...)`, spelled with iterator pairs and lambdas

- The **build-to-learn, use-to-ship** rule, and the honest exceptions where a hand-written loop is still the right call

If you can take a hand-written loop, *name* it, rewrite it as the matching algorithm call, and explain every argument — especially the lambda and the initial value — you are ready for Chapter 7.

Part III

Week 3 — What's underneath: the layered machine and memory model

Week three is the deliberate pivot from *what the language lets you write* to *what the machine actually does*. Across four lectures you walk a C++ function through the six-level Tanenbaum stack — high-level language to assembly to operating system to instruction set to microarchitecture to digital logic — and then look at memory directly: stack frames in GDB, the layout of a `std::vector`, raw pointers and references, the lifetime problem in its naked form, and finally modern pointers (`unique_ptr`, `shared_ptr`, `weak_ptr`) which will be your default for the rest of the book.

The chapters of this part:

- Chapter 7 — HLL → assembly → OS → ISA, recursion in GDB
- Chapter 8 — stack vs heap, what `vector` actually allocates, examining memory in GDB
- Chapter 9 — raw pointers, references, `nullptr`, raw arrays, why `vector` exists, the lifetime problem stated clearly
- Chapter 10 — `unique_ptr` + `make_unique` as the default, `shared_ptr` + `weak_ptr` for shared ownership, move semantics conceptually

7 The layered machine I: levels, ISA, and assembly

Every program you have written in this book so far has run on a real machine — a CPU, with registers, with memory laid out in numbered bytes, executing one instruction at a time. You have never had to think about that, and that was the point. Python hides the machine completely; C++ hides most of it most of the time. This chapter and the next take one deliberate step *down*, not because you must master the bottom in order to use the top, but because the view from one level down explains a surprising number of things that otherwise look like magic — why a `std::vector`’s elements are somewhere different from the vector itself, why a runaway recursion *crashes* rather than just being slow, what a pointer is actually pointing *at*.

The framing matters, so let me state it plainly: this is **not** “the bedrock you should have learned first.” You did fine without it. It is “now let’s look at what was underneath everything you’ve already been writing.” The abstractions you have been using — a vector, a string, a class, a function call — are all built out of the pieces below, and today we look at the pieces.

7.1 Six levels, top to bottom

A useful way to organize “what’s underneath” is the **layered-machine hierarchy**, popularized by Andrew Tanenbaum. Each level is a complete machine in its own right; each is implemented in terms of the level below it; and at each level you can write programs without knowing how the level below works.

Level	What lives there	You, in this course
5	Application — the programs you run	a user
4	High-level language (C++) — your source	here, until today
3	Assembly language — human-readable instruction text	we read it today
2	ISA (instruction set architecture) — the CPU’s contract	we name it today

Level	What lives there	You, in this course
1	Microarchitecture — pipelines, caches, the silicon’s plan	named, then EC413’s job
0	Digital logic — gates, transistors	named, then EC311’s job

You have spent the whole book at **level 4**, the high-level language. Today we drop to **level 3 (assembly)** and **level 2 (the ISA)** — close enough to the metal to make the machine concrete, but not so close that you need a circuits course. The bottom two levels are real and they matter, but they are someone else’s class; we will name them and move on.

i “Assembly” and “ISA” are not the same thing

The **ISA** is the *contract*: the set of instructions a particular CPU understands, the registers it has, how it touches memory. It is an abstract specification — **x86-64** and **ARM64** are two different ISAs. **Assembly** is the human-readable *text* for an ISA: `add eax, ebx` is one line of x86-64 assembly. Assembly is to the ISA roughly as your `.cpp` source is to the C++ language — a textual surface over an underlying thing. The CPU does not actually execute the text; an *assembler* turns each assembly line into the binary **machine code** the silicon decodes. For our purposes the text *is* the machine, one rung up where we can read it.

7.2 The toy machine: a CPU you can see all of

Real x86-64 assembly at -00 is twenty-odd lines of bookkeeping wrapped around the one instruction that does the work. Before we look at that, we build the intuition on a *toy machine* small enough to hold entirely in your head: about fifteen instructions, four general-purpose registers, a stack, two flags, and a flat memory of numbered bytes. Nothing is hidden. You single-step it and watch every cell change. (The course ships this as an interactive notebook — `lecture_08_machine_sim.py` — that runs in the browser with nothing to install.)

Here is the whole machine, named:

- **Registers** — `rax`, `rbx`, `rcx`, `rdx`: four tiny named slots, each holding one integer, *inside* the CPU. This is the first idea Python hid from you completely: a CPU does almost all of its work in a handful of registers, not in “variables.” Variables are a fiction the compiler maintains *on top of* registers and memory.
- **Memory** — a flat array of bytes, addressed by number. `[256]` means “the value stored at byte address 256.” A register can *hold* an address; `[rbx]` means “the value at whatever address `rbx` currently holds.” Hold onto that last sentence — it is the seed of Chapter 9.
- **The stack pointer `rsp`** — a register that always points at the top of the **stack**, a region of memory that grows *downward* from a high address as the program nests function calls.

- **Two flags** — ZF (zero) and SF (sign): single bits the CPU sets as a side effect of arithmetic and comparison, and that conditional jumps read.
- **The program counter** — an internal pointer to the *next instruction*. The CPU’s entire life is the **fetch–decode–execute** loop: fetch the instruction the program counter points at, decode what it means, execute it, advance the program counter (unless the instruction was a jump, which sets it somewhere else).

That loop — fetch, decode, execute, repeat — *is* what a CPU does. Everything else is detail.

7.2.1 Program one: sum

The smallest interesting program. Add two numbers.

```
mov rax, 5      ; rax <- 5
mov rbx, 7      ; rbx <- 7
add rax, rbx    ; rax <- rax + rbx    (now 12)
hlt
```

Three things to read here, and they will carry all the way up to real x86.

mov dst, src puts a value somewhere. The destination comes *first*. This is “Intel order,” and it trips up everyone once: `mov rax, 5` means “move 5 *into* rax,” not the other way around. Read it as an assignment with the arrow pointing left: `rax <- 5`.

add dst, src is two-operand and destructive. `add rax, rbx` means `rax <- rax + rbx`. The destination is *also* the first operand. There is no “put the sum somewhere new” — `rax`’s old value is gone, replaced by the sum. This is exactly the `+=` you already know: `add rax, rbx` is `rax += rbx`.

hlt stops the machine. Real CPUs do not stop; they `ret` back to whoever called them. The toy machine halts so a demo program has somewhere to end.

Single-step this and watch `rax` go `0 → 5`, `rbx` go `0 → 7`, then `rax` go `5 → 12`. That is the entire program. When you later see real x86-64 do the same addition, the `add` will be in there — surrounded by setup, but in there.

7.2.2 Program two: max — flags and a branch

How does an `if` work one level down? There are no curly braces in assembly. There is a **comparison that sets flags**, and a **conditional jump that reads them**.

```
mov rax, 3      ; a
mov rbx, 9      ; b
cmp rax, rbx    ; sets flags from (rax - rbx)
```

```

        jge  a_wins      ; jump if rax >= rbx
        mov  rax, rbx    ; else: answer is b
        jmp  done
a_wins: nop              ; answer (a) already in rax
done:   hlt

```

`cmp rax, rbx` computes `rax - rbx` and throws the result away — keeping only the flags. If the two are equal, ZF is set; the sign of the difference lands in SF. `cmp` is *subtraction you do only for its side effects*.

`jge a_wins` reads those flags and jumps to the label `a_wins` only if `rax >= rbx`; otherwise it falls through to the next instruction. So the pair `cmp` + conditional-jump is the machine-level shape of *every* if you have ever written. `if (a > b) { ... }` compiles to a compare and a branch. The ternary `?:`, the loop test, the `&&` short-circuit — all of them are compare-then-branch underneath.

Swap the two immediates (`mov rax, 9, mov rbx, 3`) and step it again: the flags come out differently, `jge` is taken instead of falling through, and the other branch runs. Watching the branch flip with your own eyes is the moment “control flow” stops being syntax and becomes a thing the machine *does*.

7.2.3 Program three: factorial, two ways

A loop, at the machine level, is a **compare plus a backward jump**. Here is `factorial(n)` as a loop:

```

        mov  rax, 1      ; result
        mov  rcx, 4      ; n, counts down
loop:   cmp  rcx, 1      ; while n > 1
        jle  done
        imul rax, rcx    ; result *= n
        dec  rcx        ; n--
        jmp  loop       ; <-- backward jump = "loop again"
done:   hlt

```

The only new shape is the `jmp loop` at the bottom: an *unconditional backward jump*. A **while** loop is a forward conditional jump to skip out (the `jle done`) plus a backward unconditional jump to go around again. That is all a loop is. The compiler gives loops nice names (**while**, **for**); strip the names and you get `cmp` and a backward `jmp`.

Now the headliner — the **recursive** factorial, which makes the *stack* visible:

```

factorial:
    cmp rdi, 1          ; if n <= 1
    jg  recurse
    mov rax, 1          ; base case: return 1
    ret
recurse:
    push rdi            ; save n on the stack
    dec rdi             ; n - 1
    call factorial      ; recurse: pushes a return address
    pop rcx             ; restore our n into rcx
    imul rax, rcx       ; rax = (n-1)! * n
    ret

```

Two new instructions and the whole point of the toy machine:

call factorial pushes a return address and jumps. When `factorial` calls *itself*, the CPU first writes onto the stack the address of the instruction to come back to, then jumps to `factorial`'s first instruction. **ret pops that address and jumps back to it.**

push rdi and pop rcx save and restore values on the stack. Each recursive call needs its *own* copy of `n`, because the multiply at the end needs the `n` from *this* level after the inner call has clobbered the register. So each level **pushes** its `n` before recursing and **pops** it back afterward.

Step `factorial(4)` slowly and watch the **stack** panel. As the recursion goes deeper, return addresses and saved `ns` pile up — `rsp` marching downward from `0x200`, one frame per call. Then, as each call hits the base case and returns, it all unwinds: **pop**, multiply, **ret**, **pop**, multiply, **ret**. This is the picture that makes recursion (which you met in Chapter 4) concrete — and it is the exact thing you will see in a real debugger in Chapter 8.

i The toy machine is x86-flavored on purpose

Two-operand destructive arithmetic, memory operands, implicit flags, `call/ret` that push the return address onto the stack — every one of these is an x86-64 trait. We picked that dialect so the jump to real Intel assembly is short. When we meet ARM64 below, a few of these will be done differently; the *ideas* survive the translation.

7.3 The same program, on a real machine

Now the authenticity anchor: the *same* `sum` function, compiled by a *real* compiler for a *real* CPU. You do not need a special tool — `g++` will print the assembly it would have generated if you ask with `-S` instead of compiling all the way to an executable.

```
// l8_sum.cpp
int sum(int a, int b) {
    return a + b;
}

int main() {
    return sum(5, 7); // exit status is the result; check with: echo $?
}
```

Compile it to assembly text. We pass `-O0` (no optimization) so the output maps line-for-line to the source, and `-masm=intel` so the syntax matches the destination-first order from the toy machine:

```
$ g++ -std=c++20 -O0 -S -masm=intel l8_sum.cpp -o l8_sum.s
```

💡 Getting *x86* assembly on an Apple-Silicon Mac

If you are on an Apple-Silicon Mac, plain `g++ -S` gives you **ARM64** output, not *x86* — that is your machine’s actual ISA, and reading it is the point of the last section of this chapter. To see *x86-64* output anyway (to follow along with an Intel-laptop classmate), add one flag:

```
$ g++ -arch x86_64 -std=c++20 -O0 -S -masm=intel l8_sum.cpp -o l8_sum_x86.s
```

Apple’s compiler is a cross-compiler, so it cross-emits clean *x86-64* assembly text with no extra toolchain. On an Intel box or under WSL2, drop `-arch x86_64` — you are already on *x86*. A browser tool, **Compiler Explorer** (godbolt.org), shows the same output for either ISA side by side if you prefer not to install anything.

Inside the `sum` function, past the prologue bookkeeping, the body is:

```
mov    eax, dword ptr [rbp - 4]    ; load a (a local, spilled to the stack)
add    eax, dword ptr [rbp - 8]    ; eax += b      <-- arithmetic reads memory
ret
```

Find the add. There it is, doing `eax += b`, exactly the two-operand destructive add from the toy machine. The differences are all *bookkeeping*: because we asked for no optimization, the compiler kept the locals `a` and `b` in memory (on the stack, at `[rbp - 4]` and `[rbp - 8]`) and loaded them into the register `eax` before adding. The toy machine’s `add rax, rbx` was *this*, minus the stack-frame setup. “Same idea, plus housekeeping” is the whole reveal.

i `eax` vs `rax` — same register, different width

You will notice the real output says `eax` where the toy machine said `rax`. On x86-64, `rax` is the full 64-bit register and `eax` is its lower 32 bits — *the same physical register*, viewed at a narrower width. The compiler used `eax` because C++ `int` is 32 bits. `rax/eax/ax/al` are four names for one register at 64/32/16/8 bits. This name-soup is one of x86’s quirks; ARM64 makes it cleaner, as we will see.

7.3.1 What the optimizer does

Now ask for optimization with `-O2` and look again:

```
$ g++ -std=c++20 -O2 -S -masm=intel l8_sum.cpp -o l8_sum_O2.s
```

```
sum:
    lea  eax, [rdi + rsi]    ; eax = a + b, in one instruction
    ret
```

The stack traffic is *gone*. With optimization on, the compiler kept `a` and `b` in registers (`rdi`, `rsi` — where the caller had already put the arguments) and computed the sum with a single `lea` (an address-arithmetic instruction the compiler cleverly repurposes to mean “add these and put the result in `eax`”). And `main` is even more dramatic — knowing the arguments are the constants 5 and 7, the optimizer computes `5 + 7` *at compile time* and emits:

```
main:
    mov  eax, 12    ; the whole program, precomputed
    ret
```

The function call vanished; the addition vanished; the result 12 is baked into the binary. **This is what “the compiler optimizes your code” actually means** — and why benchmarking a `-O0` build tells you almost nothing about how fast your program really is. (For `l8_max.cpp`, `-O2` does something even slicker: it replaces the `if/branch` entirely with a `cmov`, a *conditional move*, so the CPU never has to guess which way a branch will go. The compiler out-clevered the obvious code.)

! You will never write this. You should be able to read it.

Nobody ships hand-written assembly for application code — the compiler is better at it than you are, as the `cmov` trick shows. The reason to read assembly is *diagnostic*: to confirm a hot loop compiled the way you hoped, to understand a profiler’s output, to see that a “zero-cost abstraction” really did cost zero. Reading is the skill; writing is a specialist’s job.

7.4 Two machines, one model: x86-64 vs ARM64

Here is a fact that surprises people: an Intel/AMD Windows laptop and an Apple-Silicon Mac are *nearly the same machine* on five of the six levels. They run the same C++ source. They have the same OS model (processes, virtual memory, syscalls). They are both **von Neumann** machines — registers, a flat byte-addressed memory, a stack that grows down, a program counter, flags, fetch–decode–execute. Their microarchitectures use the same ideas (pipelined, superscalar, out-of-order, cached). Down at the silicon, both are CMOS transistors forming gates.

They differ at exactly **one** level — the ISA — and even there the difference is **dialect, not kind**. x86-64 is **CISC** (complex instruction set), grown backward-compatibly from the 1978 Intel 8086. ARM64 is **RISC / load-store** (reduced instruction set), a clean 2011 design with no legacy baggage. The differences that matter to a reader:

Dimension	x86-64 (CISC)	ARM64 (RISC / load-store)
Instruction length	variable , 1–15 bytes	fixed 4 bytes , always
Arithmetic form	two-operand, destructive : <code>add eax, ebx → eax += ebx</code>	three-operand, non-destructive : <code>add w0, w1, w2 → w0 = w1 + w2</code>
Touching memory	arithmetic may read/write memory directly: <code>add eax, [rbp-8]</code>	load/store only — <code>ldr</code> , compute in registers, <code>str</code> back
General registers	16 (<code>rax...r15</code>)	31 (<code>x0...x30</code>) + a zero register
Register views	<code>rax/eax/ax/al</code> — one register, four widths	<code>x0</code> (64-bit) / <code>w0</code> (32-bit) — one clean pair
Flags	implicit — most arithmetic sets flags as a side effect	explicit — you opt in (<code>adds</code> , <code>cmp</code>), then branch
Return address	<code>call</code> pushes it on the stack ; <code>ret</code> pops it	<code>bl</code> puts it in the link register ; spilled only when needed

7.4.1 The same add, two dialects

The cleanest way to feel CISC-vs-RISC is to put the two `sum` outputs side by side (both at `-00`).

x86-64 — the memory load is *folded into* the add:

```
mov  eax, dword ptr [rbp - 4]    ; load a
add  eax, dword ptr [rbp - 8]    ; eax += b      <-- arithmetic reads memory
ret
```

ARM64 — the loads are *separate*, and the add is pure register, three-operand:

7.5 AI touchpoint — have an LLM explain a chunk of -O2 assembly

```
ldr  w8, [sp, #12]      ; load a into a register
ldr  w9, [sp, #8]       ; load b into a register
add  w0, w8, w9         ; w0 = w8 + w9  <-- registers only
ret
```

Both compute `a + b`. x86 hides a memory read *inside* the `add`; ARM64 makes you `ldr` first and keeps the `add` register-only. That single contrast *is* CISC vs RISC. The three-operand `add w0, w8, w9` is also non-destructive — `w8` and `w9` survive — whereas x86's `add eax, ...` overwrites `eax`. And the toy machine's two-operand `add rax, rbx` is the shared idea both dialects are expressing.

The takeaway, in one line: *your Mac and a Windows laptop are the same machine in almost every way that matters. They speak different assembly the way Spanish and Italian are different languages — same grammar (von Neumann), different words (the ISA). The C++ you write does not care; the compiler is the translator.*

💡 See your own machine's dialect

Compile any small function on your own laptop with `g++ -O0 -S 18_sum.cpp -o out.s` and open `out.s`. If you see `rax/eax` and `add eax, ...`, your machine speaks **x86-64**. If you see `x0/w0` and a `ldr/add/str` rhythm, it speaks **ARM64**. Write one sentence in your notes: “my machine speaks _____.” It is a small thing, but it is the difference between *believing* your code runs on a physical CPU with an ISA and *knowing* which one.

7.5 AI touchpoint — have an LLM explain a chunk of -O2 assembly

Optimized assembly is exactly the kind of dense, jargon-heavy text where an LLM earns its keep as a patient explainer — and exactly the kind of text where you must *check its answer*, because it will state things with total confidence whether or not they are true.

Compile `18_max.cpp` at `-O2` and find the `cmov`:

```
$ g++ -std=c++20 -O2 -S -masm=intel 18_max.cpp -o 18_max_02.s
```

Paste the whole `max2` function's assembly into an AI assistant and ask: “*Explain what this x86-64 assembly does, line by line, and tell me what C++ it came from.*” Then — this is the disciplined part — **check its account against the toy machine you now understand**. Did it correctly say `cmp` sets flags and `cmov` reads them? Did it notice there is *no branch*, that the compiler replaced the `if` with a conditional move? If it hand-waves or gets the operand order backward, you are now equipped to catch it.

This is the **AI as coach / IT aide** role from the AI Tools chapter: the LLM accelerates your reading of unfamiliar machine output, and your toy-machine mental model is the ground truth you

grade its answer against. Using AI to read assembly you could not otherwise read is great; trusting it blindly on something you cannot verify is exactly the failure mode the course asks you to avoid.

7.6 What this sets up

Two threads from this chapter are left deliberately open, and the next chapters pick them up:

- **The stack you watched grow** in the recursive factorial is real, and in Chapter 8 we look at it in a live debugger (`gdb, bt full`) on your own binary — the real-machine twin of the toy machine’s stack panel — and use it to draw the single most important picture of the unit: where a `std::vector` object lives versus where its *elements* live.
- **A register holding an address** — `[rbx]` meaning “the value at the address in `rbx`” — is the seed of the pointer. We named it here at the machine level and stopped. Chapter 9 answers the question it raises: *what do you call a C++ variable whose value is an address?* The word is **pointer**, and the `*` and `&` syntax that goes with it is the next chapter’s job, not this one’s.

7.7 What you should understand by the end of this chapter

- The six levels of the layered (Tanenbaum) machine, and that C++ lives at the high-level-language level while assembly and the ISA sit just below it
- That the framing is “look at what was underneath,” not “bedrock you must master first”
- What a register, the memory address space, the stack pointer, the flags, and the program counter are — the things Python hid from you entirely
- The fetch–decode–execute loop as the entire job of a CPU
- Reading toy-machine assembly: `mov dst, src` (destination first), two-operand destructive `add`, `cmp` + conditional jump as the shape of an `if`, a backward `jmp` as the shape of a loop, and `call/ret/push/pop` as the shape of a function call that grows the stack
- Finding the `add`, the `cmp`, and the `call` in real `g++ -S` output, and reading `-O0` (maps to source) versus `-O2` (the optimizer folds, inlines, and precomputes)
- “Two machines, one model”: x86-64 (CISC) and ARM64 (RISC/load-store) share the von Neumann model and differ only in ISA *dialect* — variable vs. fixed instruction length, two- vs. three-operand arithmetic, memory operands vs. load-store, implicit vs. explicit flags
- That an LLM is a useful but fallible reader of optimized assembly, to be checked against the model you understand

If you can read the toy machine’s recursive `factorial` out loud, point to the `push/call/ret`, and explain why each level needs its own saved `n`, you are ready to watch that same stack in a real debugger in Chapter 8.

8 The layered machine II: stack, heap, and the lifetime question

The previous chapter, Chapter 7, took you down to the *instruction* level: `add` really is `+`, an `if` really is a compare-and-jump, and a function call really does `push` a return address onto a *stack* and `ret` pops it back off. You watched the toy machine’s recursive `factorial` push one frame per call and pop them back off in reverse order.

This chapter stays on the same machine but changes the question from *what instructions run* to **where the data lives**. There are two answers — the **stack** and the **heap** — and the difference between them is the single most important thing a Python programmer has to internalize when moving to C++. It is the foundation under pointers (Chapter 9), under smart pointers (Chapter 10), and under the entire `std::vector` cost model you have been using since Chapter 3 without being told what it costs.

We finish with the question this chapter cannot answer — *who frees heap memory, and when?* — stated cleanly, because the next two chapters exist to answer it.

8.1 Python hid this from you, and that was the point

In Python, you never thought about *where* a value lived. You wrote `xs = [10, 20, 30]` and the language found room for the list, the three integers, and the name `xs`, and it freed all of that automatically whenever the last reference to it went away. A *garbage collector* — a background bookkeeper — walked your objects, noticed which ones nothing pointed at anymore, and reclaimed them. You never wrote “give this memory back,” and you never had a bug from forgetting to.

That convenience has a price you have already met: Python is slow, because every one of those conveniences is paid for at run time. C++ makes the opposite trade. It hands you the two memory regions directly and asks you to know which one each value is in. In exchange it gives you predictable speed and no background bookkeeper.

The good news for week 3: for everything you have written *so far*, the compiler has been managing both regions for you correctly and silently. The goal of this chapter is not to make you manage memory by hand — that comes later, carefully. It is to make the picture visible, so that when a `std::vector` or a pointer behaves a certain way, you can see *why*.

8.2 One memory, two ends

Your program’s memory is, at bottom, **one flat array of bytes**, each with a numeric address. The operating system hands your process a giant range of these addresses and your program carves it up into regions. Two of those regions matter today, and they grow toward each other from opposite ends:

```

high addresses  [ stack ]   ↓   locals, return addresses, saved registers

                (free space the two grow into)

                [ heap  ]   ↑   vector elements, big or long-lived objects
                [ globals / code ]

low  addresses
```

- **The stack** grows *down*, from high addresses toward low ones. It holds your **local variables** and the bookkeeping of every function call in progress.
- **The heap** grows *up*, from low addresses toward high ones. It holds things that are **too big** for the stack or that must **outlive** the function that created them.

You do not have to take this on faith. In a few pages you will print the address of a local variable in a debugger and watch it come back as a *high* number, then print the address of a **vector**’s elements and watch it come back as a *much lower* one — the same program, looking at both ends of its own memory.

i “Down” and “up” are conventions, not laws

The stack growing toward lower addresses is a convention baked into the calling conventions of essentially every mainstream CPU, so you can rely on it in practice. But nothing in the C++ *language* promises it; a portable program never depends on the numeric direction. We lean on it here only because it makes the debugger output legible — high address means stack, low address means heap, on the machines this course runs on.

8.3 The stack: automatic, LIFO, bounded

Everything you have declared as a plain local variable so far has lived on the stack:

```

int tag = 42;           // 4 bytes on the stack
double x{3.14};         // 8 bytes on the stack
char letter{'A'};       // 1 byte on the stack
```

The stack has three properties worth naming, all of which you saw in Chapter 7 when you watched `factorial` recurse:

It is automatic. You never write code to allocate or free a local. When execution enters a function, the machine reserves a block of stack — the function’s **stack frame** — for that call’s locals, saved registers, and the return address. When the function returns, `ret` releases the whole frame in one step. Nobody writes “free this frame”; it is part of what `call` and `ret` do.

It is LIFO. Last in, first out. The most recently called function is the first to return, exactly the way a stack of plates works. This is why nested calls never get tangled: `main` calls `f`, `f` calls `g`, and `g` must finish (and have its frame popped) before `f` can resume. Calls *nest*; they never cross. That clean nesting is a direct consequence of structured control flow — the same discipline that retired the `goto` decades ago is what makes the stack picture so simple.

It is fast but bounded. Reserving a frame is nearly free — it is just an adjustment to a register that tracks the top of the stack. But the total stack is *small*: a few megabytes (the addendum to this lecture measured it at about **8 MiB** on both the course Linux VM and macOS). That bound is deliberate, and it is the source of the failure mode we trigger next.

i Why only ~8 MiB?

The 8 MiB stack limit is not a compiler setting like `-O2`; it is a *resource limit* the operating system imposes when your program starts (the same `ulimit` machinery that caps open files and CPU time). The size is a tradeoff: big enough for ordinary recursion and reasonable local arrays, but small enough that every thread can afford its own stack and that runaway recursion is *caught fast* instead of silently eating gigabytes first. You can raise the limit (`ulimit -s`, a linker flag), but the real fixes for “I ran out of stack” are to move big data to the heap or to tame the recursion — both of which this chapter is about.

8.4 Stack overflow, on purpose

The cleanest way to feel the stack’s boundary is to run off the end of it. Recall from Chapter 4 that a recursive function calls itself, and that every recursion needs a **base case** that eventually stops the descent. Here is one with the base case deliberately removed:

```
// 19_overflow.cpp -- the bug is the missing base case
#include <iostream>

void descend(long depth) {
    volatile char pad[1024]; // burn a kilobyte of stack per call, to fail faster
    if (depth % 1000 == 0) {
        std::cout << "depth = " << depth << '\n';
    }
    descend(depth + 1);
}
```

```

    }
    descend(depth + 1); // <-- recurses forever; nothing ever returns
}

int main() {
    descend(0);
    return 0;
}

```

Every call to `descend` pushes a fresh frame — its own `depth`, its own kilobyte-wide `pad` — and *nothing ever pops*, because `descend` never returns. The stack grows down, frame after frame, until it hits its limit:

```

$ g++ -std=c++20 -Wall -Wextra -g -o l9_overflow l9_overflow.cpp
$ ./l9_overflow
depth = 0
depth = 1000
depth = 2000
...
Segmentation fault

```

That `Segmentation fault` is the operating system killing your program. The next frame would have spilled past the slice of memory reserved for the stack, into a region your process is not allowed to touch; the CPU raised a fault and the OS shut the program down. (The addendum to this lecture explains the machinery — the *page table*, the *MMU* — behind that fault; it is the same machinery that produces the segfault from a bad pointer in Chapter 9.)

💡 The compiler often warns you first

For a recursion this blatant, you do not even have to run the program to find the bug. `g++` with `-Winfinite-recursion` (included in `-Wall` on recent versions) flags `descend` at *compile time*: the function calls itself on every path and has no way to stop. The fix is one line — a base case, exactly like `factorial`'s `if (n <= 1) return 1;`. When the compiler tells you a function can never return, believe it.

8.5 What a `std::vector` actually allocates

Now the central picture of the chapter — the one that explains every `std::vector` you have written since Chapter 3.

```
int tag = 42; // a plain local
std::vector<int> v{10, 20, 30}; // <-- where does this live?
```

A `std::vector` is **two things in two places**:

1. **The vector object `v` itself** is a small *handle* that lives on the **stack**, right next to `tag`. It is about three machine words — roughly 24 bytes on a 64-bit machine — and it holds three pieces of information:
 - a **pointer** to where the elements are stored,
 - the **size** (how many elements there are right now),
 - the **capacity** (how many it can hold before it has to grow).
2. **The elements 10 20 30** live somewhere else entirely: on the **heap**, in a contiguous block that the pointer inside `v` points at.

STACK

```
tag = 42
v: [ ptr | size = 3 | cap = 3 ]
```

HEAP

```
[ 10 | 20 | 30 ]
```

The surprising number is the size of the handle. `sizeof(v)` is 24 — three words — **no matter how many elements the vector holds**. A vector of three ints and a vector of three million ints have the *same-sized handle on the stack*; the difference is entirely in how big the heap block its pointer points at is.

i This is the shape of *every* container

`std::vector` is not special here. `std::string` is the same shape — a small handle on the stack (pointer, length, capacity) pointing at the characters on the heap. So are `std::map`, `std::unordered_map`, and most of the standard library’s containers. Once you see “small handle on the stack, buffer on the heap,” you have seen the layout of nearly everything you will store data in this term.

This picture also *explains the advice you have already been following*. Back in Chapter 3 we told you to iterate large elements with `const auto&` and to pass big objects by `const&`. Here is why: **copying a vector copies the handle *and* the heap buffer**. Passing one by value duplicates all the elements; passing it by `const&` passes only a reference to the existing handle — no buffer copy at all. The cost model you were asked to take on trust is right here in the two-region picture.

And the pointer inside the handle — the one that crosses from the stack down to the heap — is *exactly* the thing Chapter 9 makes explicit. A vector is, among other things, a well-behaved wrapper around a pointer you have not been allowed to see yet.

⚠ Capacity is not size

`size()` is how many elements the vector currently holds; `capacity()` is how many the current heap block can hold before the vector must allocate a bigger one and copy everything over. When you `push_back` past the capacity, the vector grows — typically doubling — which means a *new, larger* heap block, a copy of the old elements into it, and the old block freed. This is why `push_back` is cheap *on average* but occasionally expensive, and why `reserve(n)` (pre-sizing the buffer once) is the standard move when you know roughly how many elements are coming. We return to this when we discuss amortized cost in Chapter 16.

8.6 Seeing both regions in GDB

None of this is something you have to believe on the strength of a diagram. The debugger will show you the real addresses on your own binary. The commands below are the minimum needed to make the picture concrete; the full debugger command card lives in the Lab Tooling appendix at Appendix C.

Build with `-g` so the debugger has symbol information, and with `-O0` so the compiler does not optimize the variables out from under you:

```
$ g++ -std=c++20 -O0 -g -o l8_factorial l8_factorial.cpp
$ g++ -std=c++20 -O0 -g -o l9_vector l9_vector.cpp
```

8.6.1 The stack growing: `bt full` on recursion

First, the stack. Set a breakpoint inside the recursive `factorial` from Chapter 7, run, and continue a few times to get partway into the recursion. Then ask for a full backtrace:

```
(gdb) break factorial
(gdb) run
(gdb) continue          # a few times, deeper into the recursion
(gdb) bt full
#0 factorial (n=1) at l8_factorial.cpp:18
#1 factorial (n=2) at l8_factorial.cpp:21
#2 factorial (n=3) at l8_factorial.cpp:21
#3 factorial (n=4) at l8_factorial.cpp:21
#4 main () at l8_factorial.cpp:25
```

`bt` is short for **backtrace**: the live stack, top (most recent) frame first. Each `#k` is one **stack frame** — one call still in progress, with its own copy of the local `n`, its own saved registers, and the return

address that `ret` will jump back to. Read the listing bottom-up and you can watch the recursion descend: `main` called `factorial(4)`, which called `factorial(3)`, down to `factorial(1)`, which is where execution is paused. This is the real-x86 twin of the toy machine’s stack panel from Chapter 7 — the frames really did stack up, one per call.

8.6.2 Raw bytes: `x/8xg`

You can go one level lower and look at the stack as undifferentiated memory. `rsp` is the register that points at the top of the stack:

```
(gdb) info registers rsp
rsp    0x16b0e6a90          # top of the stack -- a HIGH address
(gdb) x/8xg $rsp
0x16b0e6a90:  0x0000000000000002  0x00000000100003f...
               ^ a saved n=2      ^ a return address
```

The `x/8xg` command reads **examine 8 giant** (8-byte) words, in hex, starting at the address in `rsp`. What you are looking at *is* the saved `n` values and return addresses from the backtrace, as raw bytes. (Use `x/16xw` to read 4-byte **words** instead — the right width for looking at `int` layout, which Lab 3 leans on.) Note the address: `0x16b...` — a high number, because the stack lives at the top of memory.

Reading memory in a debugger is not magic

The entire skill is three choices: pick an **address**, pick a **format** (width and base), and *look*. `x/8xg $rsp` is “8 eight-byte words in hex, starting at the stack top.” Once you internalize that the `x` command is “examine *this many*, of *this size*, in *this base*, starting *here*,” the debugger stops being intimidating and becomes a window. The full set of `x` format letters is in Appendix C.

8.6.3 The vector picture, made real

Now the payoff. `19_vector.cpp` declares the `tag` and `v` from the layout section; break after `v` has been filled and print some addresses:

```
(gdb) p &tag                # address of the plain local
$1 = 0x16b0e6b24            # STACK -- a high address
(gdb) p &v                  # address of the vector OBJECT (the handle)
$2 = 0x16b0e6b08            # STACK -- right next to tag
(gdb) p sizeof(v)
$3 = 24                    # 3 words (ptr, size, cap) -- NOT 3 ints
```

8 The layered machine II: stack, heap, and the lifetime question

```
(gdb) p v.data()          # where the ELEMENTS live
$4 = 0x1055edac0          # HEAP -- far away, a LOW address
(gdb) x/3xw v.data()
0x1055edac0: 0x0000000a 0x00000014 0x0000001e  # 10 20 30 in hex
```

Read the addresses. `&tag` and `&v` are both `0x16b...` — high, on the stack, inches apart. But `v.data()`, the pointer to the elements, is `0x105...` — *low*, on the heap, megabytes away. `sizeof(v)` is 24 regardless. And `x/3xw v.data()` confirms the elements really are sitting there: `0x0a`, `0x14`, `0x1e` are 10, 20, 30 in hexadecimal. The diagram from Section 8.5 is not a metaphor; it is a screenshot of your own running program.

i llldb on a Mac maps over one-for-one

The native debugger on macOS is `lldb`, not `gdb`. The moves translate directly: `break` → `b`, `bt full` → `bt`, and `x/8xg $rsp` → `memory read -s8 -fx -c8 $sp`. The course’s consistent GDB experience lives in the Linux VM (Lab 3); the Appendix C card notes the `lldb` equivalents where they matter.

8.7 Addresses are values

Here is the observation that the next chapter is built on. Look again at what GDB just printed:

```
(gdb) p &tag
$1 = 0x16b0e6b24
(gdb) p v.data()
$4 = 0x1055edac0
```

Both of those are **numbers**. `0x16b0e6b24` is the address of `tag` — and it is a value you can print, store, and compare, exactly like 42 or 3.14. The pointer hidden inside `v` is also just a number: `0x1055edac0`, the address where the elements begin. The vector *uses* that number every time you index into it — `v[2]` means “start at the address `data()`, step forward two ints’ worth of bytes, read what is there.”

So a perfectly natural thing to want is a *variable whose value is an address*. Python never gave you one — its references are invisible and unmanipulable. C++ does. A variable whose value is an address is called a **pointer**, and it is the whole subject of Chapter 9. Everything in this chapter has been quietly seeding that idea: the stack and heap are two regions of *addressable* memory, a vector is a handle that *holds an address*, and the debugger let you *print addresses as the numbers they are*.

 A pointer is a sharp tool

The reason we have spent a whole chapter on the memory picture before introducing pointers is that a pointer is the first language feature this course gives you that can corrupt memory if you misuse it. An address that points at nothing valid — because the thing it pointed at was freed, or never existed — is a *dangling* or *null* pointer, and dereferencing one hands the hardware an address it cannot translate. That is the segfault from Section 8.4, seen from the other side. You will meet pointers in Chapter 9 with this picture already in place so that “what does this address actually refer to?” is always a question you know how to ask.

8.8 The lifetime question

We can now state the one thing this chapter cannot resolve, cleanly enough that the next two chapters have a target.

The stack manages itself. A local’s lifetime is exactly the function call that created it: the frame is reserved on entry and released on **ret**, and you never think about it. That automatic discipline is why nothing in chapters 1 through 7 ever asked you to free anything.

But the heap is not automatic. The **vector**’s elements live there, and *someone* allocated that heap block when the vector was filled. So:

Who frees the heap memory, and when?

For **std::vector** specifically, you have already been getting the right answer for free. When **v** goes out of scope — when the function holding it returns and its stack frame is released — the vector’s **destructor** runs, and *that* destructor frees the heap buffer. The handle on the stack disappears automatically, and on its way out it hands back the heap block it owned. This pattern — tie a heap resource’s lifetime to a stack object’s lifetime, so that the stack’s automatic cleanup drives the heap’s manual cleanup — is called **RAII** (Resource Acquisition Is Initialization), and it is the single most important idiom in C++. You have been benefiting from it since your first **std::vector** and your first **std::string** without knowing its name.

The questions RAII raises, and that this chapter deliberately leaves open:

- *How*, mechanically, does the vector hold and free that buffer? What is the pointer it carries, in C++ terms, and what are the operations (**new**, **delete**) that allocate and free heap memory by hand? — That is Chapter 9.
- What goes wrong if heap memory is freed *too early* (a dangling pointer), *twice* (a double free), or *never* (a memory leak)? And how do you get RAII’s safety for resources you manage yourself, without writing destructors by hand every time? — That is Chapter 10, and the smart pointers (**std::unique_ptr**, **std::shared_ptr**) it introduces.

Why C++ leaves this to you at all

Python answered “who frees it, and when?” with a garbage collector that you never see and never control, at the cost of run-time overhead and unpredictable pauses. C++ answers it with **ownership**: every heap allocation has an owner whose job is to free it, and the language gives you tools (destructors, RAII, smart pointers) to make ownership explicit and automatic *without* a background collector. The deal is that you must be able to answer “who owns this?” for every heap allocation in your program. The next two chapters are about making that answer easy and safe to give.

8.9 AI touchpoint — narrate a debugger session

Build `19_vector.cpp` with `-O0 -g`, start it under GDB, and run the address- printing session from Section 8.6: `p &tag, p &v, p sizeof(v), p v.data(), x/3xw v.data()`. Copy the *actual* addresses your machine prints into an AI assistant along with this question: *“Here are addresses from my own program. Which of these are on the stack and which are on the heap, and how can you tell from the numbers? Why is the vector handle next to my local but its elements far away?”*

Then push further: *“My `sizeof(v)` came back as 24 even though the vector has three ints. Explain why the handle size does not depend on the number of elements.”* Read the answer against the picture in Section 8.5 and check that it matches what you actually saw — not just what the model expects a textbook vector to look like. This is the **AI as study partner** role from the AI Tools chapter: you bring the real evidence, the model supplies the explanation, and *you* confirm the two agree.

Make the model explain *your* numbers, not generic ones

An AI assistant will happily produce a correct-sounding paragraph about stacks and heaps from memory. The value here is forcing it to reason about the *specific* addresses your binary printed — high versus low, adjacent versus distant. If its story does not fit your numbers, that mismatch is the most useful thing you will learn in the exercise.

8.10 What you should understand by the end of this chapter

- That Python hid the stack/heap distinction behind a garbage collector, and that C++ exposes it in exchange for predictable speed and explicit ownership
- The two memory regions: the **stack** (automatic, LIFO, bounded, grows down, holds locals and call bookkeeping) and the **heap** (explicit, unstructured, large, grows up, holds big or long-lived data)

- That a stack frame is reserved on a call and released on `ret`, with no manual cleanup, and why that makes nested calls clean
- What a **stack overflow** is — runaway recursion with no base case exhausting the bounded stack — and that the compiler often warns about it first
- That a `std::vector` is a **small (~3-word, 24-byte) handle on the stack** holding a pointer/size/capacity, pointing at its **elements on the heap**, and that this is the shape of every standard container
- Why that layout explains the “pass big objects by `const&`” cost model, and the difference between `size()` and `capacity()`
- How to see both regions in GDB: `bt full` for the stack frames of a recursion, `x/8xg $rsp` for raw stack bytes, and `p &v` versus `p v.data()` for the two regions of a vector (full command card in Appendix C)
- That **addresses are themselves values** — printable numbers — which is the seed of the pointer concept in Chapter 9
- The open **lifetime question** — *who frees heap memory, and when?* — and that `std::vector` already answers it via RAII, which Chapter 9 and Chapter 10 make explicit

If you can explain, looking at a GDB session, why `&v` is a high address and `v.data()` is a low one, and why `sizeof(v)` is 24 no matter how many elements `v` holds, you are ready for Chapter 9.

9 Raw pointers, references, and the lifetime problem

The two previous chapters, Chapter 7 and Chapter 8, took the machine apart. You saw instructions run, you saw the stack and the heap, and you saw — in GDB — that a `std::vector` is a small three-word *handle* on the stack pointing at a buffer of elements out on the heap. We printed `&v` and got an address. We printed `v.data()` and got a different address: the one inside the handle, pointing at the buffer. Those two chapters closed on one deliberately unanswered question: *the vector's elements live on the heap — who frees them, and when?* And one exit ticket: *a variable whose value is an address — what would you call it?*

This chapter is the answer to the exit ticket, and the sharp statement of the question. A variable whose value is an address is called a **pointer**, and C++ gives it a type (`int*`) and two operators (`&` and `*`). Its safer, restricted sibling — an **alias** for an object that already exists — is a **reference** (`int&`), which you have actually been using since the Functions chapter without naming it (`const&` parameters were references all along). Once we have both, we can state the *lifetime problem* — leak, dangling, double-free — precisely enough that you will feel the absence of an answer. The answer is the next chapter, Chapter 10.

9.1 Python has no pointers — but it has the harder half already

It is tempting to tell a Python programmer “Python has no pointers” and stop there. That is half true and the wrong half to lead with.

Here is the true half. In Python you never type an address, never see one, never dereference one. There is no `&` and no `*`. The language manages every object's lifetime for you with a garbage collector, and it never hands you a raw machine address to misuse. All of that is real, and all of it is exactly what makes the material in this chapter feel foreign.

Here is the half that is *already familiar*, and the more important half. Consider:

```
a = [1, 2, 3]
b = a          # NOT a copy
b.append(4)
print(a)       # [1, 2, 3, 4] -- a changed too
```

`b = a` did not copy the list. It made `b` a *second name for the same object*. Mutating through `b` mutates what `a` sees, because there is only one list and two names for it. Python calls this nothing — it is just how assignment of objects works — but C++ has a name for “a second name for the same object”: a **reference**. You already have the mental model. You acquired it the first time a Python aliasing bug bit you. What changes in C++ is that the distinction between *a copy* and *another name* becomes something you spell out explicitly, in the type, every time — and that you additionally get a lower-level tool, the pointer, that can do things a reference cannot.

So the plan of this chapter is: **references first** (the safe tool you reach for 90% of the time), **then pointers** (the sharp tool for the other 10%), then the problem that having raw pointers creates and cannot solve.

9.2 References: a second name, bound once

A reference is an *alias* — another name for an object that already exists. You declare one with `&` in the type and you must give it something to refer to at the moment you create it:

```
int x = 42;
int& r = x;    // r is now a second name for x. Not a copy.
```

From here on, `r` and `x` are two names for one `int`. Reading `r` reads `x`; writing `r` writes `x`:

```
r = 7;
std::cout << x << '\n';    // prints 7 -- r IS x
```

There is no `*`, no `&` at the use site, no ceremony. You use `r` exactly as if it were `x`, because it is. Three properties make references the *safe* tool:

1. **A reference can never be null.** It must bind to a real object at birth. There is no such thing as a reference that refers to nothing, so there is no “I forgot to point it somewhere” bug.
2. **A reference binds once and never re-seats.** Once `r` names `x`, it names `x` forever. Writing `r = 7` changes `x`’s *value*; it does not make `r` name a different object. (There is no syntax that would.)
3. **A reference is not its own object.** It does not occupy its own storage you can take the address of as distinct from `x`. `&r` gives you `&x`.

You met references already, on the read-only side, in the Functions chapter and again in the `const auto&` row pattern of Chapter 3. A function parameter written `const std::string& name` is a reference: it lets the function *see* the caller’s string without copying it, and the `const` promises not to modify it. That is the single most common use of references — passing big objects into functions cheaply — and you have been doing it for two chapters.

💡 Pass by `const&` to read, by `&` to modify, by value to copy

The three calling conventions, named:

```
void inspect(const std::vector<int>& v); // read-only, no copy (default)
void scale(std::vector<int>& v);        // modify the caller's vector
void take(std::vector<int> v);         // get your own private copy
```

Reach for `const&` first: it is cheap (no copy) and safe (`const` blocks accidental modification). Use a plain `&` when the function's *job* is to change the caller's object. Use by-value only when you genuinely want an independent copy to mutate locally. We met by-value and `const&` in the Functions chapter; the plain `&` is the new one here.

9.3 Pointers: & and * are inverses

A reference hides the machine. A **pointer** shows it. A pointer is a variable whose value is an *address* — exactly the thing GDB printed for `&v` in Chapter 8, now given a type and two operators.

Two operators, and they are *inverses* of each other:

- `&x` — *address-of*. Read it “the address of `x`.” It produces the location in memory where `x` lives.
- `*p` — *dereference*. Read it “the thing `p` points at.” It follows the address stored in `p` and gives you back the object there.

```
int x = 42;           // a plain int, somewhere on the stack
int* p = &x;          // p HOLDS the address of x
```

The type `int*` reads “pointer to `int`.” (The course style writes the `*` left-aligned against the type — `int* p`, not `int *p` — to say “the type of `p` is *pointer-to-int*.”) Now `p` is a box holding an address; `x` is the box that address points at. Drawing it as a box and an arrow is not a metaphor, it is the literal picture:

p	x
+-----+	+-----+
0x...c	42
+-----+	+-----+
(an address)	(an int)

`*p` follows the arrow:

```
std::cout << x << '\n';    // 42  -- the value
std::cout << &x << '\n';    // 0x... -- x's address
std::cout << p << '\n';     // 0x... -- the SAME address; p holds it
std::cout << *p << '\n';    // 42  -- follow the arrow back to x
```

That `&` and `*` undo each other is worth saying in symbols: `*(&x)` is `x`. Take the address of `x`, then follow it, and you are back at `x`.

The reason pointers are powerful — and dangerous — is that you can write *through* one:

```
*p = 99;                      // write to wherever p points...
std::cout << x << '\n';      // 99  -- ...which is x. x changed.
```

We did not touch `x` by name. We followed `p` to `x` and overwrote it. This is the same power a reference gives you (`r = 99` would do the same), with one syntactic difference — the pointer needs the `*` to reach through, the reference does not — and several *capability* differences, which are the whole point of having both.

9.4 Pointer vs. reference: nullable, re-seatable, arithmetic

A reference is a pointer with the dangerous parts removed. Three things a pointer can do that a reference cannot:

A pointer can be re-seated. Assigning to a pointer changes *which object it points at*. Assigning to a reference changes the *value of the object it names*.

```
int x = 42;
int y = 1000;
int* p = &x;    // p points at x
p = &y;         // p now points at y -- legal, p was re-seated
// int& r = x; r = y; // this assigns y's VALUE into x; r still names x
```

A pointer can be null. It can point at *nothing*. The literal for “points at nothing” is `nullptr`:

```
int* nothing = nullptr; // a valid pointer that points nowhere
```

A reference has no equivalent — there is no “refers to nothing” reference. This is precisely why a reference is safer: the “I forgot to set it” failure mode is unrepresentable.

A pointer supports arithmetic. You can add an integer to a pointer to step it through memory (`p + 1` is “the address one `int` further along”). References cannot. We will see this — and regret it — in the array section.

 Which to reach for, by default

Default to a reference. Reach for a pointer only when you genuinely need one of the three capabilities above:

- “*maybe nothing*” — a function that might or might not return a result, a field that is sometimes absent. (`nullptr` is the “absent” value.)
- *re-seating* — a variable that needs to point at different objects over its lifetime (a cursor walking a linked list, which is Chapter 11).
- *pointer arithmetic* — almost never, in modern code; the containers do it for you.

If none of those apply — and most of the time none do — use a reference. It cannot be null, cannot be re-seated by accident, and reads more cleanly. The pointer is the tool you keep in the drawer for the genuine 10%.

9.5 `nullptr` and the billion-dollar mistake

A `nullptr` is a pointer pointing at nothing. Printing it is fine. *Dereferencing* it — writing `*nothing` to follow an arrow that goes nowhere — is undefined behavior, and in practice it crashes your program with a **segmentation fault**:

```
int* nothing = nullptr;
std::cout << nothing << '\n'; // fine: prints 0 (or "0x0")
std::cout << *nothing << '\n'; // CRASH: segfault
```

The address 0 is not memory your program is allowed to touch. The CPU’s memory hardware notices the access and the operating system kills the process. The xkcd the course opens this lecture with (#138, “Pointers”) draws exactly this: a sentence that dereferences something it shouldn’t and stops mid-word — “*...all humans ar- SEGMENTATION FAULT.*”

The null pointer has a famous origin story, and it is worth knowing because the person responsible apologized for it in public. Tony Hoare — the inventor of Quicksort and a Turing Award winner — added the null reference to the ALGOL W language in 1965, “simply because it was so easy to implement.” In a 2009 talk he called it his **billion-dollar mistake**: the decades of crashes, security holes, and lost work that trace back to “a pointer might be null and you might forget to check.” Every segfault-from-null you ever cause is a small instalment on that bill.

 Check for null before you dereference — and `not p` is how we write it

Whenever a pointer *might* be null, you must check before dereferencing:

```
if (p not_eq nullptr) {      // p is real; safe to follow it
    std::cout << *p << '\n';
}
```

But the idiomatic spelling in this course leans on the fact that a pointer converts to `bool` — a non-null pointer is “true,” a null pointer is “false.” So the null check reads:

```
if (not p) {                  // p is null -- bail out
    return;
}
std::cout << *p << '\n';    // here, p is guaranteed non-null
```

This is one place the course’s `and/or/not` spelling earns its keep: `if (not p)` reads as the English “if there is no `p`,” where the symbolic `if (!p)` hides the test in a single easily-missed character. (Recall from Chapter 2 that `not` is exactly `!`, `and` is `&&`, `or` is `||` — named here on first use in a pointer context, identical machine code, and you must read both fluently because every library writes the symbols.)

9.6 Raw arrays and decay: the `sizeof` lie

We met C-style arrays in Chapter 3 and concluded “we do not write these in 2026.” Now that you have pointers, you can see *why* — the reason is that a raw array is really a pointer wearing a disguise, and the disguise slips off at the worst possible moment.

A raw array is a fixed block of elements with no handle around it:

```
int a[5] = {10, 20, 30, 40, 50};
```

It has three sins, and all three follow from the same fact: **an array name decays to a pointer to its first element** the instant you do almost anything with it.

Sin 1: it decays, and the length is lost. Pass an array to a function and it arrives as a bare pointer — the `[5]` evaporates. The function has no way to know how long it is:

```
void sins(int arr[]) {        // arr is really int*
    std::cout << sizeof(arr) << '\n';    // 8 -- the size of a POINTER
}

int main() {
    int a[5] = {10, 20, 30, 40, 50};
    std::cout << sizeof(a) << '\n';    // 20 -- five ints, in its home scope
}
```

```
sins(a);                                // but inside sins(), the length is gone
}
```

This is the **sizeof lie**: `sizeof(a)` is 20 in `main` (where `a` is still an array) but `sizeof(arr)` is 8 inside `sins` (where it has decayed to `int*`, and 8 is the size of a pointer on a 64-bit machine). The compiler is so sure this is a mistake that `g++ -Wall -Wextra` warns about it directly: `-Wsizeof-array-argument`. As in Chapter 8, the warning *is* the lesson.

Sin 2: `a[i]` is `*(a + i)`, with no bounds check. Subscripting is pure syntactic sugar over pointer arithmetic. `a[2]` is *defined* to mean “take the address `a`, step two `ints` forward, dereference”:

```
std::cout << a[2]      << '\n';    // 30
std::cout << *(a + 2) << '\n';    // 30 -- literally the same operation
std::cout << (a == &a[0]) << '\n'; // 1 (true): a IS the address of its first element
```

And because it is just `*(a + i)`, nothing checks that `i` is in range. `a[7]` on a five-element array computes `*(a + 7)`, reads two `ints` past the end, and hands you whatever bytes happen to be there — garbage, or a crash, with no warning:

```
std::cout << a[7];    // UNDEFINED BEHAVIOR: reads past the end, no check
```

This is the same undefined behavior that bit `grade_v3` in Chapter 3 at the 100 input, with the protection turned down even further: a raw array gives you *no* checked alternative at all.

 The index expression is the #1 bug hotspot — and raw arrays make it worse

In Chapter 3 we said the expression *inside* `[]` is the first thing to check when a program misbehaves, and that `std::vector` at least offers `.at()` as a checked alternative when you are not certain the index is in range. A raw array offers *nothing*. There is no `a.at(i)`. There is no `a.size()`. There is no bounds check anywhere, ever — the language will happily compute `*(a + 7)` and move on. An out-of-bounds *write* through a raw array (`a[7] = 0;`) does not just read garbage; it *corrupts* whatever was sitting past the end, which is the classic **buffer overflow** at the root of a large fraction of real-world security vulnerabilities. This is the single strongest reason the course writes `std::vector`, not raw arrays.

Sin 3 (recap): it cannot grow, cannot be returned, cannot remember its size. These were the points of Chapter 3. The new understanding is *why*: an array is barely more than a pointer plus a compile-time length that the language forgets the moment it can.

9.6.1 Why `std::vector` exists, in one sentence

Everything above motivates a single sentence you should be able to say cold: **`std::vector` exists because it carries its own size, never decays to a bare pointer, and offers `.at()` for a**

checked subscript. It is the same buffer of elements a raw array would give you — out on the heap, exactly as Chapter 8 showed — wrapped in a handle that *remembers how long it is* and *survives being passed to a function*:

```
void safe(const std::vector<int>& v) {
    std::cout << v.size() << '\n';    // works: the vector knows its length
}

std::vector<int> v{10, 20, 30, 40, 50};
safe(v);                                // size survives the call -- no decay
v.at(7);                                // throws std::out_of_range -- checked
```

This is the “build to learn, use to ship” rhythm of the course made concrete: you write a raw array *once* to see the mechanism, and you ship `std::vector` because it is the same mechanism with the holes plugged.

9.7 The lifetime problem

So far every object we have pointed at — `x`, `y`, the elements of `a` — has been managed *for* us. A local variable lives until its enclosing block ends; the compiler emits the code to reclaim its stack space automatically. The vector’s heap buffer is freed automatically too, when the vector’s handle goes out of scope (that was the unanswered “who frees it?” of Chapter 8; the short answer is “the vector’s destructor,” and the longer answer is the RAII idea from Section 5.6).

But pointers let you reach memory that *nobody manages for you*. You can ask the heap directly for memory, and when you do, **you** become responsible for giving it back. This is where the lifetime problem lives.

9.7.1 new and delete, shown exactly once

Here is the raw mechanism, shown once and then retired from the rest of the book:

```
int* p = new int(42);    // ask the heap for one int, value 42; p points at it
std::cout << *p << '\n';
delete p;                // give it back -- exactly once, at the right time
p = nullptr;             // defensively: a deleted pointer should point nowhere
```

`new int(42)` allocates one `int` out on the heap, initializes it to 42, and returns its address. `delete p` reclaims that heap memory. **This is what `std::vector` does for you, underneath** — its constructor calls something like `new` to get its buffer, and its destructor calls something like `delete` to release it, on schedule, every time, with no chance to forget. When you write a `std::vector`,

you are leasing that discipline. When you write `new/delete` by hand, you are taking the discipline back — and taking on the three ways it can go wrong.

i You will rarely write `new/delete` again after this page

This is the one and only place in the book we write raw `new/delete`. They are shown so that you understand what a `std::vector` (and, next chapter, a `std::unique_ptr`) is automating. From Chapter 10 onward, the rule is: *owning a heap object is the container's job, not yours*. If you find yourself typing `new` in your own code after this chapter, that is a signal to stop and reach for the safe wrapper instead.

9.7.2 Three failure modes raw pointers cannot prevent

A raw pointer carries an address and *nothing else*. It does not record who owns the memory, or whether it has already been freed. That missing information is exactly what makes the following three bugs possible — and all three are undefined behavior.

Leak — you never delete. The memory is allocated and then forgotten. It is never returned to the heap, so your program's memory footprint grows for as long as it runs:

```
int* leaked = new int[1'000'000];
// ... and we never delete[] it. The million ints are lost until the
// process exits. In a server that runs for weeks, leaks are fatal.
```

Dangling — you delete, then use. You free the memory, then follow a pointer to it anyway. The pointer still holds the old address, but that address now belongs to nobody (or to something else). Reading or writing through it is a *use-after-free*:

```
int* d = new int(7);
delete d;                // freed...
std::cout << *d;         // ...but read anyway. UNDEFINED BEHAVIOR.
```

This is the “dangling pointer” footgun the course names on this day: a pointer to memory that has already been freed. It reads fine — `d` still looks like a perfectly good pointer — but it points at a corpse.

Double-free — you delete twice. Two pieces of code both believe they own the same memory, and both free it. The first `delete` is correct; the second corrupts the heap's internal bookkeeping:

```
int* e = new int(7);
delete e;
delete e;                // freed twice. heap corruption. UNDEFINED BEHAVIOR.
```

9.7.3 The question raw pointers cannot answer

Look at a bare pointer in isolation:

```
void process(int* p);    // ...is this function supposed to delete p? or not?
```

You cannot tell. Does `process` *own* `p` (and so must free it), or is it just *borrowing* `p` to read it (and must leave it alone)? Two reasonable programmers will guess differently. If both think they own it: double-free. If neither thinks they own it: leak. If one frees it while the other still holds it: dangling. **The type `int*` does not say who owns the pointee, or when it should be freed.** That missing answer is the lifetime problem, and no amount of careful manual `delete`ing makes it go away — because the discipline lives in your head and your comments, not in the code the compiler checks.

Hold that question. It is the entire setup for Chapter 10, where the answer is to *put ownership into the type* — `std::unique_ptr<int>` is a pointer that owns its pointee and frees it automatically, and `std::shared_ptr` is one that is shared and freed when the last owner goes away — so that “who frees it, and when?” becomes a question the compiler answers for you, the same way RAII (Section 5.6) already answered it for the vector.

9.8 Watching undefined behavior fire: AddressSanitizer

“Undefined behavior” sounds abstract until you see a tool catch it in the act. The dangling-pointer bug above is the perfect demonstration, because by default it *does not crash* — it reads stale heap memory and prints a plausible-looking number, and the program continues as if nothing happened. That silence is what makes use-after-free so dangerous: the bug is real, the answer is wrong, and nothing told you.

AddressSanitizer (ASan) breaks the silence. It is a feature built into `g++` and `clang`; you turn it on at compile time with two flags, `-fsanitize=address` and `-g` (the `-g` gives it line numbers):

```
$ g++ -std=c++20 -Wall -Wextra -fsanitize=address -g lifetime.cpp -o lifetime
$ ./lifetime
```

With the dangling block uncommented, ASan stops the program at the exact moment of the bad access and prints a diagnosis:

```
==12345==ERROR: AddressSanitizer: heap-use-after-free on address 0x...
READ of size 4 at 0x... thread T0
    #0 ... in main lifetime.cpp:41
0x... is located 0 bytes inside of 4-byte region
freed by thread T0 here:
```



```
#1 ... in main lifetime.cpp:40    <- the delete
previously allocated by thread T0 here:
#2 ... in main lifetime.cpp:39    <- the new
```

Read what it gives you: the *kind* of bug (**heap-use-after-free**), the line that did the bad read (41), the line that freed the memory (40), and the line that originally allocated it (39). It has reconstructed the whole crime scene. The same flags catch the leak (reported at program exit) and the double-free.

This is your single most important debugging habit for memory bugs, and it costs two flags. The full reference — the other sanitizers, what each catches, how to read the reports, and how to wire ASan into your build — is Appendix B.

💡 AI touchpoint — make the sanitizer report legible

ASan reports are dense, and the first few you read will be intimidating. This is a textbook use of the **AI as IT aide** role. Build `lifetime.cpp` with `-fsanitize=address -g`, uncomment the dangling block, run it, and paste the *entire* **heap-use-after-free** report into an AI assistant with the question: “*What does this report mean, line by line, and which line of my code caused it?*” Then — and this is the part that makes it learning rather than copying — **check its answer against what you already know you did**. You know you deleted `d` and then read `*d`; does the AI’s explanation match? Confirming a tool’s explanation against your own understanding is how you learn to read these reports without it. The disclosure rules for this kind of use are in the AI Tools chapter.

i Why this is a security beat, not just a correctness one

Dangling pointers and out-of-bounds array writes are not merely “your program gives a wrong answer.” They are the **memory-safety bug class** behind a large share of real-world security vulnerabilities (CVEs): an attacker who can steer a use-after-free or a buffer overflow can often steer the whole program. This is one reason newer languages like Rust enforce ownership at compile time, and one reason this course ships `std::vector` and (next chapter) smart pointers instead of raw arrays and raw `new`. ASan is the cheap catch in the meantime. This is an awareness beat; the dedicated treatment belongs to a later security course.

9.9 What you should understand by the end of this chapter

- That Python’s “`b = a` is not a copy” is the same idea as a C++ **reference**: a second name for one object
- A **reference** (`int& r = x`) is an alias — it cannot be null, binds once and never re-seats, and is the safe tool you reach for by default

9 Raw pointers, references, and the lifetime problem

- Passing by `const&` (read), `&` (modify), and by value (copy), and which to use when
- That `&x` (address-of) and `*p` (dereference) are **inverse** operations, and that writing `*p = ...` writes through the pointer to the pointee
- The three things a **pointer** can do that a reference cannot — be null, be re-seated, and do arithmetic — and that you reach for a pointer only when you need one of them
- `nullptr`, why dereferencing it segfaults, Hoare’s “billion-dollar mistake,” and the idiomatic `if (not p)` null check
- **Array-to-pointer decay**: that a raw array name becomes a bare pointer when passed, that `sizeof` therefore lies inside a function, and that `a[i]` is literally `*(a + i)` with no bounds check
- Why `std::vector` exists, in one sentence (carries its size, never decays, `.at()` checks)
- `new/delete` as the raw mechanism a `std::vector` automates — shown once and then retired
- The three lifetime failure modes — **leak**, **dangling**, **double-free** — and that each is undefined behavior
- The open question raw pointers cannot answer: *given an `int* p`, who owns it and when is it freed?*
- That AddressSanitizer (`-fsanitize=address -g`) catches use-after-free, leaks, and double-frees, and how to read its report (full reference: Appendix B)

If you can state the lifetime problem in your own words — *a raw pointer holds an address and nothing else, so it cannot tell you who owns the memory or when to free it* — and you can feel why that is a problem the language ought to solve for you, you are ready for Chapter 10, where it does.

10 Modern pointers: `unique_ptr`, `shared_ptr`, and `move`

The previous chapter, Chapter 9, ended on an unanswerable question. We gave the address a C++ type — `int*` — and showed `new` and `delete` exactly once, as “what `std::vector` does for you underneath.” Then we listed the three failure modes a raw pointer cannot prevent: a **leak** (you never `delete`, and the memory is gone until the program exits), a **dangling pointer** (you `delete`, then keep using the address, which now points at a corpse), and a **double-free** (two pieces of code both think they own the object, both `delete` it, and the heap corrupts). Each one is *undefined behavior* — the program does not error out; it produces *some* answer, on *some* machines, some of the time.

And underneath all three sat one question you could not answer by looking at a bare `int* p`:

Who owns this pointer? When is it freed?

This chapter is the answer. The answer is to **put ownership into the type**. Once a pointer’s type says “I own this, and I will free it on schedule,” the compiler can enforce the rule mechanically, and all three bugs from Chapter 9 become *impossible to write*. The mechanism that does this is not new — it is **RAII**, the destructor-runs-on-scope-exit idea you first met in Section 5.6. There, a class’s destructor freed the buffer it had allocated. Here we generalize: a destructor can free *any* resource — heap memory, a file handle, a network socket, a lock — on a schedule the language guarantees.

i You already know this idea from Python — you just never chose it

In Python, when you write `a = SomeObject()` and the last name referring to that object goes away, CPython’s reference counter hits zero and the object is freed. You never call `delete`; you never think about it. C++’s `std::shared_ptr`, which we reach near the end of this chapter, is *essentially what CPython does for every object* — a reference count that frees on the last release.

The difference is that in C++ you **choose** this behavior, deliberately, per object, because it is not free. CPython pays the reference-counting cost on every object whether you need it or not. C++ lets you pay it only where shared ownership is genuinely the design — and the rest of the time gives you something cheaper. This chapter is largely about learning *which* tool to choose.

10.1 `std::unique_ptr` — the default owning pointer

Start with the common case: exactly one part of your program owns a heap object and is responsible for freeing it. The tool is `std::unique_ptr<T>`, and you construct one with `std::make_unique<T>(args...)`. Both live in the `<memory>` header.

```
#include <memory>

struct Widget {
    std::string name;
    explicit Widget(std::string n) : name{std::move(n)} {}
    void use() { std::cout << "using " << name << '\n'; }
    ~Widget() { std::cout << "~Widget(" << name << ")\n"; }
};

void demo() {
    auto a = std::make_unique<Widget>("A"); // heap Widget, owned by a
    a->use();                               // use it exactly like a pointer
    // ... no delete anywhere ...
}                                           // a goes out of scope here:
                                           // ~Widget(A) runs automatically
```

Read what just happened. `std::make_unique<Widget>("A")` allocates a `Widget` on the heap, forwards "A" to its constructor, and hands you back a `unique_ptr<Widget>` that *owns* that object. The `auto` lets the compiler write out the unwieldy type `std::unique_ptr<Widget>` for you.

You then use `a` exactly as if it were a raw pointer: `*a` dereferences to the `Widget`, and `a->use()` calls a member through it. The arrow `->` and the star `*` mean the same thing they did for a raw `Widget*` in Chapter 9 — `unique_ptr` is designed to feel like a pointer at the point of use.

The payoff is the last line. When `a` goes out of scope at the closing brace of `demo()`, the `unique_ptr`'s own destructor runs, and *it* calls `delete` on the `Widget` for you. You see `~Widget(A)` print. **You wrote no delete.** There is no code path — not an early `return`, not a thrown exception, not a `break` out of the middle of a loop — on which the `Widget` leaks, because the destructor of a stack object *always* runs when that object's scope ends. That is RAII, and it is exactly the guarantee Section 5.6 gave you for a hand-written class; here the standard library wrote the class for you.

Prefer `make_unique` to a bare `new`

You *can* write `std::unique_ptr<Widget> a{new Widget("A")};` and it does the same thing. Prefer `std::make_unique<Widget>("A")` anyway, for three reasons:

1. **You write the type once, not twice.** `make_unique<Widget>` names `Widget` a single

time; the `new` form names it on both sides.

2. **It is exception-safe in a way the `new` form is not.** In an expression that constructs two arguments, a bare `new` can leak if the *other* argument’s evaluation throws before the `unique_ptr` is built. `make_unique` closes that window.
3. **It is one allocation, and it reads as a single intent** — “make me a uniquely-owned `Widget`” — rather than “allocate, then wrap.”

After this chapter, a bare `new` should look as out-of-place in your code as a C-style array did after Chapter 3. You saw `new/delete` once, in Chapter 9, to understand what is being automated. From here it is retired.

10.1.1 Move-only: there is exactly one owner

Here is the property that makes `unique_ptr` *unique*: **you cannot copy it.**

```
auto b = std::make_unique<Widget>("B");
// auto c = b;           // COMPILE ERROR: the copy constructor is deleted
auto c = std::move(b);    // OK: TRANSFER ownership from b to c. b is now empty.
```

Think about why copying must be forbidden. If `auto c = b;` made a *second* `unique_ptr` pointing at the same `Widget`, then when both went out of scope, *both* destructors would run, and the `Widget` would be *deleted* twice. That is precisely the double-free bug from Chapter 9. So the language does not let you write it: `unique_ptr`’s copy constructor is **deleted**, and the line `auto c = b;` does not compile. The compiler refuses the bug at the source.

What you *can* do is **move** ownership from one `unique_ptr` to another with `std::move`. After `auto c = std::move(b);`, `c` owns the `Widget` and `b` owns nothing — `b` now holds `nullptr`. Ownership was *transferred*, not duplicated; there is still exactly one owner, it is just a different one. We unpack what `std::move` really is in Section 10.4; for now read it as “hand the owned thing over to the new owner, and leave the old one empty.”

The same mechanism passes ownership *into* a function. A function that takes a `unique_ptr` by **value** is asking to become the new owner:

```
void consume(std::unique_ptr<Widget> w) {
    w->use();
} // w's scope ends here -> the Widget is freed here

auto d = std::make_unique<Widget>("D");
consume(std::move(d)); // hand ownership to consume(); d is now empty
// d is empty here; the Widget was already freed inside consume()
```

You had to write `std::move(d)` explicitly. That is the language making you *say out loud* that you are giving the object away — there is no silent transfer of ownership. After the call, `d` is empty, and the `Widget` was destroyed when `consume`’s parameter `w` went out of scope.

💡 AI touchpoint — read the deleted-copy-constructor error in full

Type the forbidden line into a real file and compile it:

```
auto b = std::make_unique<Widget>("B");
auto c = b;    // <-- the bug
```

`g++ -std=c++20 -Wall -Wextra` will refuse it with a message containing something like `use of deleted function 'std::unique_ptr<...>::unique_ptr(const std::unique_ptr<...>&)'`. The error is *long* and mentions templates you have not met. Paste the whole thing into an AI assistant and ask: “*I’m a student learning C++. Why won’t this compile, and what is the language trying to stop me from doing?*” Then **check the answer against this section** — does it say “copy would create two owners, which would double-free”? This is the **AI as IT aide** role from the AI Tools chapter: the assistant turns a wall of template noise into the one sentence that matters, and you verify it rather than trusting it.

10.1.2 `unique_ptr` kills all three lifetime bugs at once

Line the three failure modes up against `unique_ptr` and watch each one become unwritable:

Failure mode from Chapter 9	Why <code>unique_ptr</code> makes it impossible
Leak	The destructor <i>always</i> runs on scope exit (RAII). There is no path that forgets to free.
Dangling	The object outlives a use only if you <i>moved</i> ownership somewhere it stays alive; you cannot accidentally use a freed object that you still “own.”
Double-free	The copy constructor is deleted — there is only ever one owner, so delete runs exactly once.

This is the central claim of the chapter. The three undefined-behavior bugs that made raw pointers dangerous are not *caught at run time* by `unique_ptr` — they are *prevented at compile time* by the shape of the type. A bug the compiler will not let you write is strictly better than a bug a sanitizer catches after you wrote it.

i A raw pointer is still fine — as a non-owning *borrow*

Retiring `new/delete` does not retire the raw `T*`. It retires raw *ownership*. A raw pointer is perfectly good as a **borrow** — a temporary, non-owning peek at an object somebody else owns:

```
auto owner = std::make_unique<Widget>("E");
Widget* borrowed = owner.get();    // a non-owning view; owner still owns it
borrowed->use();                    // fine to use...
// ...but NEVER `delete borrowed;` -- you do not own it
```

`owner.get()` hands you the raw address inside the `unique_ptr` without transferring ownership. The rule is simple and absolute: **a borrowed raw pointer must never be deleted, and must not outlive the owner it borrowed from.** Passing `Widget*` (or, better, `Widget&`) to a function that only *reads* the object is the normal, idiomatic thing — the function borrows, the caller keeps owning. The bug class returns only if you `delete` through a borrow or let the owner die first.

10.2 `std::shared_ptr` — when ownership is genuinely shared

`unique_ptr` says “exactly one owner.” Sometimes that is the wrong model. Sometimes several parts of a program each need an object kept alive, and *none* of them is “the” owner — the object should live as long as *anyone* still needs it, and be freed only when the *last* user is done. That is what `std::shared_ptr<T>` is for, and it works by **reference counting**.

```
auto a = std::make_shared<Person>("Ada");    // use_count() == 1
std::cout << a.use_count() << '\n';          // 1
{
    auto also = a;                            // COPY is allowed: count -> 2
    std::cout << a.use_count() << '\n';        // 2
}                                              // `also` dies: count -> 1
std::cout << a.use_count() << '\n';          // 1
// the Person is freed only when the LAST shared_ptr is gone (count hits 0)
```

The differences from `unique_ptr` are worth stating precisely.

It is copyable. `auto also = a;` is *not* an error here — copying a `shared_ptr` is the whole point. Each copy bumps a shared reference count; each destruction drops it. The managed `Person` is destroyed exactly when the count reaches zero, which is when the last `shared_ptr` pointing at it goes away. You can watch this with `.use_count()`.

It costs more. That reference count is *atomic* — safe to increment and decrement from multiple threads — so every copy and every destruction pays for an atomic operation. For a tight loop that copies *shared_ptr*s, this is real, measurable overhead. A *unique_ptr* move is just a pointer assignment; a *shared_ptr* copy is a synchronized counter bump. This is the CPython cost model, made opt-in.

make_shared is the preferred constructor, for the same reasons *make_unique* is — and a bonus one: *make_shared* can put the object and its reference-count control block in a *single* allocation.

⚠ The cycle trap — a leak even a garbage collector would catch

Reference counting has one famous hole: a **cycle**. If two *shared_ptr*s point at each other, each keeps the other's count above zero forever. Neither ever reaches zero, so neither is ever freed — a genuine leak, in a tool whose whole job was to prevent leaks.

```
struct Person {
    std::string name;
    std::shared_ptr<Person> best_friend;    // STRONG: bumps the count
    ~Person() { std::cout << "~Person(" << name << ")\n"; }
};

auto ada = std::make_shared<Person>();
auto grace = std::make_shared<Person>();
ada->best_friend = grace;    // grace's count is now 2
grace->best_friend = ada;    // ada's count is now 2
// both go out of scope: each count drops 2 -> 1, never to 0.
// NEITHER destructor runs. Both Persons leak.
```

CPython, whose garbage collector has a dedicated cycle detector, would eventually collect this. C++ will not — there is no background collector. Reference counting alone cannot break a cycle. This is the price of the cheaper, deterministic model, and the next tool is how you pay it.

10.3 *std::weak_ptr* — observe without owning

The fix for the cycle is to make *one edge of it* non-owning. A *std::weak_ptr<T>* points at an object managed by a *shared_ptr* **without** contributing to its reference count. It *observes*; it does not own; it does not keep the object alive.

```
struct Person {
    std::string name;
```



```
std::shared_ptr<Person> best_friend;    // strong: owns, bumps the count
std::weak_ptr<Person>  acquaintance;   // WEAK: observes, no count
~Person() { std::cout << "~Person(" << name << ")\n"; }
};
```

Make one direction of a relationship `weak_ptr` instead of `shared_ptr` and the cycle is broken: the weak edge does not hold the other object alive, so when the external owners go away, the counts reach zero and both objects free in order.

Because a `weak_ptr` does *not* keep its target alive, the target may already be gone by the time you look. So you are not allowed to use a `weak_ptr` directly. You must first `.lock()` it, which attempts to promote it to a full `shared_ptr` — and *tells you* whether the object still exists:

```
if (std::shared_ptr<Person> friend_now = p.acquaintance.lock()) {
    // the object is still alive; `friend_now` keeps it alive for this block
    std::cout << friend_now->name << '\n';
} else {
    // the acquaintance has already been freed; nothing to use
    std::cout << "(that acquaintance is gone)\n";
}
```

`lock()` returns a `shared_ptr` that is non-null if the object is still alive and null if it has been freed. This is *safe by construction*: you cannot accidentally dereference a dangling `weak_ptr`, because the language makes you go through `lock()`, and the `if` forces you to handle the “already gone” case. The dangling-pointer bug from Chapter 9, which raw pointers left entirely to your discipline, here cannot happen — the type makes you check.

10.3.1 Which owning pointer? — the decision

Situation	Reach for
One owner (the common case)	<code>std::unique_ptr</code>
A non-owning peek at something already owned	a raw <code>T*</code> from <code>.get()</code> — borrow, never free
Genuinely shared ownership	<code>std::shared_ptr</code>
Observe a <code>shared_ptr</code> ’s object without owning it	<code>std::weak_ptr</code> (and break cycles)

 The rule: `unique_ptr` until you can prove you need `shared_ptr`

Shared ownership is the *exception*, not the default. It costs an atomic counter on every copy, it invites cycles, and — most importantly — it muddies the one question this whole chapter is about. “Who owns this?” should have *one* answer whenever it possibly can. `unique_ptr` gives

a clear answer; `shared_ptr` gives “everyone, a little.” Reach for `shared_ptr` only when you can articulate *specifically* why no single owner can be responsible for the object’s lifetime — and when you cannot, the honest default is `unique_ptr`. Beginners reaching for `shared_ptr` “to be safe” is one of the most common ways modern C++ code goes quietly wrong.

10.4 Move semantics, conceptually

We have used `std::move` twice now without saying what it really is. It is worth slowing down, because move semantics are the idea underneath *everything* in this chapter, and the picture you need was already drawn back in Chapter 8.

10.4.1 Copy vs. move — the `std::vector` cost model

Chapter 8 drew a `std::vector` as a small **3-word handle** sitting on the stack — a pointer to the data, a size, and a capacity — pointing at a **buffer of elements on the heap**. That single picture *is* move semantics:

```
std::vector<int> big = make_a_million_ints();    // 3-word handle, huge heap buffer
std::vector<int> copy = big;                    // COPY: new heap buffer + every element
std::vector<int> moved = std::move(big);        // MOVE: hand over the 3-word handle
```

Copying `big` allocates a *brand-new* heap buffer and copies all one million elements into it, so that `copy` and `big` are fully independent. That is $O(n)$ work and a fresh allocation — expensive for a big vector, and exactly the kind of silent cost that makes people afraid to “return a vector by value.”

Moving `big` does something completely different. It copies the *3-word handle* — data pointer, size, capacity — from `big` into `moved`, and then nulls out `big`’s handle. The million-element heap buffer **never moves**; only its *owner* changes. That is $O(1)$ — three words copied, regardless of how big the buffer is. No allocation, no element copying.

This is precisely why `unique_ptr` is move-only and not copyable. The “handle” a `unique_ptr` transfers is *ownership itself*. There is no cheap way to *copy* ownership (copying it would mean two owners — the double-free), but there is a perfectly cheap way to *move* it: hand the pointer over and null the source. Move semantics and single ownership are the same idea seen from two angles.

10.4.2 `std::move` is a cast — and a promise

Here is the part that surprises people: **`std::move` does not move anything.**

`std::move(x)` is a *cast*. It does no work at run time — it does not touch memory, copy bytes, or free anything. All it does is *mark* `x` as “fair game to pillage,” which tells the compiler it is allowed

to select the cheap **move** path (steal the guts) instead of the expensive **copy** path (duplicate the guts) for whatever operation comes next. The actual stealing is done by the move that the cast enabled, not by `std::move` itself.

```
std::vector<int> big = make_a_million_ints();
std::vector<int> moved = std::move(big);    // the assignment does the stealing

// `big` is now in a VALID but UNSPECIFIED state. For std::vector, that's empty.
std::cout << big.size() << '\n';    // prints 0 -- legal to ask, but don't ASSUME 0
big = {1, 2, 3};                    // perfectly fine: assigning a fresh value is OK
```

So `std::move` is a cast *and* a promise. The promise is: *I am done with `big`'s current value; you may take its contents.* And once you have made that promise, you must keep it.

⚠ A moved-from object is valid, but its value is unspecified

After `std::move(x)`, the object `x` is still a *valid* object — the language guarantees that much. You may safely **reassign** it (give it a fresh value), ask it harmless questions, or let it be **destroyed**. What you may *not* do is **read its old value and expect it to still be there**. For a `std::vector`, the standard says a moved-from vector is left empty, but you should not *write code that depends on* it being empty — treat the value as garbage you promised to stop caring about.

The bug is reading a moved-from object as if the move never happened. The fix is discipline: after you `std::move(x)`, the only honest moves are *reassign it* or *let it go*.

i Why you stopped fearing “return by value”

Coming from Python, you may have inherited a folk rule from older C++ code: “don’t return big objects by value — it copies.” That rule is obsolete. Returning a large local object —

```
std::vector<int> build() {
    std::vector<int> result;
    // ... fill result with a million ints ...
    return result;        // NOT a copy
}
```

— is either a **move** (the $O(1)$ handle transfer) or, more often, *elided entirely* by the compiler (the object is constructed directly in the caller’s slot, so there is no transfer at all). Either way it is free. You can, and should, write functions that return vectors, strings, and `unique_ptrs` by value. The language made it cheap so that you could write the clear thing.

10.5 RAII, generalized

Step back from pointers for a moment, because the real lesson of this chapter is bigger than memory.

Section 5.6 introduced RAII with one specific job: a class allocated a buffer in its constructor and freed it in its destructor, so the buffer’s *lifetime* was tied to the object’s *scope*. `unique_ptr` is exactly that pattern, packaged: its destructor frees the heap object it owns, automatically, on scope exit.

But the destructor does not care that the resource is memory. A destructor is a hook the language **guarantees** to run when an object’s scope ends — on a normal exit, on an early `return`, on a thrown exception, on a `break`. Whatever you put in it runs *on schedule*, with no `delete`-equivalent for you to forget. So the same pattern frees *any* resource that must be released:

- A `std::ofstream`’s destructor **closes the file**.
- A `std::lock_guard`’s destructor **releases the mutex**.
- A `std::unique_ptr`’s destructor **frees the heap object**.
- A database-connection wrapper’s destructor **closes the connection**.

This is the generalization the lecture’s one-liner captured: *a `unique_ptr` is a raw pointer with a conscience — it remembers to free*. And RAII is “the ugliest name for the best idea in C++” — *Resource Acquisition Is Initialization* — which really means: **acquire the resource when you build the object, release it when the object dies, and let scope do the bookkeeping**.

The Python contrast is instructive. Python gives you `with open(...) as f:` — a `with`-block whose `__exit__` closes the file when the block ends. That is RAII with explicit syntax: you must remember to *write the with*. C++ folds the same guarantee into the object’s *type*, so it happens whether or not you remember. The file closes because the `ofstream` went out of scope, not because you wrapped it in anything. Tie the resource to a scope-bound object once, in the type, and every use site gets the guarantee for free.

10.6 Looking ahead: this is the backbone of the next two chapters

You will *need* an owning pointer for the first time in Chapter 11. A linked list is a chain of nodes, each owning the next — and “owns the next” is exactly `std::unique_ptr<Node>`. The same shape is the spine of the binary search tree we build from first principles in Chapter 14: a node owns its left and right children with `unique_ptr<Node>`, and when a node is destroyed, its destructor *recursively* frees its whole subtree, automatically, with no hand-written cleanup code anywhere. The entire memory discipline of those two chapters reduces to “the owner is a `unique_ptr`, and RAII does the rest.” This chapter is what makes that possible.

10.7 What you should understand by the end of this chapter

- `std::unique_ptr<T> + std::make_unique<T>()` as the **default** owning pointer, used through `*` and `->` like a raw pointer, freeing its object automatically on scope exit
- Why `unique_ptr` is **move-only**: copying would create two owners, which is the double-free — so the copy constructor is deleted, and `std::move` transfers ownership and empties the source
- How `unique_ptr` makes all three Chapter 9 bugs — leak, dangling, double-free — *impossible to write* rather than merely caught at run time
- That a raw `T*` from `.get()` is still fine as a **non-owning borrow** that must never `delete`
- `std::shared_ptr` as reference-counted **shared** ownership (the deliberate, opt-in version of what CPython does for every object), what `.use_count()` shows, and that it costs an atomic counter
- `std::weak_ptr` as an **observe-without-owning** pointer that breaks reference cycles, and `.lock()` as the safe-by-construction way to use one
- The rule: **`unique_ptr` until you can prove you need `shared_ptr`**
- Move vs. copy as the $O(1)$ handle transfer vs. the $O(n)$ buffer duplication, using the Chapter 8 vector picture
- That `std::move` is a **cast and a promise**, not an operation, and that a moved-from object is **valid but unspecified** — reassign it or destroy it, never read its old value
- RAII generalized from Section 5.6: a destructor frees *any* resource on schedule, which is why you can stop fearing return-by-value and why Chapter 11 and Chapter 14 can own their nodes with `unique_ptr<Node>`

If you can take L10's hand-managed `new/delete` example, rewrite it with `std::make_unique`, delete every `new` and `delete`, and explain *why* it can no longer leak, dangle, or double-free, you are ready for Chapter 11.

Part IV

Week 4 — Linked structures, modular design, BST climax

Week four assembles everything from weeks 1–3 — classes, recursion, pointers, modular design — into the course’s intellectual climax: a small binary search tree, designed from first principles, that exists as a *learning* artifact (you would reach for `std::set` or `std::map` in production code, and we discuss exactly when). Along the way: linked lists with `unique_ptr<Node>`, the associative containers, file I/O across three idioms, problem decomposition strategies, namespaces as a name-management tool, and a tightly trimmed comparative OOP survey.

The chapters of this part:

- Chapter 11 — singly-linked list with `unique_ptr<Node>`, `std::map` / `std::unordered_map`, table-driven design, the decision-tree demo
- Chapter 12 — streams, comparing `printf` / `std::cout` / `std::format`
- Chapter 13 — top-down/bottom-up/data-first decomposition, namespaces (the *why* and the hard rule against `using namespace std;`), header / public-API discipline, comparative OOP
- Chapter 14 — Tiny BST as a design exercise; the bridge to Algorithms

11 Linked lists, maps, and table-driven design

The previous chapter ended on a cliffhanger. Chapter 10 gave you `std::unique_ptr` — a smart pointer that owns *one* heap object and frees it automatically when it goes out of scope, with no `delete` you have to remember to write. The exit question was the obvious next one:

A linked list is *many* nodes, each owning the *next*. How does `std::unique_ptr<Node>` chain?

This chapter answers it. The linked list is the first data structure where you genuinely *need* an owning pointer — a variable number of nodes, created at run time, each outliving the function that built it. Once we have built one by hand, we do two more things. We pivot to the other everyday structure you reach for constantly — **associative containers** (`std::map` and `std::unordered_map`, which are essentially Python’s `dict`) — and then we step up one level to the *judgment* you exercise when you choose between all of these: **table-driven design** and the **container decision tree**.

The whole chapter is live C++. There are four runnable demos — `ec327/inclass/{l12_list,l12_map,l12_dispatcher}` and the older `datastructures.cpp` — and the prose here is built directly from them. Compile each as you read it:

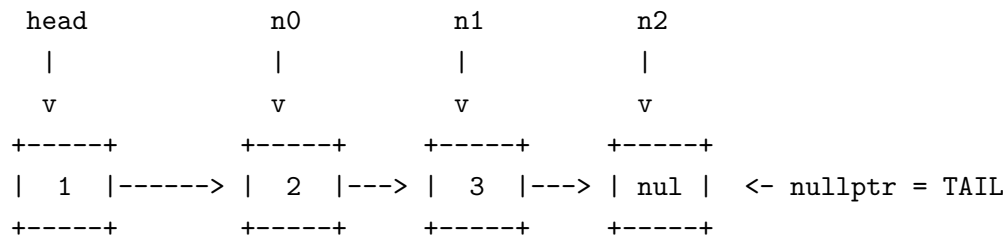
```
$ g++ -std=c++20 -Wall -Wextra -O0 -g l12_list.cpp -o l12_list && ./l12_list
```

11.1 A self-referential struct

A linked list is built from **nodes**. Each node holds a value and a pointer to the *next* node. The pointer is the link. The type that does this is *self-referential*: a `Node` contains a `std::unique_ptr` to another `Node`.

```
struct Node {  
    int value;  
    std::unique_ptr<Node> next; // owns the REST of the list  
};
```

Read the comment carefully, because it is the whole idea. `next` is not “a pointer to the next node” in the loose sense — it is an *owning* pointer. The node that holds it *owns* the entire rest of the list that hangs off it. Draw the chain as boxes and arrows:



Two facts pin the structure down:

It is an ownership chain. `head` owns `n0`; `n0` owns `n1`; `n1` owns `n2`. Every node has exactly one owner — the node before it (or `head`, for the first). This is the single-owner guarantee from Chapter 10, applied node by node down the list.

The tail's next is `nullptr`. The last node points at nothing, and “nothing” is spelled `nullptr`. This is the **end-of-list marker**. In Chapter 9 we named `nullptr` as Tony Hoare’s “billion-dollar mistake” — the null reference he introduced in 1965 and later apologized for, because every segfault traces back to dereferencing one. Here `nullptr` finally earns its keep: it is a *legitimate value* meaning “there is no next node,” and — because `next` is a `unique_ptr`, not a raw pointer — there is no way to leave it dangling or to leak the node it used to point at.

i This is just a `struct`, pointing at itself

There is no new keyword here. `struct Node { ... };` is the same `struct` you met in Chapter 3 — a named bag of fields. The only new thing is that one of the fields is a `unique_ptr<Node>`, i.e. the type names *itself*. That is legal precisely because `next` is a *pointer* to a `Node`, not a `Node` by value: a `Node` that *contained* a whole `Node` would be infinitely large, but a `Node` that contains a *pointer* to a `Node` is a fixed, small size (a value plus one pointer’s worth of bytes).

The demo adds a small constructor so a node can be born already pointing at the old head — you will see why in a moment:

```
struct Node {
    int value;
    std::unique_ptr<Node> next;

    explicit Node(int v, std::unique_ptr<Node> nxt = nullptr)
        : value(v), next(std::move(nxt)) {}
};
```

The `: value(v), next(std::move(nxt))` part is a *member initializer list* (Chapter 5); `explicit` (Section 5.6) blocks an accidental `Node n = 7;`-style **implicit conversion**. The `std::move` is the part that matters for this chapter, and it is the subject of the next section.

11.2 Growing the list: *push_front* moves ownership

To add a node to the *front* of the list, we make a new node whose `next` is the *current* head, and then make the new node the head. Here is the operation:

```
std::unique_ptr<Node> push_front(std::unique_ptr<Node> head, int value) {
    // The new node's `next` becomes the OLD head -- ownership transfers in.
    return std::make_unique<Node>(value, std::move(head));
}
```

The single most important token in those two lines is `std::move`. Recall from Chapter 10 that `std::unique_ptr` is **move-only**: you cannot copy it, because two owners of one object is exactly the double-free bug `unique_ptr` exists to prevent. You can only *move* it — transfer the ownership from one `unique_ptr` to another, leaving the source empty.

So `std::move(head)` says “take ownership of whatever `head` currently owns and hand it to the new node’s `next`.” After the move, the local `head` is empty (`nullptr`), the new node owns the old chain, and there is still **exactly one** owner of every node. No copy happened. No node was duplicated.

Building a list is then a loop of *push_front*s:

```
std::unique_ptr<Node> head; // an empty list IS nullptr

for (const int v : {3, 2, 1}) {
    head = push_front(std::move(head), v); // builds 1 -> 2 -> 3
}
```

We push 3, then 2, then 1, and because each push goes on the *front*, the list comes out in the order 1 -> 2 -> 3. Note the `std::move(head)` at the call site too: we are handing the current list *into* *push_front* and getting the grown list back out. (You could also take `head` by reference and mutate it in place; passing-and-returning the owner is the version that makes the ownership transfer most visible, which is why the demo uses it.)

Why insert at the front? Because it is $O(1)$ — constant time, no matter how long the list already is. Inserting at the *back* would mean walking every node to find the tail first, which is $O(n)$. The front is the cheap end of a singly- linked list, and *push_front* is its natural growth operation. (We will quantify “ $O(1)$ ” and “ $O(n)$ ” properly in Chapter 16; for now, “ $O(1)$ ” means “the cost does not grow with the size of the list.”)

11.3 The chain frees itself

Here is the payoff that makes `unique_ptr` worth the trouble. Look at the end of the demo’s `main`:

```

int main() {
    std::unique_ptr<Node> head;
    for (const int v : {3, 2, 1}) {
        head = push_front(std::move(head), v);
    }
    print(head.get());
    // ... use the list ...
    return 0;
} // <-- head goes out of scope HERE

```

There is no `delete` anywhere in the program. Not one. And yet when `main` returns and `head` goes out of scope, the *entire list* is reclaimed, in order, automatically. Trace the cascade:

```
~head -> ~n0 -> ~n1 -> ~n2 -> done
```

When `head`'s destructor runs, it destroys the `unique_ptr` it holds — which runs the *first node's* destructor — which destroys *that node's next unique_ptr* — which runs the *second node's* destructor — and so on, all the way to the `nullptr` tail. This is the single-object RAII guarantee from Chapter 10 (and the broader RAII idea from Chapter 5), generalized to a whole linked structure. You wrote zero `deletes`, and leak, dangling pointer, and double-free are *all three* impossible for the entire data structure.

Compare this to what the same thing would cost you with raw `new/delete`: a hand-written destructor that walks the list deleting nodes, plus the discipline to call it on every exit path, plus the bug where you `delete` a node and then still hold a pointer to it. The `unique_ptr` version deletes all of that *code* and all of those *bugs*.

 The destructor cascade is recursive — and a very long list can overflow the stack

The teardown above is one destructor call nested inside another: `~n0` is still running on the call stack when `~n1` begins, which is still running when `~n2` begins. That is one stack frame per node. For a list of a few thousand nodes this is invisible. For a list of a few *million* nodes it can overflow the call stack and crash — the same deep-recursion footgun we will meet again with the binary tree in Chapter 14. This is one concrete reason the standard library's `std::list` frees its nodes *iteratively* rather than recursively, and one more entry in the “use to ship” column below.

11.4 Recursion mirrors the structure

A linked list has a *recursive definition*: a list is either **empty** (`nullptr`) **or** a node followed by a *smaller list*. That definition is worth saying out loud, because the code that traverses a list mirrors

it exactly. Every traversal has the same shape — handle the base case `nullptr`, otherwise do something with this node and **recurse on next**.

This is recursion in the Chapter 4 sense, and the list is the textbook place to use it. Three examples from the demo:

```
int length(const Node* n) {
    if (n == nullptr) {
        return 0; // base case: the empty list has length 0
    }
    return 1 + length(n->next.get()); // this node, plus the rest
}

int sum(const Node* n) {
    if (n == nullptr) {
        return 0;
    }
    return n->value + sum(n->next.get());
}

bool contains(const Node* n, int target) {
    if (n == nullptr) {
        return false;
    }
    return n->value == target or contains(n->next.get(), target);
}
```

Each is the recursive definition turned into code: the `if (n == nullptr)` line is “the list is empty,” and the `return ... n->next.get() ...` line is “this node combined with the answer for the smaller list.” (The `or` in `contains` is the logical-OR operator; in code you import from elsewhere you will see it spelled `||`, and you must read both — see Chapter 2. `&&` is `and` and `!` is `not`.)

Two things about the parameter type are load-bearing.

We traverse with a non-owning `const Node*`, not a `unique_ptr`. The functions take a raw `const Node*` — a *borrow*, in the Chapter 10 vocabulary. They are *reading* the list, not taking ownership of it. The list itself (the chain of `unique_ptr`s) still owns every node; these functions just look. This is the right tool, and using a raw pointer here is not a step backward — it is the modern idiom: **own with a smart pointer, borrow with a raw pointer**.

.get() produces the borrow. `n->next` is a `unique_ptr<Node>`; calling `.get()` on it hands back the raw `Node*` inside, *without* transferring ownership. So `length(n->next.get())` means “recurse on the rest of the list, borrowing it.” The cardinal rule from Chapter 10 still holds: **never delete**

a **borrowed pointer**. These functions never could — they have no `delete`, and the list will free itself when it is done.

11.4.1 The iterative twin

Every recursive list traversal has an *iterative* twin: a loop that walks a borrowed pointer from the head to `nullptr`. The demo's `print` is written this way:

```
void print(const Node* head) {
    std::cout << '[';
    const char* sep = "";
    for (const Node* p = head; p != nullptr; p = p->next.get()) {
        std::cout << sep << p->value;
        sep = " -> ";
    }
    std::cout << " -> nullptr]\n";
}
```

The loop `for (const Node* p = head; p != nullptr; p = p->next.get())` is the canonical shape: start at the head, continue while `p` is not the tail marker, and advance with `p = p->next.get()`. It computes the same thing as a recursive walk, but it uses *one* stack frame instead of one per node — so it does not have the deep-recursion footgun from the callout above. When the recursion is shallow (a few thousand nodes), the recursive form is often clearer and the choice is a matter of taste; when the list could be enormous, prefer the loop. We will lean hard on the recursive form for the *tree* in Chapter 14, where the iterative twin is genuinely awkward, so it is worth being fluent in both shapes now.

💡 Build to learn, use to ship: this is `std::forward_list`

You just hand-built a singly-linked list to *understand* how owning pointers chain and how recursion mirrors a recursive data structure. That was the point. In production code you would almost never write this. The standard library ships `std::forward_list` (singly-linked) and `std::list` (doubly-linked), already correct, already iterative in their teardown, already tested by millions of users.

And even those are rarely what you want. Compare the hand-built list to a `std::vector`:

linked list	<code>std::vector</code>
$O(1)$ front insert, no shifting	$O(n)$ middle insert (shifts elements)
one heap allocation <i>per node</i>	one contiguous buffer
pointer-chasing — <i>cache-hostile</i>	sequential — <i>cache-friendly</i>

The linked list wins on paper for cheap front/middle insertion. But on real hardware, the `vector`'s contiguous memory is so much friendlier to the CPU's cache that `vector` wins in practice far more often than the big-O table suggests — walking a linked list means chasing a pointer to a random heap address at every step, which is the slowest thing a modern CPU does. The linked list is mostly a *teaching* structure and an *interview* structure. Know how it works; reach for `std::vector` unless you can prove otherwise. We make that rule precise at the end of the chapter.

11.5 Associative containers: lookup by key, not position

A `std::vector` answers the question “what is at index 3?” — lookup by *position*. An **associative container** answers a different question: “what is the value for key “Ada”?” — lookup by *key*. If you have written Python, you already know this structure intimately: it is the `dict`.

C++ gives you two flavors:

```
std::map<std::string, int>      freq; // tree: sorted keys,  O(log n)
std::unordered_map<std::string, int> ages; // hash: no order,  O(1) avg
```

Both store `key -> value` pairs and both are Python's `dict`. The difference is the implementation underneath, and it determines their ordering and their speed:

	<code>std::map</code>	<code>std::unordered_map</code>
implementation	balanced tree	hash table
ordering	sorted by key	none (arbitrary)
lookup / insert	$O(\log n)$	$O(1)$ average
reach for it when	you need sorted iteration or ranges	the default — it is faster

`std::unordered_map` is the one that behaves like Python's `dict`: a hash table, no guaranteed order, average constant-time lookup. It is the **default** — when in doubt, use it. `std::map` is the specialist: it keeps its keys in *sorted order* at all times (because a balanced tree is sorted by construction), which costs you a $\log n$ factor per operation but buys you sorted iteration and range queries for free. Reach for `std::map` precisely when that sortedness is what you want.

i `set` and `unordered_set`: when you only care about membership

`map/unordered_map` store `key -> value` pairs. Their siblings `std::set` and `std::unordered_set` store *keys only* — they answer the pure-membership question “is `x` in this collection?” with no value attached. They are the direct analogue of Python's `set`.

Same tree-vs-hash and sorted-vs-unordered tradeoff as their map cousins. You will see them in the decision tree at the end.

11.6 The frequency-count idiom

The single most common thing you do with an associative container is **count**. Given a stream of words, how many times does each appear? In Python you reach for `collections.Counter` or the `d.get(k, 0) + 1` pattern. In C++ the idiom is one line:

```
std::map<std::string, int> freq;
for (const auto& w : words) {
    ++freq[w];
}
```

The expression `++freq[w]` is doing something subtle that is worth slowing down for. When you index a map with `[]` and the key is *not present*, the map **default-constructs** a value for it — and for an `int`, default-construction means **zero**. So the very first time `freq[w]` sees a new word, it creates an entry mapping that word to 0, and the surrounding `++` immediately bumps it to 1. Every subsequent sighting just increments the existing count. One line replaces the manual *check-if-present / insert-if-not / then-increment* dance you would otherwise write — exactly what `collections.Counter` does for you in Python.

Iterating the result is a structured binding (Chapter 3), the same destructuring you know from Python's `for k, v in d.items():`:

```
for (const auto& [word, n] : freq) { // structured binding
    std::cout << std::format(" {:<6} {}\\n", word, n);
}
```

Here is the one place the `map-vs-unordered_map` choice shows up *visibly*. Because `freq` is a `std::map`, iterating it yields the keys in **sorted order** — for free, because the tree is always sorted. Counting `red blue red green blue red` and printing prints `blue 2, green 1, red 3` — alphabetical. Swap the declaration to `std::unordered_map` and the exact same loop prints the words in *some arbitrary order* that depends on the hash function. If you want the output sorted, that is a reason to pay the $\log n$ and use `std::map`. If you do not care about order, `unordered_map` is faster. (This swap is literally one of the in-class activities — change the type, recompile, watch the order change.)

11.7 Four ways to access — and they differ on a *missing* key

That `++freq[w]` trick — `[]` inserting a default when the key is absent — is wonderfully convenient when you are *building* a map. It is a genuine **footgun** when you are merely trying to *read* one. There are four ways to get at a value in an associative container, and the entire reason there are four is that they behave *differently on a key that is not there*.

```
std::unordered_map<std::string, int> ages{
    {"Ada", 36}, {"Alan", 41}, {"Grace", 85}};

ages["Ada"]           // [] : convenient, but INSERTS a default if absent (!)
ages.at("Alan")       // at : CHECKED -- throws std::out_of_range if absent
ages.contains("Bob")   // C++20 membership test -- never inserts, returns bool
if (auto it = ages.find("Grace"); it != ages.end()) { // iterator, or end()
    use(it->first, it->second);
}
```

`[]` **inserts on read — this is the footgun**. If you write `ages["Bob"]` and "Bob" is not in the map, you do not get an error and you do not get a sentinel — you *silently insert* `Bob -> 0` and the map grows by one. A read that has the side effect of a write. This is precisely the Chapter 3 lesson that **the key expression inside `[]` is the #1 bug hotspot**, in its most insidious form: a typo'd or genuinely-absent key does not crash, it *mutates the container*, and you find out three functions later when the map has phantom entries. When you are reading, almost never reach for `[]` first.

`.at()` **is the checked read — throws on a missing key**. `ages.at("Bob")` throws `std::out_of_range` if "Bob" is absent, exactly the same discipline as `vector::at()` versus `vector::operator[]` from Chapter 3. Use `.at()` when the key *must* be present and a missing one is a genuine bug you want to hear about loudly rather than paper over.

`.contains()` **is the clean membership test (C++20)**. `ages.contains("Bob")` returns a `bool`, never inserts, never throws. This is the right tool for “is this key here?” — the C++20 spelling that finally made the question a one-liner. (Before C++20 you wrote `ages.find("Bob") != ages.end()`, which still works.)

`.find()` **looks up and hands you the value in one probe**. `ages.find(key)` returns an *iterator* to the element, or `ages.end()` if the key is absent. The `if (auto it = ages.find(...); it != ages.end())` shape — an `if` with an initializer (C++17), so it is scoped to the branch — is the idiom for “look it up, and if it is there, use it.” Crucially it does this in a **single** lookup: `it->first` is the key, `it->second` is the value, with no second probe. Compare this to `if (ages.contains(k)) use(ages.at(k));`, which hashes `k` *twice*. `.find()` is what you reach for when you both need to know whether the key is present *and* want its value.

💡 A four-way rule of thumb

- **Building / counting** → `[]` (you *want* the insert-on-absent behavior).
- **Key must be present** → `.at()` (loud failure if your assumption is wrong).
- **Just asking “is it here?”** → `.contains()`.
- **“Is it here, and if so give me the value”** → `.find()` (one lookup, not two).

The mistake to avoid is reflexively using `[]` everywhere because it is the shortest. On a read path, `[]` is the one that can silently corrupt your container.

11.8 Table-driven design: when an if/else chain wants to be a map

We now have enough to revisit a theme that started back in Chapter 2 and ran through Chapter 3: a long chain of `if/else` if branches that maps an *input* to a *result* is almost always a **table wearing a costume**. In Chapter 3 we turned a five-branch grade chain into a **vector** of threshold/letter rows. The associative container lets us do the same thing for the more general case — a mapping from arbitrary keys to values.

Fred Brooks put the thesis in one sentence in *The Mythical Man-Month* (1975):

*Show me your flowcharts and conceal your tables, and I shall continue to be mystified.
Show me your tables, and I won't usually need your flowcharts; they'll be obvious.*

Here it is in code. **Before** — the control flow *is* the data, and the mapping is buried in branches:

```
std::string_view status_text_ifelse(int code) {
    if (code == 200) {
        return "OK";
    }
    if (code == 404) {
        return "Not Found";
    }
    if (code == 418) {
        return "I'm a teapot";
    }
    if (code == 500) {
        return "Internal Server Error";
    }
    return "Unknown";
}
```

Every new status code means a new `if`. The fact that this is fundamentally a *lookup table* — code 200 maps to “OK”, 404 maps to “Not Found” — is hidden inside five near-identical branches a reader has to scan to reconstruct.

After — the data *is* the data:

```
std::string_view status_text_table(int code) {
    static const std::unordered_map<int, std::string_view> kStatus{
        {200, "OK"},
        {404, "Not Found"},
        {418, "I'm a teapot"},
        {500, "Internal Server Error"},
    };
    const auto it = kStatus.find(code);
    return it != kStatus.end() ? it->second : "Unknown";
}
```

The table is obvious at a glance, the lookup is one line, and adding a status code is a **one-row edit** instead of a new branch. Three details earn their place:

- **static** means the table is built *once*, the first time the function is called, and reused on every subsequent call — not rebuilt on every invocation.
- **const** says the table never changes after construction, which both documents intent and lets the compiler help.
- **.find() plus the ternary** `it != kStatus.end() ? it->second : "Unknown"` is exactly the missing-key discipline from the previous section: look it up once, return the value if present, fall back to a default if not. Note we did *not* use `kStatus[code]` — that would insert `code -> ""` on every unknown code and quietly grow a table we declared **const** (it would not even compile, which is the point: **const** *forces* you to the safe access method).

This is the same arc you have watched three times now: push the work out of the *code* (handwritten branches) and into the *data* (a table). It is one of the central rhythms of the book.

11.9 The container decision tree

You now know `std::vector` (Chapter 3), the hand-built list and its library cousins `std::forward_list`/`std::list`, `std::map`/`std::unordered_map`, and `std::set`/`std::unordered_set`. The engineering skill is not knowing *that* they exist — it is knowing *which one* to reach for. Here is the decision tree the course asks you to be able to recite:

what you have	reach for
a sequence, mostly <code>push_back</code> , index access	<code>std::vector</code> (default)

what you have	reach for
a sequence, fixed size known at compile time	<code>std::array</code>
key -> value, need sorted keys / ranges	<code>std::map</code>
key -> value, order doesn't matter	<code>std::unordered_map</code> (faster)
just membership (is <i>x</i> in the set?)	<code>std::set</code> / <code>std::unordered_set</code>
frequent middle-insert while holding iterators	<code>std::list</code> (<i>rare</i>)

The one line to memorize sits at the top: **`std::vector` unless proven otherwise**. The `vector`'s contiguous memory makes it the fastest container in practice for an enormous fraction of real problems, and you should treat any other choice as something you can *justify* — “I need key lookup,” “I need sorted iteration,” “I genuinely insert in the middle a million times and hold iterators across it.” Absent such a reason, reach for `vector`.

The runnable form of this table is `datastructures.cpp` — an interactive *recursive yes/no walker* that asks you questions (“Do you look things up by key?”, “Do you need them sorted?”) and emits the C++ container type it has decided on. It is, in xkcd #518's terms, a flow chart you can *run*. It is also a nice demonstration of **mutual recursion**: `get_data_structure` calls `get_atomic_data` which can call back into `get_data_structure` — the “*f* calls *g* calls *f*” pattern, a generalization of the single-function recursion you wrote for `length` and `contains` above.

```
# answer the yes/no prompts and it designs a container for you:
$ g++ -std=c++20 -Wall -Wextra -O0 -g datastructures.cpp -o ds && ./ds
```

11.10 AI touchpoint — review the table-driven refactor

Take the `status_text_ifelse` / `status_text_table` pair above (or write your own 4-plus-branch `if/else` chain — a day-of-week namer, an HTTP verb dispatcher, a menu) and hand *both* versions to an AI assistant with the prompt: “Here are two implementations of the same function. Compare them on readability, maintainability, and performance. Which would you ship, and is there a case where the *if/else* version is actually the better choice?”

Read the answer critically — and notice that the honest answer is *not* “the table is always better.” For a *two*-case mapping, the `if/else` is shorter and clearer; the table earns its keep as the number of rows grows and as the mapping becomes something a non-programmer might want to edit. A good assistant will name that tradeoff; a glib one will just cheer for the table. This is the **AI as code reviewer** role from the AI Tools chapter — and learning to tell a thoughtful review from a reflexive one is most of the skill.

💡 Prove the list leaks nothing

The whole claim of Section 11.3 is that the `unique_ptr` chain frees itself with no `delete` and no leak. Don't take it on faith — prove it with the AddressSanitizer (Appendix B):

```
$ g++ -std=c++20 -O0 -g -fsanitize=address l12_list.cpp -o l12_list_asan
$ ./l12_list_asan
```

A clean run with no leak report is the evidence. As an experiment, hand-rewrite `Node::next` as a raw `Node*` with `new/delete`, “forget” one `delete`, and watch ASan catch the leak the `unique_ptr` version made impossible to write.

11.11 What you should understand by the end of this chapter

- Declaring a **self-referential struct** `Node { T value; std::unique_ptr<Node> next; };` and explaining the **ownership chain** — head owns `n0` owns `n1` ..., and the tail's `next` is `nullptr`
- Why `push_front` uses `std::move` (because `unique_ptr` is move-only) and why front insertion is $O(1)$
- That the whole chain **frees itself** when the head is destroyed (the destructor cascade, RAII generalized) — with no `delete` — and that the cascade is recursive and can overflow the stack on a very long list
- Writing a **recursive** list traversal with base case `n == nullptr`, *borrowing* nodes through a non-owning `const Node*` via `.get()`, and the iterative twin
- The difference between `std::map` (balanced tree, sorted, $O(\log n)$) and `std::unordered_map` (hash, $O(1)$ average), and that both *are* Python's `dict`
- The frequency-count idiom `++freq[w]` (the analogue of `collections.Counter`), and that iterating a `std::map` is sorted for free
- The four access methods and how they treat a missing key: `[]` (inserts on read — the footgun), `.at()` (checked, throws), `.contains()` (C++20), `.find()` (one probe, returns an iterator)
- **Table-driven design** — recognizing that a long `if/else` chain mapping input to output is a `map` in disguise, and the `static const` lookup rewrite
- The **container decision tree** and the rule “`std::vector` unless proven otherwise”
- **Build to learn, use to ship**: the hand-built list is a learning exercise; `std::forward_list`/`std::list` exist, and in production you almost always reach for `std::vector` anyway

The list-as-recursion and the borrow-with-a-raw-pointer habits both carry directly into Chapter 14, where the nodes gain a *second* `unique_ptr` child and the recursion gains a second branch. That is the climax of the week-4 data-structures arc.

12 File I/O and the three I/O dialects

Up to this point, every program in this book has lived entirely **in memory**. You read a number, built a `std::vector`, walked a linked list (Chapter 10), looked something up in a `std::map` (Chapter 11) — and when the process exited, all of it vanished. This chapter answers the two questions that the in-memory arc let you dodge:

1. How do I get data **in from** and **out to** a *file*, so it survives the program exiting?
2. How do I turn values into **text a human can read** — aligned columns, two decimal places, hexadecimal — *on purpose*, not by accident?

Both questions are answered by one idea: the **stream**. A stream is a single, uniform interface — the `<<` and `>>` operators you have been using since Chapter 2 — that works the same way whether the bytes are going to your terminal, to a file on disk, or to an in-memory string. Learn it once; it works everywhere.

This is also the chapter that finally **explains** `std::format`. The demos have been using it since the early labs without ever stopping to teach it; here is where it gets taught, alongside its two older cousins, `printf` and `std::cout <<`. By the end you will be able to produce the same formatted line three ways, and — more importantly — say what each way *costs* and *guarantees*.

12.1 A stream is a typed conduit

A **stream** is a sequence of bytes you read from or write to with the *same two operators* no matter what is on the other end:

- `<<` — the **insertion** operator: write a value *into* a stream.
- `>>` — the **extraction** operator: read a value *out of* a stream.

You already met both in Chapter 2, pointed at the terminal: `std::cout << x` writes, `std::cin >> x` reads. The insight of this chapter is that the terminal is not special. The *same* `<<` and `>>` work when the other end of the stream is a file or a string. The formatting code does not know, and does not care, where the bytes ultimately go.

The standard library gives you a small family of streams:

Stream	Type	What's on the other end
<code>std::cout</code>	<code>std::ostream</code>	the terminal (normal output)
<code>std::cerr</code>	<code>std::ostream</code>	the terminal (errors, unbuffered)
<code>std::cin</code>	<code>std::istream</code>	the keyboard (input)
<code>std::ofstream</code>	output file stream	a file on disk (writing)
<code>std::ifstream</code>	input file stream	a file on disk (reading)
<code>std::ostringstream</code>	output string stream	an in-memory <code>std::string</code>

The two halves of the family share a base type. Every output stream *is a* `std::ostream`; every input stream *is a* `std::istream`. That single fact — a file stream is just an `ostream` pointed at disk — is what lets you write one piece of formatting code and send its output anywhere. We will lean on it hard when we overload `operator<<` later in the chapter.

i “Everything is a file”

This unified view is C++’s take on an old Unix idea: *everything is a file*. On a Unix system the terminal, a real file, a network socket, and a pipe all present the same **read/write** interface, so a program written to read “a file” can just as easily read from a pipe. C++ streams are the same move one level up: `std::cin`, an `ifstream`, and an `ostringstream` all present the same `<</>>` interface, so code written against “a stream” does not care which one it got.

12.2 The same line, three ways

C++ has accumulated **three dialects** for turning values into formatted text. They were each the right answer in their era, and you will see all three in real code, so you need to read all three. The honest comparison is the point of this section — so here is one line of output produced three different ways. All three print exactly:

```
Ada      scored  92.50% over   3 runs
```

12.2.1 Dialect 1: `printf` — the C heritage

```
#include <cstdio>
// name is "Ada", pct is 92.5, runs is 3
std::printf("%-8s scored %6.2f%% over %3d runs\n", name.c_str(), pct, runs);
```

`printf` is **variadic**: it takes a format string with *% conversion codes* followed by “some arguments”. `%-8s` means “a string, left-justified in a field 8 wide”; `%6.2f` means “a floating-point number, 6 wide, 2 decimals”; `%3d` means “an integer, 3 wide”; `%%` is a literal percent sign. It is terse and, to a trained eye, fast to write. It came from Dennis Ritchie’s C in 1972 and almost every language you will meet — Python’s old %-formatting, C, Go’s `fmt`, Rust’s macros — inherited its vocabulary.

Notice `name.c_str()`. `printf` predates `std::string` by thirteen years; it cannot print one directly, so you hand it the underlying C string. That small friction is a preview of the deeper problem we name in a moment.

12.2.2 Dialect 2: `std::cout` << — idiomatic, type-safe, verbose

```
#include <iostream>
#include <iomanip>
std::cout << std::left << std::setw(8) << name << " scored " << std::fixed
           << std::setprecision(2) << std::setw(6) << pct << "% over "
           << std::setw(3) << runs << " runs\n";
```

This is the dialect you have been using in its simple form since Chapter 2. It is **type-safe**: each << knows the type of its right-hand operand, so there is no way to ask it to print a `double` as if it were an `int`. The cost is **verbosity**. Field width, alignment, and precision are controlled by *manipulators* from `<iomanip>` — `std::setw`, `std::left`, `std::fixed`, `std::setprecision` — and most of them are **sticky**: once you set `std::fixed`, it stays in effect for every subsequent number on that stream until you change it back. A formatted line becomes a wall of manipulators with the actual data buried inside, and a sticky setting you forgot to reset bites a line ten statements later.

12.2.3 Dialect 3: `std::format` — the C++20 default

```
#include <format>
std::cout << std::format("{:<8} scored {:.6.2f}% over {:>3} runs\n",
                        name, pct, runs);
```

If you know Python f-strings, you already know this. `std::format` takes a format string with `{}` placeholders and a list of arguments; each `{:...}` says how to format the next argument. `{:<8}` is “left-justified, width 8”; `{:.6.2f}` is “width 6, 2 decimal places, fixed”; `{:>3}` is “right-justified, width 3”. It is **type-safe** like `std::cout`, **reads like one intentional string** like `printf`, and — the part neither of the others can claim — the format string is **checked at compile time**.

`std::format` *returns* a `std::string`; it does not print anything itself. You then send that string wherever you like — to `std::cout`, to a file, to two places at once. That separation (build the text, then route it) is exactly what makes the report pipeline at the end of this chapter clean.

12.3 What each dialect costs you

Same output, three very different sets of promises:

	<code>printf</code>	<code>std::cout <<</code>	<code>std::format</code>
era	C, 1972	C++, 1985	C++20
type-safe?	no (variadic)	yes	yes
checked when?	never	compile (per <<)	compile time
for-mat-ting	% codes	sticky manipulators	{ } mini-language
reads like	terse, cryptic	verbose, noisy	one clear string
%d on a double	undefined behavior	(can't express it)	won't compile
ex-tend to your own type?	never	<code>operator<<</code> (one function)	<code>std::formatter</code> (a struct)

 The recommendation, as a tradeoff

Write `std::format`. Read all three. It is the only dialect that is type-safe *and* reads like a single intentional string *and* catches a bad format string before the program runs — `printf`'s readability with `std::cout`'s safety, and then some. But “write `std::format`” is a default, not a law. You will *read* `printf` in every C codebase, the Linux kernel, and a thousand Stack Overflow answers; you will read `std::cout <<` in thirty-five years of accumulated C++. The dialect to *reach for first* is `std::format`; the dialects you must stay fluent in are all three.

 `std::format` needs a recent standard library

`std::format` arrived in C++20 but real compiler support lagged the standard by a couple of years. You need a recent `libstdc++` (GCC 13+) or `libc++` (LLVM 17+). If `#include <format>` fails to compile on an older toolchain, the standard fallback is the [fmt library](#) — `std::format` is modeled directly on it, the syntax is identical, and you write `fmt::format` instead. The course's reference toolchain (the Lab 3 Linux VM) has

`std::format`; if you are on an older machine, install `fmt` and the examples transfer verbatim. C++23 adds `std::print("{}\n", x)` — a one-call shorthand that fuses the `std::cout <<` and the `std::format` — but it needs `-std=c++23`, so we mention it and move on.

12.4 The footgun, named: `printf` cannot check you

The single most important line in the comparison table is the `%d-on-a-double` row, because it is where one dialect crosses from “verbose” into “dangerous”.

```
std::printf("%d\n", 3.14);    // asks: read a DOUBLE as if it were an int.
                             // UNDEFINED BEHAVIOR -- no error, garbage or crash.

std::format("{}\n", 3.14);    // prints 3.14. There is no way to mismatch.
```

`printf` is variadic — it accepts “some arguments” and *trusts the format string to describe them*. The compiler does not, in general, know what the format string says, so it cannot verify the promise. When you write `%d` (expect an `int`) but pass a `double`, the language does not stop you. This is **undefined behavior**, the most feared phrase in the C++ vocabulary, which we first met at the boundary of a C-style array in Chapter 3: the program does not error out, it does not stop, it produces *some* result — which may differ across machines, compilers, and runs.

`g++ -Wall` will warn here, via the `-Wformat` check, because GCC happens to understand the literal `printf` format strings in the standard library. This is one more reason `-Wall -Wextra` is non-negotiable. But the *language* permits the mismatch; only the warning catches it, and only when the format string is a literal the compiler can see through. `std::format` closes the hole at the language level: the format string must be a compile-time constant, so a type mismatch is a *compile error*, not a runtime surprise.

⚠ Security: the format-string vulnerability and Bobby Tables

The footgun gets worse when the format string comes from *untrusted input*. Suppose you write:

```
std::printf(user_input);      // NEVER do this
```

If a user types `%n`, you have just handed them a primitive that *writes to memory*: the `%n` conversion code instructs `printf` to write the number of characters printed so far into an address taken from the argument list — an argument list the attacker controls the shape of. This is the classic **format-string vulnerability**, and for years it was a reliable way to turn a careless log statement into arbitrary code execution. The fix is one character:

```
std::printf("%s", user_input);    // the input is data, not a format string
```

The cultural touchstone here is xkcd #327, *Bobby Tables* — “Little Bobby Tables, we call him” — where a student named Robert'); DROP TABLE Students;-- exploits a school that splices unescaped input straight into a database query. That is SQL injection, not a format-string bug, but it is the *same root mistake*: **untrusted input was treated as code instead of as data**. Format-string injection is the `printf` flavor of exactly that error. `std::format` cannot be abused this way at all — its format string *must* be a compile-time constant, so user input can only ever land in the argument list, where it stays data. This is the first appearance of a security through-line that runs to Chapter 17; the lesson is the same every time: *know which of your strings is code and which is data, and never let an attacker move the boundary*.

12.5 The format mini-language

Here is the part you already know, if you have written a Python f-string. The spec inside the braces is, deliberately, the *same grammar*:

```
{[index]:[fill][align][sign][#][width][.precision][type]}
\____/  \_____ the format spec _____/
which          everything after the ':' is optional
argument
```

The Python equivalence is exact for the common cases:

```
# Python
f"{pct:6.2f}"
```

```
// C++
std::format("{:6.2f}", pct)
```

A runnable cheat-sheet of the pieces you will use ninety percent of the time:

```
std::format("{:<10}", "Ada")    // left,   width 10: "Ada      "
std::format("{:>10}", "Ada")    // right,  width 10: "      Ada"
std::format("{:^10}", "Ada")    // center, width 10: "   Ada   "
std::format("{:.*>10}", "Ada")  // fill with '.':  ".....Ada"
std::format("{:8.3f}", pi)      // width 8, 3 decimals: "   3.142"
std::format("{:#x}", 255)       // hex with 0x prefix:  "0xff"
std::format("{:#b}", 5)        // binary with 0b:     "0b101"
```

```
std::format("{0} and {0}", x)    // reuse argument 0 by index
std::format("{literal braces}") // {{ and }} escape to literal { and }
```

Learn four things — **align**, **width**, **precision**, **base** — and look the rest up when you need it. The grammar is large, but you carry most of it across from Python already.

i One small divergence from Python

Python writes thousands separators with `f"{n:,}"`. C++ does *not* spell it that way — `std::format` uses a locale-aware `{:L}` specifier instead (and even that depends on a locale being installed). It is the one place the muscle memory from Python misfires, so it is worth knowing the C++ spelling exists before you reach for the comma that does not work.

12.6 Files are just streams pointed at disk

Now the first of the chapter's two questions. To read and write files you `#include <fstream>` and open one of two file streams:

```
#include <fstream>
#include <format>

std::ofstream out("scores.txt");           // OUTPUT stream -> writes
out << std::format("{} {}\\n", name, score); // exactly like std::cout

std::ifstream in("scores.txt");            // INPUT stream -> reads
std::string n;
int s{};
in >> n >> s;                             // exactly like std::cin
```

There is, genuinely, *nothing new to learn about the formatting*. An `ofstream` is an `std::ostream`, so the same `<<` you have used on `std::cout` works on it unchanged, and `std::format` composes straight into it. The only new vocabulary is how the file gets opened and closed.

Truncate vs. append. By default an `ofstream` **truncates** the file — opening `scores.txt` for output throws away whatever was in it and starts fresh. If you want to *add to* the end of an existing file instead, open it in append mode:

```
std::ofstream log("run.log", std::ios::app); // append; don't clobber
```

This is the difference between Python's `open(path, "w")` and `open(path, "a")`, and the default — truncate — is the same default Python picks. It is also a real bug class: opening a file you meant to append to in the default mode silently erases it.

12.6.1 Two things that bite Python programmers

The formatting transfers for free, but two C++ wrinkles surprise people coming from Python.

```
std::ofstream out("scores.txt");
if (not out) {                                     // (1) opening CAN FAIL -- check it
    std::cerr << "could not open scores.txt\n";
    return 1;
}
out << "...";
// (2) NO out.close() here -- RAII closes it when `out` leaves scope
```

(1) Opening can fail silently. In Python, `open("/nope/scores.txt", "w")` *raises* — the failure is loud and unmissable. In C++, opening a file that cannot be opened (a missing directory, a read-only location, a permissions problem) does **not** throw. The stream quietly sets an internal *fail bit* and every subsequent `<<` or `>>` becomes a silent no-op. If you skip the check, you write to a dead stream and lose every byte, with no error anywhere. So the check is **mandatory**:

```
if (not in) { /* handle the failure */ }
```

This `if (not in)` idiom deserves a close look, because it is doing two slightly magical things at once. `not` is the keyword spelling of the `!` operator we named in Chapter 2 (the symbolic form is `if (!in)`). And `in` is a *stream*, not a `bool` — so why can you negate it? Because a stream is convertible to `bool`: it tests as `true` while it is healthy and `false` once a fail bit is set. So `if (not in)` reads, exactly, as “*if the stream is not in a good state.*” You will see `in.is_open()` used for the same check; `if (not in)` is the more general idiom because it also catches a stream that *opened* fine but *failed* mid-read, which is precisely what we exploit in the read loop below.

(2) You do not call `close()`. A file stream is an **RAII** object, the same design you met when a `class` cleaned up after itself in Section 5.6 and again when `std::unique_ptr` freed heap memory in Chapter 10. RAII — *Resource Acquisition Is Initialization* — ties a resource’s lifetime to a variable’s lifetime: the file is opened in the stream’s constructor and **flushed and closed in its destructor**, automatically, when the stream variable goes out of scope. You *can* call `out.close()` explicitly if you need to close early, but in the common case you simply let the stream fall out of scope and trust the destructor. It is the *same guarantee* `unique_ptr` gives for memory, now for a file handle: a resource tied to a lifetime, released without you having to remember.

12.7 Reading: `getline` vs. `>>`, and the loop condition

Once a file is open for reading, you choose between two record shapes.


```
// (a) getline: a record is a WHOLE LINE (spaces included).
std::string line;
while (std::getline(in, line)) {
    process(line);
}

// (b) >> : field by field; skips whitespace, CONVERTS as it reads.
std::string name;
int score{};
while (in >> name >> score) {
    use(name, score);
}
```

`std::getline(in, line)` reads up to (and discarding) the next newline, giving you one whole line — spaces and all — as a `std::string`. This is the direct analogue of Python’s `for line in file:`. Reach for it when a record is a line: a CSV row, a log entry, a line of free text.

`in >> name >> score` reads *field by field*, skipping whitespace between fields and **converting as it goes** — `>> score` parses the next token as an `int` and fails if it is not one. Reach for it when a record is a fixed sequence of typed fields.

The load-bearing rule for both: **put the read in the loop condition.**

Both `std::getline` and `>>` *return the stream*, and a stream tests as `true` while it is healthy and `false` the moment a read fails — at end-of-file, or when `>> int` hits a non-number. So `while (in >> name >> score)` reads, exactly, as “*keep going as long as a record reads cleanly.*” This is the same trick as `if (not in)` above, used as a loop condition: the read and the success test are one expression.

⚠ Never `while (not in.eof())`

The tempting-but-wrong shape is to test for end-of-file *first* and read *inside*:

```
while (not in.eof()) {           // WRONG: processes one phantom record past the end
    in >> name >> score;
    use(name, score);           // runs once more after the last real record
}
```

The bug is subtle and classic. `eof()` does not become true until you *attempt* a read that runs off the end — so after the last real record, `eof()` is still false, the loop runs one more time, the `>>` fails (leaving `name` and `score` holding their *previous* values), and `use()` is called on a **phantom duplicate record**. The fix is the idiom above: read *in the condition*, so the same expression that reads also decides whether to continue. `while (in >> name >> score)`

cannot produce a phantom record, because the failed read *is* the loop exit. This footgun is common enough that it is worth recognizing on sight in other people's code.

12.8 Make your own class streamable: overload operator<<

So far we have streamed `int`, `double`, and `std::string`. What about a type *you* defined — a `Point`, say, from the class machinery of Chapter 5? You can teach C++ to stream it, because `<<` is *just an operator*, and operators can be **overloaded** for your own types.

```
#include <ostream>
#include <format>

// A FREE function -- not a member -- because the left operand is the STREAM,
// not the Point. Return the stream so calls chain.
std::ostream& operator<<(std::ostream& os, const Point& p) {
    return os << std::format("{}, {}", p.x(), p.y());
}
```

With that one function in scope, a `Point` flows through *any* output stream:

```
std::cout << home << " -> " << bu << "\n";           // (0, 0) -> (2, 3)
```

Three details carry the design.

It takes `std::ostream&`, the base type — not `std::cout`. Because the parameter is the *base* output stream, the *same* function works on `std::cout`, on `std::cerr`, on an `ofstream` writing to a file, and on an `ostringstream` building a string. One function, every endpoint. This is the payoff of the shared base type we noted at the very start of the chapter.

It returns the stream (`return os;`), so calls `chain`. `std::cout << home << " -> " << bu` works because `std::cout << home` *evaluates to* `std::cout` again, ready for the next `<<`. Each `<<` consumes the stream and hands it back. If you forgot the `return`, the chain would break after the first `Point`.

It is a free function, and it needs no friend. The left operand of `<<` is the stream, not the `Point`, so this *cannot* be a member of `Point` (a member's left operand is always the object). It is a free function living next to the class. And because it only uses the **public** accessors `p.x()` and `p.y()`, it needs no special access — **no friend declaration**. Reach for `friend` only when an operator genuinely needs a class's *private* members; if public accessors suffice, a plain free function is cleaner.

The input twin, `operator>>`, completes the round trip:

```

#include <istream>

std::istream& operator>>(std::istream& is, Point& p) {
    double x{};
    double y{};
    if (is >> x >> y) {           // only write p if BOTH reads succeed
        p.set(x, y);
    }
    return is;                     // so `while (in >> p)` can drive a whole file
}

```

Note the second parameter is a non-`const` reference (`Point& p`) — we are *writing into* the point — and we only modify `p` if both extractions succeed, so a partial read does not leave the point half-updated. Returning `is` means `while (in >> p)` reads a whole file of points with the exact loop idiom from the previous section.

12.9 Why streams earn their keep: user-extensibility

That one-line `operator<<` is the heart of the argument for *keeping* streams around even though `std::format` is the day-to-day default. The honest comparison, across all three dialects, is about **extending the formatting to a type the library never heard of**:

Dialect	How you teach it to print <i>your</i> type
<code>printf</code>	You can't. There is no <code>%</code> -code for a <code>Point</code> , and no way to add one.
<code>operator<<</code>	One short free function (above). Your type now streams everywhere.
<code>std::format</code>	Possible, but heavier: a <code>std::formatter<Point></code> specialization — a whole struct with <code>parse</code> and <code>format</code> member functions.

`printf` is a dead end here: its conversion codes are a fixed, closed vocabulary. `std::format` *can* be extended, but it costs you a `std::formatter` specialization — a small struct you write to teach the library how to parse a `Point`'s spec and emit it. That is the right investment when you want a type to participate in the full mini-language (`{:>10}` on a `Point`). But for the common case — “I just want my class to be printable” — `operator<<` is one function, and it has been the idiomatic answer since 1985.

The division of labor

Use `std::format` for scalars and strings — the everyday default, type-safe and compile-checked. Use `operator<<` to make your own class streamable — the minimal, idiomatic, no-friend-needed way to give a type a printable form. When a type needs the *full* format

mini-language, *then* you write the heavier `std::formatter`. The user-extensibility of `<</>>` is exactly why the stream design has survived four decades and is not going anywhere.

12.10 Putting it together: read → compute → write a report

Almost every real program has the same shape: **read** structured input, **compute** something, **write** a readable report. Every piece below is from this chapter; this just assembles them.

```
#include <fstream>
#include <format>
#include <vector>
#include <iostream>

struct Day {
    std::string city;
    int high{};
    int low{};
};

int main() {
    std::ifstream in("weather.txt");
    if (not in) {
        std::cerr << "could not open weather.txt\n";
        return 1;
    }

    // READ: field-by-field into a vector of records, read in the loop condition.
    std::vector<Day> days;
    Day d;
    while (in >> d.city >> d.high >> d.low) {
        days.push_back(d);
    }

    // WRITE: build each line ONCE, send it to TWO streams (terminal + file).
    std::ofstream report("report.txt");
    if (not report) {
        std::cerr << "could not open report.txt\n";
        return 1;
    }
}
```

```

std::string header =
    std::format("{:<12} {:>4} {:>4} {:>6}\n", "CITY", "HIGH", "LOW", "RANGE");
std::cout << header;
report << header;

for (const auto& day : days) {
    std::string row = std::format("{:<12} {:>4} {:>4} {:>5}F\n", day.city,
                                   day.high, day.low, day.high - day.low);

    std::cout << row;
    report << row;
}

return 0;
}

```

Read the structure, not the details. The `while (in >> ...)` is the loop idiom from earlier — read in the condition, stop when a record fails. The `std::format` calls build aligned columns with the mini-language. And the punchline is the pair of lines after each `std::format`:

```

std::cout << header;
report << header;

```

We build the formatted line **once**, as a `std::string`, and send the *same string* to *two* streams — the terminal so you can see it, and the file so it persists. That is the whole payoff of streams and `std::format` working together: the formatting code does not know or care how many endpoints there are, or what they are. A third destination — an `ostreamstream` to capture the report for a unit test (Appendix E) — would be one more line with no change to the formatting at all.

💡 AI touchpoint — port a `printf` line to `std::format`

Find a `printf` line in your current homework’s console output — or one in any C codebase you are reading — and paste it to an AI assistant with the prompt: “Port this *printf* call to `std::format`, and explain what each part of the format spec means.” Then push further: “Now write the line that sends the result to both `std::cout` and a file called `out.txt`.” Read the answer critically against this chapter — does it check `if (not out)`? does it rely on `RAII` to close, or does it call `close()` unnecessarily? does the format spec actually match the % codes it replaced? This is the **AI as translator-of- idioms** role from the AI Tools chapter: you already understand the output you want; you are using the assistant to bridge a dialect you read into the one you write.

12.11 What you should understand by the end of this chapter

- A **stream** as one <</>> interface over many endpoints — terminal, file, and in-memory string — and the names `std::cout` / `std::cin` / `std::cerr`, `std::ofstream` / `std::ifstream`, `std::ostringstream`
- The three I/O dialects compared by era, cost, and guarantee: `printf` (terse, **unchecked**), `std::cout` << (type-safe, verbose), `std::format` (type-safe, **compile-checked** — the modern default)
- The type-safety footgun: `printf("%d", 3.14)` is undefined behavior the language allows and `-Wformat` only *warns* about; `std::format` rejects it at compile time
- The **format-string vulnerability** (`printf(user), %n`) and Bobby Tables as the cultural touchstone: never let untrusted input *be* code
- The `std::format` mini-language `{:[fill][align][width].[prec][type]}` and its equivalence to Python f-strings
- Reading and writing files with `ofstream/ifstream`: truncate vs. **app**, that opening can **fail silently** (`if (not in)`), and that **RAII closes the file** with no explicit `close()`
- Reading with `getline` (line records) vs. `>>` (field records), **in the loop condition** — and why `while (not in.eof())` produces a phantom record
- Overloading `operator<<` / `operator>>` as free functions taking `std::ostream&` / `std::istream&`, returning the stream to chain, needing no `friend` when accessors are public
- The user-extensibility argument: `printf` can't extend, `operator<<` is one function, `std::format` needs a `std::formatter` specialization
- The **read** → **compute** → **write-a-report** pipeline, and sending one `std::format`-ed line to two streams at once

If you can read the weather-report program end to end — and explain *why* the read is in the loop condition, *why* opening is checked, and *why* nobody calls `close()` — you are ready for Chapter 13, where we ask how to cut a program like this one into files.

13 Problem decomposition, namespaces, and modular design

i Chapter status: stub

Mirrors L14 of the Summer 2026 lecture plan — the SE-judgment beat of the term.

13.1 Outline

- **Problem decomposition** (opening ~20 min):
 - Top-down
 - Bottom-up
 - Data-first
 - Worked side-by-side on the same small problem
- **Modular design** — managing complexity *without* deep inheritance: composition over inheritance
- **Namespaces as a software-organization tool** — both sides:
 - *Theoretical* — namespaces as the language’s mechanism for **carving names into scopes you control**; the same problem Python’s `import math` / module objects, Java packages, and Rust modules each solve differently. Every serious language needs an answer to “two libraries that both define `sort`”.
 - *Practical* — **hard course rule (Summer 2026): using namespace std; does not appear in course-distributed code, and is flagged as a deprecated style anti-pattern in student submissions.** The reasons, named so the rule is not arbitrary:
 - * **Header pollution** — any TU that includes a header inheriting it loses control of its own name graph.
 - * **ADL collisions.**
 - * The `std::byte` / `std::data` / `std::distance` **name-clash class**.
 - * It papers over *where names come from* exactly when students are learning to read C++.
 - The Google C++ Style Guide (this course’s baseline) forbids using `namespace` at namespace scope; `clang-tidy`’s `google-build-using-namespace` check — enabled

- **in the course `.clang-tidy` from Lab 0 — flags it automatically.** Not a separate manual-grading dimension.
- **Selective using `std::cout`; is the disciplined alternative** when `std::` qualification gets noisy — function-scope by default, file-scope sparingly; what Stroustrup writes in *PPP*.
- Defining your own namespace for a multi-file project closes the loop.
- **Header / public-API discipline** — the design anchor of the L2→Lab 2 P0→ L4→L14 arc:
 - What belongs in a header vs a `.cpp`
 - What makes a public API stable
 - *Library vs header dump* — when a `.h` is actually reusable code and when it is just internal scaffolding someone hoisted up
- **Security thread callback:** every `#include` injects code into your TU; every linked library executes in your process. Designing your own headers so others can include them is the dual problem.
- C++20 modules teaser
- Inheritance + `override` / `final` + a brief operator-overload survey (compact)
- **Comparative OOP (~15 min, trimmed from 30):** the same `class Foo` in Java, C#, Objective-C side-by-side with C++ — which features are language-design choices vs. universal OOP ideas

14 A tiny BST: data structures as a design exercise

i Chapter status: stub

Mirrors L15 of the Summer 2026 lecture plan — the data-structures climax and the bridge to Algorithms.

14.1 Outline

- **Tiny BST** as the data-structures climax framed as a **design exercise**:
 - **Pick the operations first**: what should `insert` / `lookup` / `contains` / `size` look like to the caller?
 - *Then* the representation
- Implement `insert` / `lookup` recursively
- Instantiate on `int` and on `std::string`
- **Build-to-learn, use-to-ship callout**: `std::set` / `std::map` is what you'd reach for in production — we wrote this to *understand* what it's doing underneath
- Ties classes (Section 5.6) + modern pointers (Chapter 10) + recursion (Chapter 4) + API design (Chapter 13)
- **Bridge to Algorithms** (the downstream course)

Part V

Week 5 — Generics, complexity, tooling deep-dive

Week five steps back from writing more code and asks better questions about the code you already have: how do you express algorithms that work for *any* reasonable type, how do you reason about *how expensive* a program is, and how do you know *that* it is correct, and *that* it is well-formed? Templates and concepts give you the first; algorithmic complexity, `std::optional`, and `std::string_view` give you the second; documentation discipline, linters, formatters, sanitizers, unit tests, the debugger, and a working profiler give you the rest.

The chapters of this part:

- Chapter 15 — function templates, concepts (`std::integral`-style constraints), templating as a complexity-management tool
- Chapter 16 — big-O intuition, `std::optional`, `std::string_view`, when to optimize (and the “step 0.5” library check)
- Chapter 17 — documentation taxonomy, `clang-tidy`, `clang-format`, ASan/UBSan, unit testing with Catch2 or doctest
- Chapter 18 — debugging strategy decision tree, GDB beyond breakpoints, timing + simple profiling, optimization as a *practice*

15 Templates and concepts

i Chapter status: stub

Mirrors L16 of the Summer 2026 lecture plan.

15.1 Outline

- Templates
- Function template basics
- **Concepts** (`std::integral`-style constraints) for readable errors
- **Templating as a complexity-management tool**
- Re-templating the L15 BST (Chapter 14) to be generic on `T` is **HW** (no longer in-lecture)
- `if constexpr` cameo dropped (out of scope for lighter pacing)

16 Algorithmic complexity and when to optimize

i Chapter status: stub

Mirrors L17 of the Summer 2026 lecture plan.

16.1 Outline

- Algorithmic complexity
- Big-O intuition
- `std::optional` / `std::string_view` as efficiency-aware types
- **When to optimize:**
 - Step 0: measure
 - **Step 0.5: check whether the library already does it**
 - Then optimize the actual hot path
- **HW (off-lecture): implement mergesort + binary search**, then compare against `std::sort` / `std::lower_bound` — the “build to learn, use to ship” rule made concrete

17 Skill block: docs, linters, formatters, sanitizers, unit testing

i Chapter status: stub

Mirrors L18 of the Summer 2026 lecture plan. Full how-to is in the Lab Tooling appendix; this chapter is the *concept* block. See Appendix B and Appendix E for the runnable reference.

17.1 Outline

- **Documentation taxonomy** (what does and does *not* belong in each level):
 - Line comments — *why*
 - Function doc comments — *what / contract*
 - Module-level READMEs — *how the pieces fit*
 - Project README — *what this is, how to build, how to run*
- **clang-tidy** — static checking against project policy
- **clang-format** — mechanical formatting
- **ASan / UBSan** — runtime checking for whole classes of UB
- **Catch2 or doctest** — unit testing in C++
- **Static vs dynamic checking** — what each catches, what each misses
- **What CI does for you**
- **AI touchpoint:** test-case generation as a pair-programming exercise

18 Skill block: debugger, profiler, optimization-as-practice

i Chapter status: stub

Mirrors L19 of the Summer 2026 lecture plan. Full GDB reference is in Appendix C.

18.1 Outline

- **Debugging strategy — the decision tree:** when to add a `std::cout` / `std::print`, when to add a real log line, when to set a breakpoint, when to attach a sanitizer; what each is good and bad at
- **GDB beyond breakpoints:**
 - Watchpoints
 - Conditional breakpoints
 - Memory exam
 - `bt full`
- Timing + simple profiling
- **Optimization discipline:**
 - When to optimize (after measurement, not before)
 - Reading a profile
 - Hot-path identification
 - “Premature optimization” as a real failure mode
- Closes the “optimization” core competency from the course description.

Part VI

Week 6 — AI tools, project ramp, capstone

The closing week pulls back to look at *the activity of software engineering* rather than the language. A buffer day absorbs the slack the term has accrued (or covers function pointers and `std::function` if it has not). Then a retrospective on the eight AI roles you have been practicing since L1, with the course's security through-line as the landing beat. Then code review as a practice — what to look for, how to write actionable comments, and where AI as a reviewer helps and where it fails. Then demos and wrap-up.

The chapters of this part:

- Chapter 19 — buffer day; function pointers, `std::function`, callbacks
- Chapter 20 — eight AI roles retrospective, AI policy and disclosure recap, the security through-line lands
- Chapter 21 — code review as practice; AI as code reviewer
- Chapter 22 — project demos, wrap-up, what to learn next

19 Buffer day; function pointers, `std::function`, callbacks

i Chapter status: stub

Mirrors L20 of the Summer 2026 lecture plan. This day is a **deliberate buffer**; the function-pointer / `std::function` content runs only if the schedule has held.

19.1 Outline

- **Buffer / catch-up day** — absorbs the inevitable overruns from earlier lectures (week 3 layered-machine and week 4 BST are the likeliest sources).
- If the schedule has held: cover **function pointers + `std::function`**
 - **callbacks** as a compact lesson.
- If not: this is where the slack lives.
- Ranges teaser dropped (mentioned in L23 “what to learn next” instead).

20 AI tools — retrospective and discipline

i Chapter status: stub

Mirrors L21 of the Summer 2026 lecture plan. This is a **retrospective**, not an introduction — AI has been part of the course since L1 / Lab 1.

20.1 Outline

- **Revisit the eight AI roles** now that students have used them for five weeks:
 1. Pair programmer
 2. Coach
 3. IT aide
 4. Reviewer
 5. Test generator
 6. Spec clarifier
 7. Rubber duck
 8. Translator
- What worked
- **When *not* to use AI**
- AI policy and disclosure recap
- **Security through-line landing** (per thread #7):
 - Claude Code’s permission prompts as the model for student practice — pause before risky ops, name what could go wrong, then proceed
 - Brief recap of blast-radius, grant-of-execution, and trust-by-default
 - **xz-utils** as the cautionary example
 - What changes when AI generates the code you are about to run

21 Code review as a practice

i Chapter status: stub

Mirrors L22 of the Summer 2026 lecture plan.

21.1 Outline

- **Code review as a practice** — what to look for:
 - Correctness
 - Clarity
 - API
 - Tests
 - Invariants
 - Naming
- How to write actionable comments
- The “approve / request changes / comment” decision
- **AI as code reviewer** (role #4 from Chapter 20) — what it catches, what it misses, how to use it without being captured by it
- Live walkthrough reviewing student project code
- Common-mistakes gallery rides along (see Appendix F)
- Second-half project work time

22 Project demos, wrap-up, what to learn next

i Chapter status: stub

Mirrors L23 of the Summer 2026 lecture plan.

22.1 Outline

- Project demos
- Wrap-up — the threads, named
- **What to learn next:**
 - Full C++20 / C++23 modules in earnest
 - Coroutines
 - Ranges (deferred from the original L20 teaser)
 - Concurrency: threads, atomics, `std::jthread`
 - Build systems beyond the basic CMake of Appendix [A](#)
 - Reading suggestions

A Build systems: g++, make, and cmake

i Appendix status: stub

The build-system reference. Companion to Chapter 4, Chapter 13, and the Lab 2 / Lab 4 mechanics.

A.1 Outline

- The bare-metal `g++` command — what each flag does
- `make` and a minimal `Makefile` (the kind you can read at a glance)
- Why we move to `cmake` once a project crosses ~3 files
- A working `CMakeLists.txt` for a course-sized project
- `target_compile_features(... cxx_std_20)`, `target_compile_options`, `target_link_libraries`
- Out-of-source builds (`cmake -B build && cmake --build build`)
- Integrating `clang-format`, `clang-tidy`, and the sanitizers
- `compile_commands.json` and what your editor's LSP does with it

B Sanitizers and clang-tidy workflow

i Appendix status: stub

Companion to Chapter 17.

B.1 Outline

- What a sanitizer is and why you would use one
- **AddressSanitizer (ASan)** — out-of-bounds, use-after-free, leaks
- **UndefinedBehaviorSanitizer (UBSan)** — signed overflow, null deref, etc.
- **ThreadSanitizer (TSan)** — when you need it, when you do not
- Build flags and the cost of running with sanitizers on
- The course `.clang-tidy` and what its checks mean
- Reading and acting on a sanitizer report
- The `clang-tidy` autofix workflow

C GDB reference

i Appendix status: stub

Companion to Chapter 7, Chapter 8, Chapter 18.

C.1 Outline

- Starting GDB on a course-sized program (`g++ -g, gdb ./prog`)
- The command alphabet: `run`, `break`, `continue`, `next`, `step`, `finish`, `list`, `print`, `bt`, `frame`, `info args`, `info locals`
- Watchpoints
- Conditional breakpoints
- Memory exam (`x/16xw`)
- TUI mode (`Ctrl-X A`)
- Reading a stack trace
- Common gotchas: optimized builds, missing debug info, inline functions

D Git and the GitHub team workflow

i Appendix status: stub

The Lab 4 spine.

D.1 Outline

- What git tracks and what it does not
- The three-area model: working tree, index, history
- Daily commands: `status`, `diff`, `add`, `commit`, `log`
- Branches: `branch`, `checkout` / `switch`, `merge`
- Remotes and `push` / `pull`
- The GitHub team workflow: feature branches, pull requests, reviews
- Resolving merge conflicts without destroying anyone's work
- `.gitignore` for C++ projects (and why)
- Recovering from common mistakes (`reset`, `revert`, `reflog`)

E Unit testing in C++ with Catch2 / doctest

i Appendix status: stub

Companion to Chapter 17 and Chapter 21.

E.1 Outline

- Why unit tests matter even more in a compiled language
- Catch2 vs doctest — same shape, different tradeoffs
- The header-only setup
- `TEST_CASE`, `SECTION`, `REQUIRE`, `CHECK`
- Parameterized tests
- Floating-point tolerances
- Tests as documentation
- Wiring tests into your `CMakeLists.txt`
- Wiring tests into CI

F Common errors

i Appendix status: stub

Port and modernize the Common Errors material from
`prior_semesters/summer_2024/lecture_notes/ec327_ln_sum24_ch15.md`.

F.1 Outline

- Reading C++ compiler errors (the famous “template error wall”)
- Missing semicolons, missing braces, mismatched `#includes`
- Mismatched `()` and `{}` — declaration ambiguity
- Off-by-one errors in indexing and loops
- Dangling references, dangling pointers
- Lifetime errors that the sanitizers catch
- Confusing `=` and `==`
- Confusing `=`, `==`, and `<=>` (since C++20)
- Forgetting `const`
- The `using namespace std;` cluster of name clashes
- Linker errors (ODR violations, undefined references)

G C++ language summary

i Appendix status: stub

Port and modernize the Language Summary from `prior_semesters/summer_2024/lecture_notes/ec327_ln_s`
The 2024 summary covered C++11/14; this version targets C++20.

G.1 Outline

- What is a C++ program (declarations, definitions, translation units)
- Compound statements
- Expression statements
- Declaration statements
- Selection statements (`if`, `if constexpr`, `switch`)
- Iteration statements (`for`, `range-for`, `while`, `do-while`)
- Jump statements (`return`, `break`, `continue`, `goto` and why not)
- Operators and precedence (with the keyword spellings of `and` / `or` / `not`)
- Common standard-library types you will reach for first (`std::string`, `std::vector`, `std::array`, `std::optional`, `std::string_view`, `std::map`, `std::unordered_map`, `std::span`)

